Machine Learning

Final Review

Spring 2025

Instructor: Xiaodong Gu



期末考试

考试形式：开卷，可带任何纸质资料
考试语言：中文
考试题型：判断、选择、简答、计算、系统设计
注意事项：带计算器

样卷

上 海 交 通 大 学 试 卷（A 卷）

（ 2020 至 2021 学年 第 1 学期 ）

班级号_____ 学号_____ 姓名 _____

课程名称 机器学习（Machine Learning） 成绩 _____

一、判断题 （12 分）

1. （2 分）当数据的特征空间（feature space）有非线性决策边界时(nonlinear decision boundary)，逻辑回归（Logistic Regression）模型适合对其进行分类。
2. （2 分）初始状态的设置直接影响到 K-means 聚类的结果。
3. （2 分）由于不需要人工标注，强化学习通常被认为是一种无监督学习方法。
4. （2 分）当神经网络的层数增多，它会具有更强的表征学习(representation learning)能力。
5. （2 分）池化(pooling)层在卷积神经网络中的作用是降低参数量。
6. （2 分）随机梯度下降(stochastic gradient descent)这种优化方法相比批梯度下降(batch gradient descent)会更快收敛，所以它更加稳定且效果更好。

二、选择题 （18 分）

1. 以下哪个训练/验证曲线更可能体现欠拟合？
   图 A B. 图 B C. 图 C D. 图 D
2. 下列哪个优化算法引入了“动量” （momentum）概念？
   A. SGD B. RMSProp C. Adam D. Adagrad
3. 以下哪种方法不利于缓解过拟合？
   A. 数据增强 B. 增加隐藏层神经元数量 C. L2 正则化 D. Dropout
4. 下列哪个问题可以转化为线性可分问题？
   A. 非线性分类 B. 图像生成 C. 文本聚类 D. 时间序列预测
5. 下列哪个模型最适合用于处理序列预测任务？
   A. KNN B. 决策树 C. LSTM D. PCA
6. 考虑下图所示的三层卷积神经网络（感受野如图），每层卷积核大小为 3×3，stride 为 1，不使用 padding。则输出层每个像素的感受野大小是？
   A. 3×3 B. 5×5 C. 7×7 D. 9×9

三、简答题 （10 分）

1. （5 分）SVM 和 Logistic Regression 有什么相同点和不同点？
2. （5 分）从误差反向传播(Backpropagation)的角度谈谈 LSTM 相比 RNN 的优点。

三、计算题 （30 分）

1. （15 分）下表列出了一个数据集，其中每一行代表一个数据：

x_1	x_2	x_3	y
0	0	1	0
0	1	0	0
1	1	0	0
0	0	1	1
1	0	0	1
1	1	0	1
1	1	1	1

我承诺，我将严格遵守考试纪律。

承诺人：_____

题号	一	二	三 1	三 2	四 1	四 2
得分						
批阅人 (流水阅卷教师签名处)						

(a) (10 分) 用贝叶斯规则和朴素贝叶斯假设，写出 y 的后验概率表达式 $P(y|x_1, x_2, x_3)$。

(b) (5 分) 用（a）的表达式和表格中的数据，分别计算后验概率 $P(y=0|\ x_1=0,\ x_2=0,\ x_3=1)$ 和 $P(y=1\ |\ x_1=0,\ x_2=0,\ x_3=1)$，预测 $x_1=0,\ x_2=0,\ x_3=1$ 时 y 的数值。

2. （15 分）我们有一组二维数据(x, y)包含 3 个点，分别是(1, 3)，(2, 5)，(3, 7)，我们使用线性回归(linear regression)模型进行预测，限定使用批梯度下降（Batch Gradient Descend）的方法，模型参数初始值为 0，求解的迭代次数为 3 次，学习率固定为 0.1。请写出完整的参数更新过程并预测当给定一个新的数据 x_new = 4 时候，相应 y_new 的值估计为多少？

四、系统设计题（30 分）

1.（10 分）某互联网公司的“读书”部门主要为用户提供在线阅读服务。现有活跃用户 2000 万人。有在线图书 12000 套，该部门对每本书 B_i 记录了如下属性信息：（标题、简介、出版日期、定价），比如 B_{12}=（“机器学习”， “人工智能的核心技术”， “2020 年 10 月”， 52.5）。

（1）为了便于图书推广并实现利润最大化，需要对新加入的图书进行价格区间预测，拟将图书分为低价(<10 元)、中等(10~50 元)、高价(>50 元)三大类。 请设计机器学习系统实现新图书的自动价格区间预测。

（2）为了提高用户粘度，并扩大书籍受众，现对用户进行定向书籍推荐。对每个用户 U_i，系统收集了最近 50 次的阅读图书记录，用序列 $U_i=(R_{i1}, R_{i2},\ldots,R_{i50})$表示，$R_{it}\in\{B_1, \ldots, B_{12000}\}$代表该用户 U_i 在第 t 次阅读的某一本书，请设计机器学习系统对每位用户进行图书推荐，也就是预测该用户下一次最有可能阅读哪一本书。

2. (20 分) 随着自动驾驶时代的来临，AI 自动驾驶受到越来越多投资者的青睐。右图展示了一种自动驾驶的模拟场景。



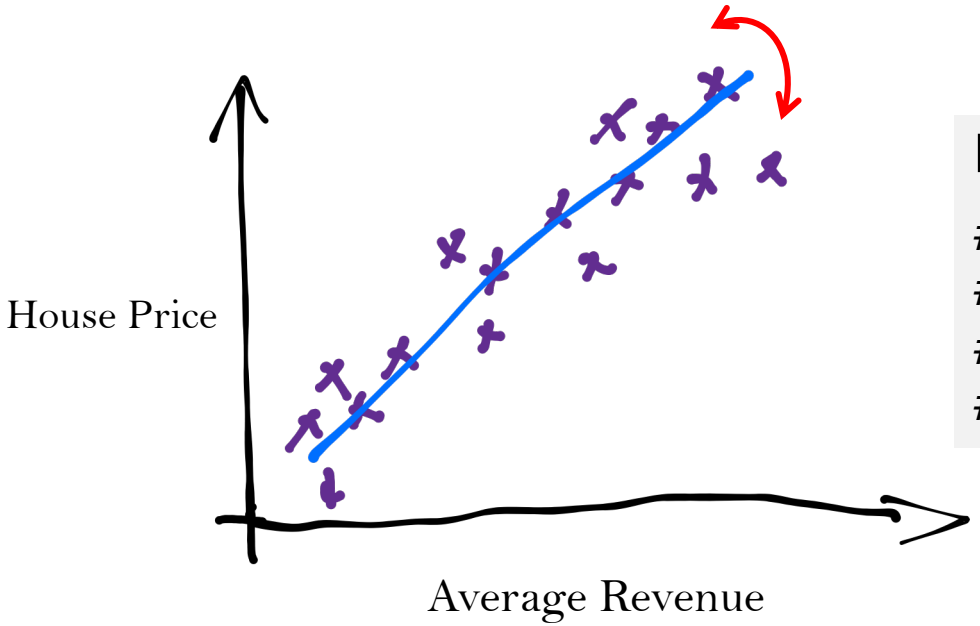
(1) 请尝试设计自动驾驶中采用深度学习进行障碍物识别的一种思路。（提示：CNN 用于图像分类，包括数据收集与处理、模型、训练、预测等）

(2) 结合强化学习的原理，谈谈应用强化学习训练自动驾驶控制系统的简要思路。注：必要时可用公式说明，但不要求严密的推导。

重要知识点回顾

What Is Machine Learning

A subset of artificial intelligence which uses **statistical methods** to enable machine to **improve** towards an **objective** using **data** without requiring explicit programming by human.



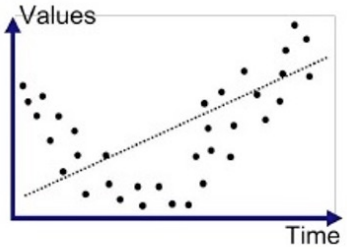
Key Elements:

#1 Data (Experience)
#2 Model (Hypothesis)
#3 Loss Function (Objective)
#4 Optimization (Improvement)

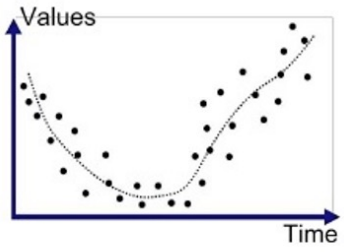
Overfitting

Underfitting – the model isn't complex enough to capture the real knowledge, the assumption may not hold.

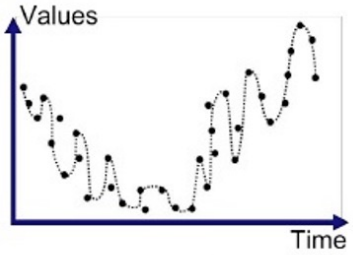
Overfitting – the model is too complex and thus describes the details of data (e.g., random noise) instead of underlying knowledge



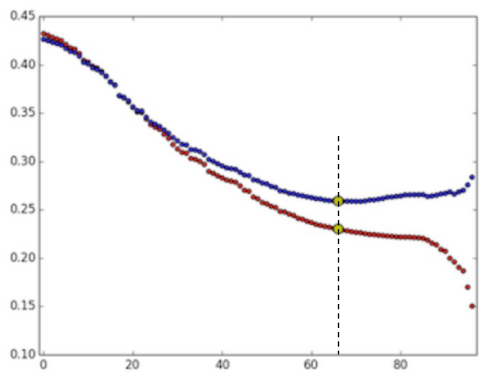
Underfitted



Good Fit/Robust



Overfitted



Prevent Overfitting

Selecting models with appropriate complexity

Widely used approaches (to mitigate overfitting)

- Increase training data
- Regularization (penalizing model complexity)
- Hold-out & cross validation (unseen data to ensure generalization)
- Early stopping
- Prior knowledge (e.g., Bayesian prior)
- …

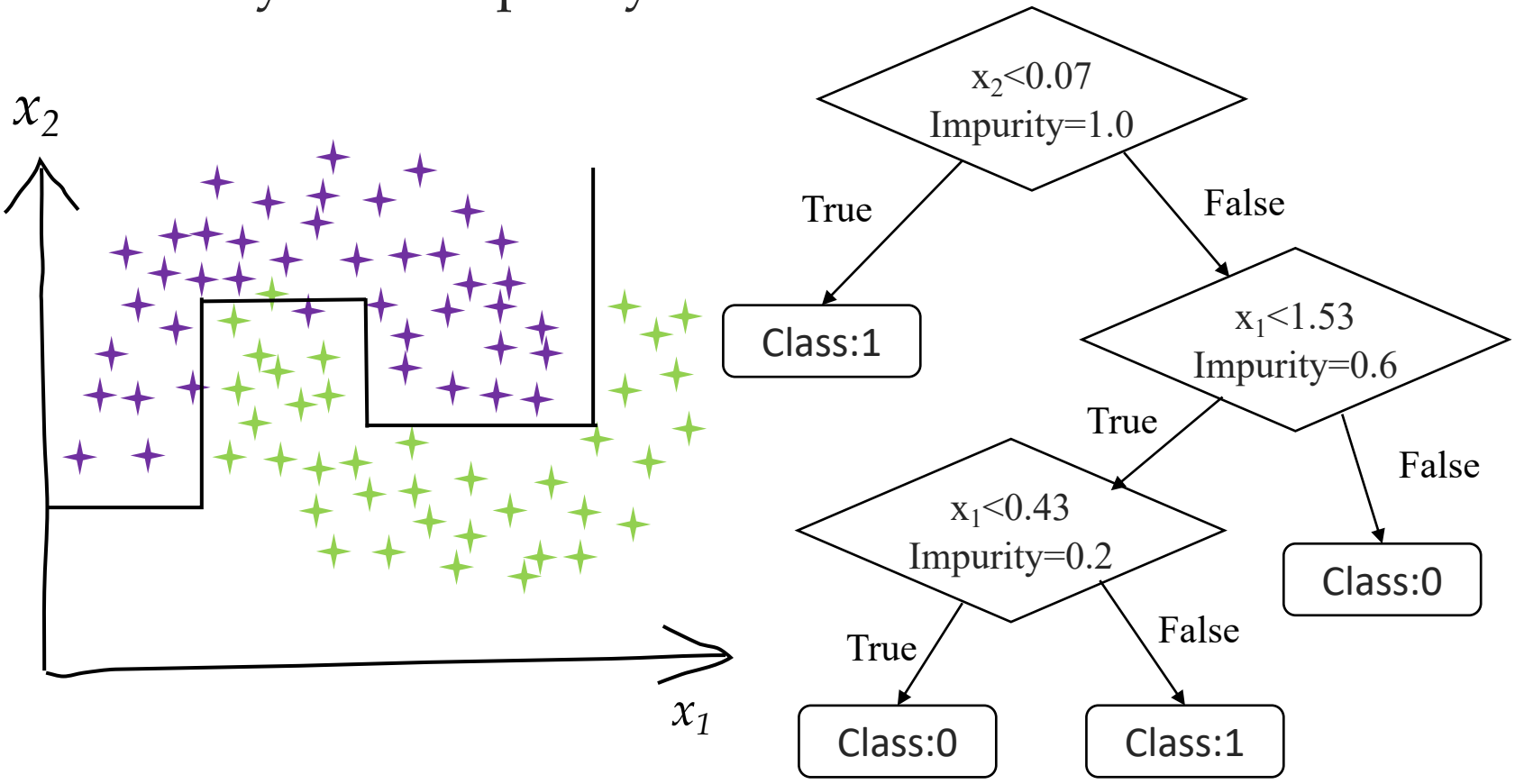
Prevent Overfitting

Fit the overall distribution instead of the training set.

- To estimate the generalization error, we need data unseen during model training.
- Data Splitting (Hold-Out):
  - Training set (e.g., 50%) — 平时练习
  - Validation set (e.g., 25%) — 模拟考试
  - Test set (e.g., 25%) — 高考!

Decision Trees

Decision by data impurity

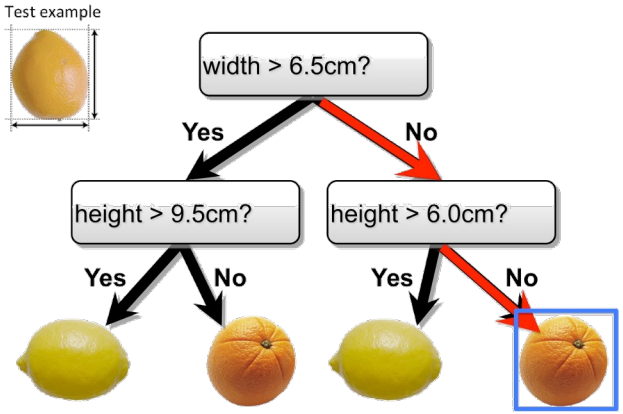


Build a Decision Tree

A top-down divide-and-conquer learning procedure

1. Construct a root node which contains the whole data set.
2. Selecting an attribute that benefits the task most according to some criterion
3. Split the examples of the current node into subsets based on values of the selected attributes.
4. Create a child node for each subset and passes the examples in the subset to the node.
5. Recursively repeat step 2~4 until some stopping criterion is met.

ID	width	height	Type
1	5.2	8.0	lemons
2	6.7	9.8	lemons
3	7.2	7.5	orang
4	6.1	5.3	orang
5	4.1	6.5	lemons



ID3

$$H(D) = -\sum_{k=1}^{K} \frac{|C_k|}{|D|} \log \frac{|C_k|}{|D|}$$

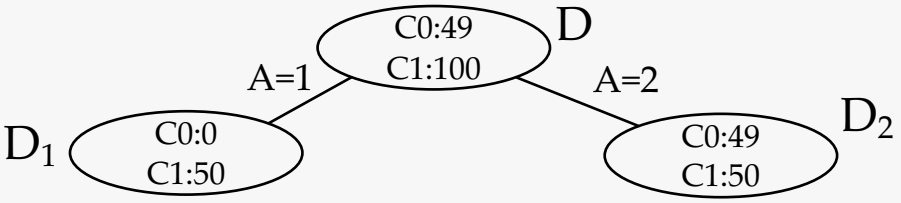
($|C_k|$: # samples of class C_k in dataset D)

$$H(D|A) = \sum_{i=1}^{n} \frac{|D_i|}{|D|} H(D_i) = -\sum_{i=1}^{n} \frac{|D_i|}{|D|} \left(\sum_{k=1}^{K} \frac{|D_{ik}|}{|D_i|} \log \frac{|D_{ik}|}{|D_i|} \right)$$

($|D_i|$: # samples whose attribute A is set to the i-th value in D; $|D_{ik}|$: #samples of class C_k in D_i)

$$\text{Gain}(D,A) = H(D) - H(D|A)$$

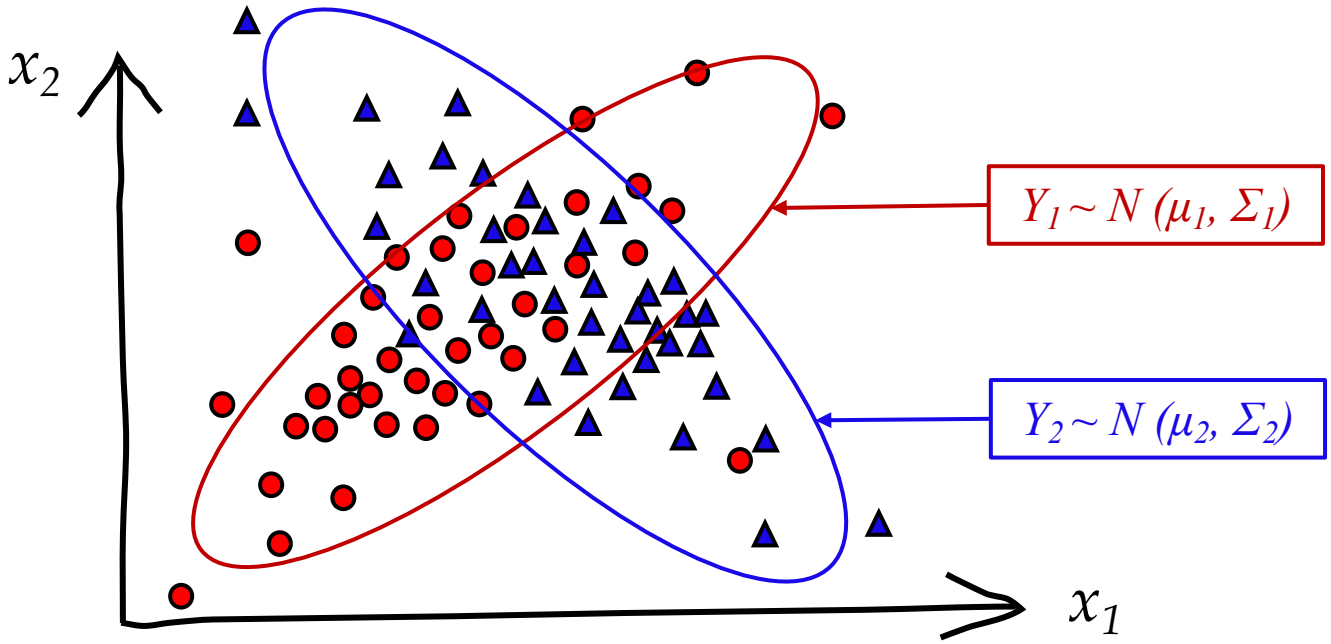
Example: What is the information gain of this split?



- root entropy: $H(D) = -\frac{49}{149}\log(\frac{49}{149}) - \frac{100}{149}\log(\frac{100}{149}) \approx 0.91$
- leaves entropy: $H(D|A=1) = 0,\ H(D|A=2) \approx 1$
- IG(D|A) $\approx 0.91 - (\frac{1}{3} \cdot 0 + \frac{2}{3} \cdot 1) \approx 0.24 > 0$

Other perspectives on the data?

What if the data attributes look like this?



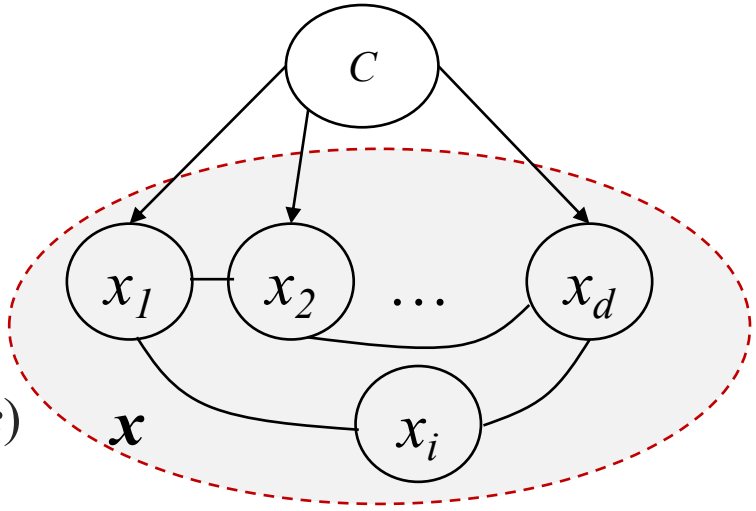
- Can not be separated linearly or recursively)
- But have a clear pattern of probabilistic distribution and dependence

Bayes Rule for multiple dimensions

X_1: address	X_2: topic	X_3: length	X_4:	C: class
Irregular	CS	<25	…	spam
Regular	Finance	>25	…	normal
…	…	…	…	…
Irregular	CS	<25	…	spam

- The most probable class $c \in C$:

$$c_{\text{MAP}} = \arg\max_{c \in C} P(c|x)$$
$$= \arg\max_{c \in C} \frac{P(x|c)P(c)}{P(x)}$$
$$= \arg\max_{c \in C} P(x|c)P(c)$$
$$= \arg\max_{c \in C} p(x_1, x_2, \ldots, x_{|d|}|c)\, P(c)$$



Bayesian Networks for Classification

$$c_{\text{MAP}} = \arg\max_{c \in C} p(x_1, x_2, \ldots, x_d | c) P(c)$$

Question

How to estimate $P(c)$ and $P(x_1, x_2, \ldots, x_d | c)$?

- $\hat{P}(c) \leftarrow \dfrac{\text{count}(\text{C=c})}{N}$

n_c: # of training samples for which C=c
N: total # of training samples

- $\hat{P}(x_1, x_2, \ldots, x_d | c) \leftarrow$



too complicated

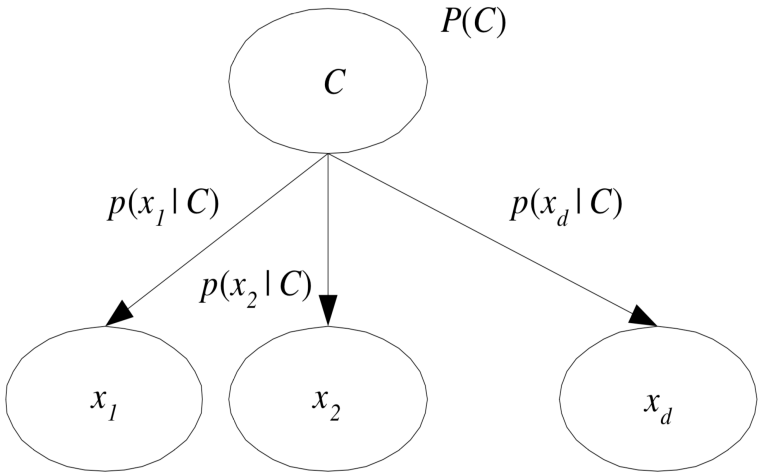
of parameters in the joint probability is too huge

x_1	x_2	…	x_d	p
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…

Naïve Bayes Independent Assumption

Conditional Independence: assume the input features x_j are independent given the class c

$$P(x_1,\ldots,x_d\,|c) = P(x_1|c)P(x_2|x_1,c)\ldots P(x_d|x_1,\ldots x_{d-1},c)$$
$$\approx P(x_1|c)\bullet P(x_2|c)\bullet P(x_3|c)\bullet \ \ldots\ \bullet P(x_d\,|c)$$



$$c_{\text{MAP}} = \arg\max_{c\in C} p(x_1, x_2, \ldots, x_d|c)P(c)$$

$$c_{\text{NB}} = \arg\max_{c\in C} P(c)\prod_{i=1}^{d} p(x_i|c)$$

Naïve Bayes

$$c_{\text{NB}} = \arg\max_{c \in C} P(c) \prod_{i=1}^{d} p(x_i|c)$$

- Training amounts to estimating parameters: $P(c)$'s, $\text{P}(x_1|c), \ldots, \text{P}(x_\text{d}|c)$ from data.

Q: How to estimate each P(c)?

– Straightforward

Q: How to estimate $\text{P}(x_i|c)$ for each c?

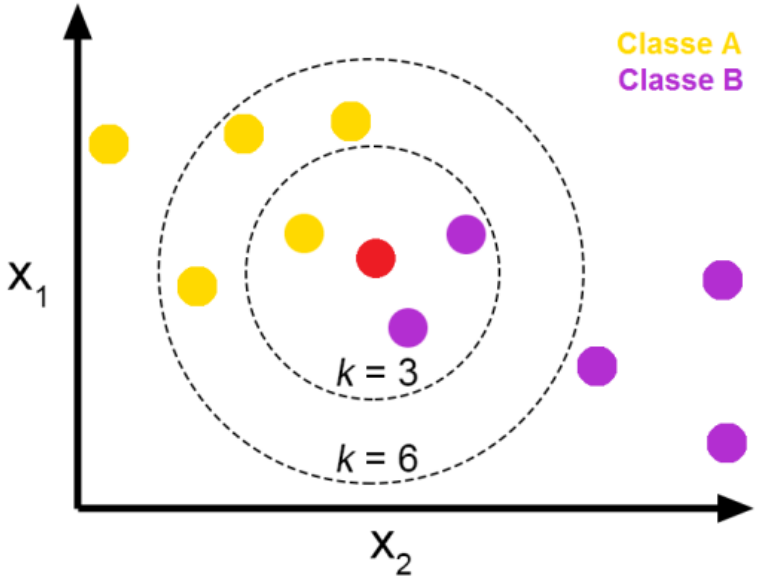
$$\widehat{P}(x_i|c) \leftarrow \frac{\text{count}\,(x_i, c)}{\sum_{x \in |x|} \text{count}\,(x, c)}$$

training samples for which C=c and x = x_i

training samples for which C=c

K-NN Classification

- Classified by "MAJORITY VOTES" from neighbor classes.
- An object is classified to the most common class amongst its *k* nearest neighbors **measured by a "distance" function**





How to determine whether an object falls into the k-nearest neighbors?

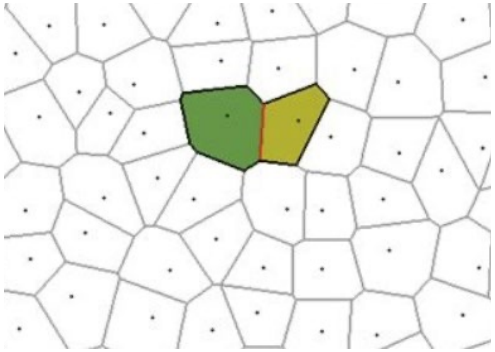
Decision Boundary

Voronoi Tessellation (维诺图，沃罗诺伊分割)

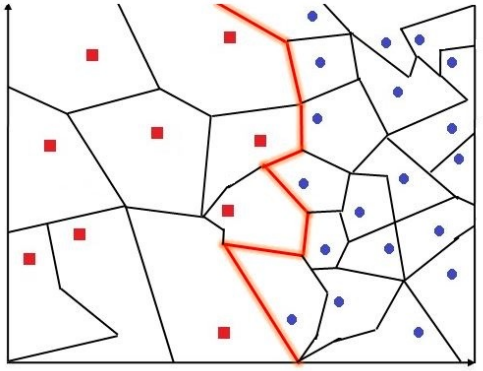
- Partition the space into areas that are nearest to any given point
- Boundary: points at the same distance from two different training examples.

Decision Boundary: boundaries that separates two different classes.

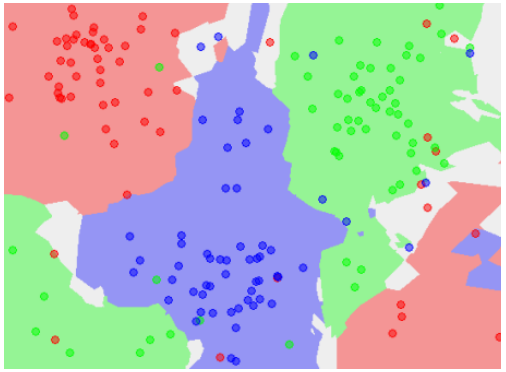
K=1



K=1



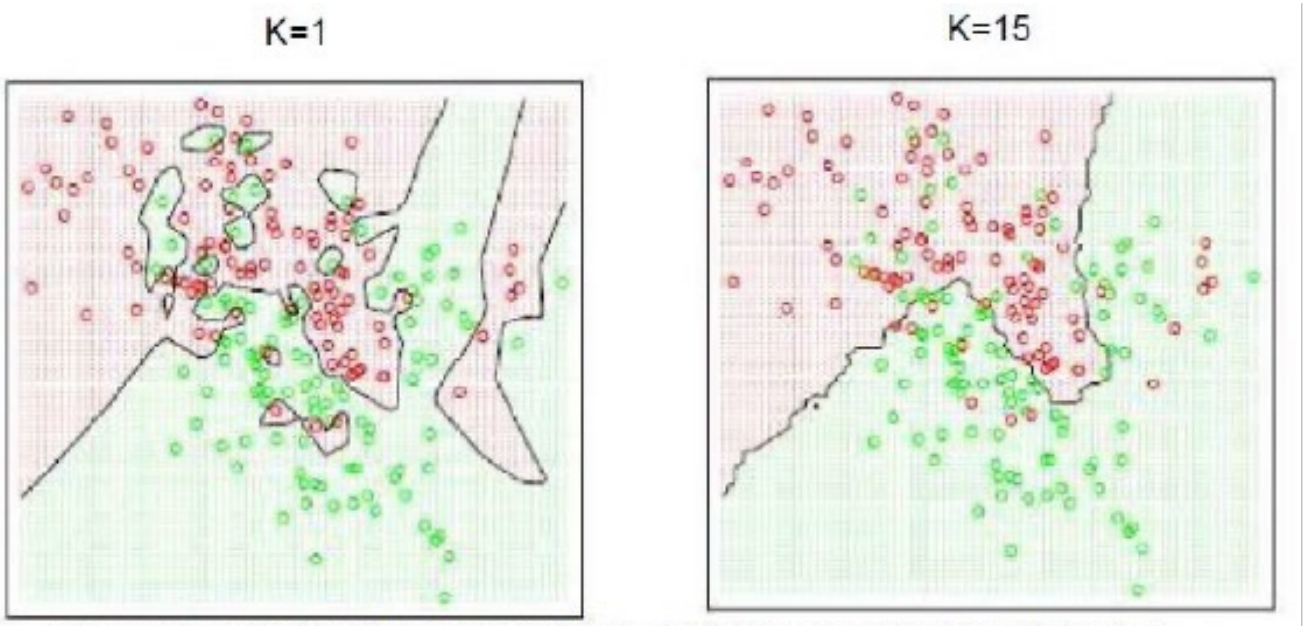
K>1



With large number of examples and possible noise in the labels, the decision boundary can become nasty!

Effect of K

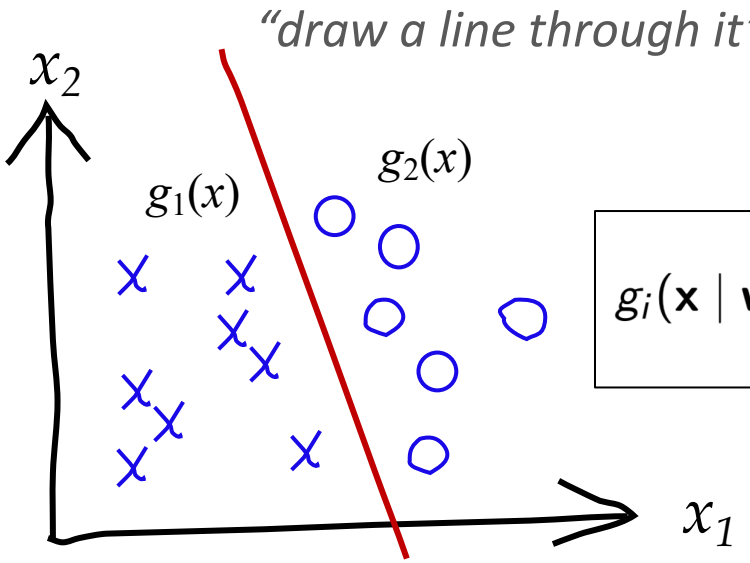
- Larger *K* produces smoother boundary effect
- When *K*==*N*, always predict the majority class



Figures from Hastie, Tibshirani and Friedman (Elements of Statistical Learning)

Linear Discriminant Functions

- A linear function to model the class affiliation of data.



$$g_i(\mathbf{x} \mid \mathbf{w}_i, w_{i0}) = \mathbf{w}_i^T \mathbf{x} + w_{i0} = \sum_{j=1}^{d} w_{ij} x_j + w_{i0}$$

Decide C_1 if $g_1(x) > g_2(x)$ else C_2.

Advantages:

- simplicity: $O(d)$ time and space complexity;
- understandability: final output is a weighted sum of attributes;
- accuracy: quite accurate if some assumptions are satisfied.

Linear Classification with Uncertainty

- What is the posterior probability of choosing C1 (or C2)?

Let

model output

$$P(C_1 \mid x) = y \qquad P(C_2 \mid x) = 1 - y$$

Classification rule:

$$\text{Choose} \begin{cases} C_1 & \text{if } y > 0.5 \\ C_2 & \text{otherwise} \end{cases}$$



What is the relationship between y and x?

Equivalent Rule:

$$\frac{y}{1-y} > 1 \qquad \text{or} \qquad \log \frac{y}{1-y} > 0$$

odds of y

Log odds (logit function) of y



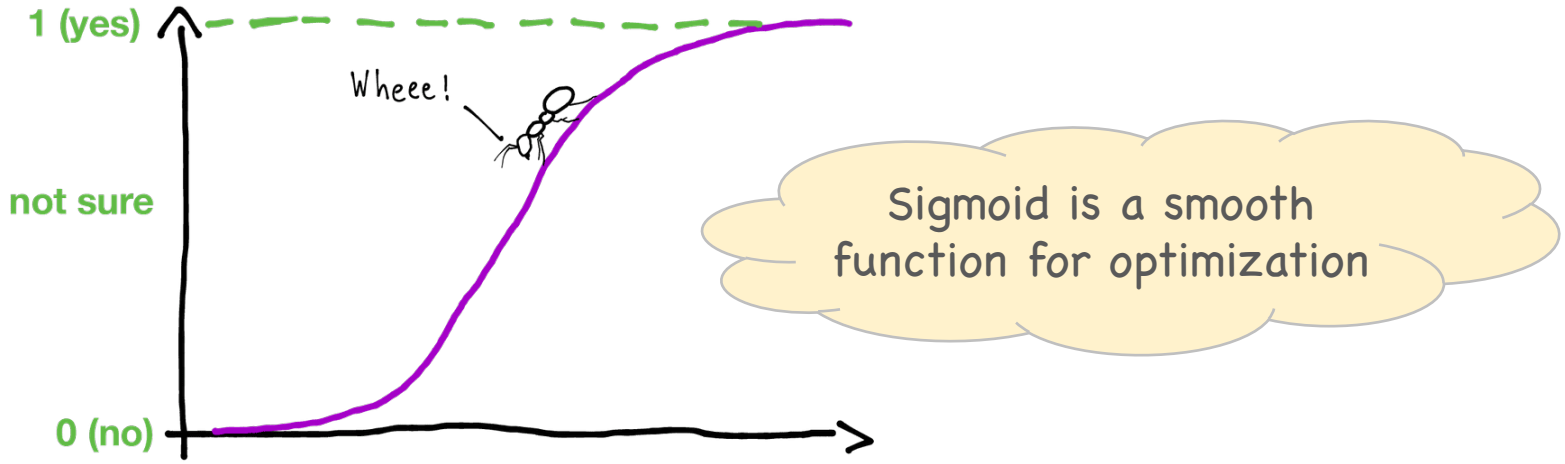
What is the relationship between log(y/1-y) and x?

The Sigmoid Function

$$\log \frac{y}{1-y} = \mathbf{w}^{\mathrm{T}} x + w_0$$

$$\Rightarrow \quad y = \text{sigmoid}(\mathbf{w}^T \mathbf{x} + w_0) = \frac{1}{1 + \exp[-(\mathbf{w}^T \mathbf{x} + w_0)]}$$

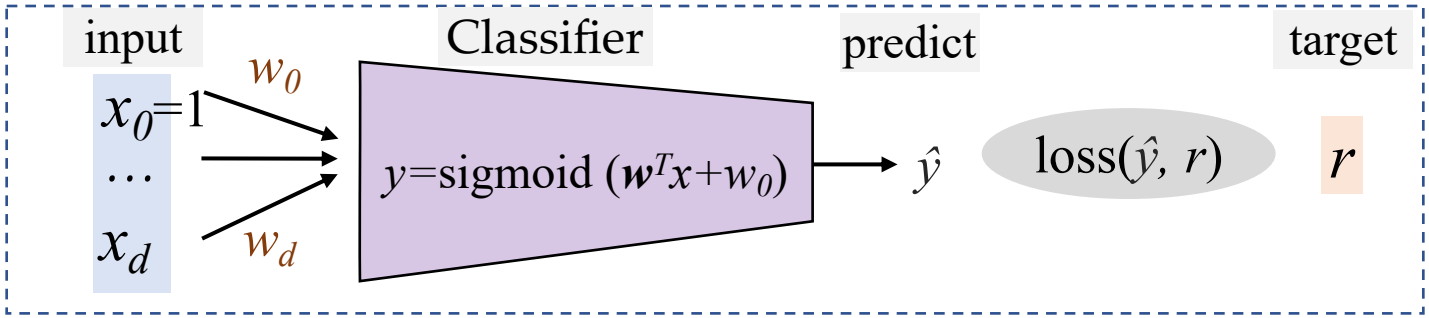
- The sigmoid function (or logistic function) is the inverse function of logit, which directly computes the posterior class probability $P(C_1|x)$.



Logistic Regression

- A classifier that estimates the decision boundary as a logistic function:

$$y = \text{sigmoid}\,(w^{\mathrm{T}}x + w_0) = \frac{1}{1+\exp[-(w^{T}x+w_0)]}$$



Loss Function

$y \equiv \mathrm{p}(C_1|x) = \mathrm{sigmoid}(w^{\mathrm{T}}x+w_0)$ is the estimated posterior probability of $x \in C_1$.

- For a given input x, the model outputs a probability y of $x \in C_1$. Let $r \in \{0, 1\}$ be the label of the real class ($r = 1$: $x \in C_1$, $r = 0$: $x \in C_2$):
  - if $r = 1$: we aim to maximize $\log p(C_1|x) = \log y$, cost is $-\log y$
  - if $r = 0$: we aim to maximize $\log p(C_2|x) = \log (1-y)$, cost is $-\log (1-y)$
- Can writhe this succinctly as a cross-entropy loss:

$$\ell(w, w_0 | x, r) = -r \log y - (1-r) \log (1-y)$$

nonzero only if r=1 nonzero only if r=0

$$\mathrm{CE}(\mathrm{p}, \mathrm{q}) = \mathbb{E}_{x\sim p(x)}\left\{\log \frac{1}{q(x)}\right\} = -\sum_{x\in X} p(x)\log_2 q(x)$$

- Equivalent to maximize the likelihood:

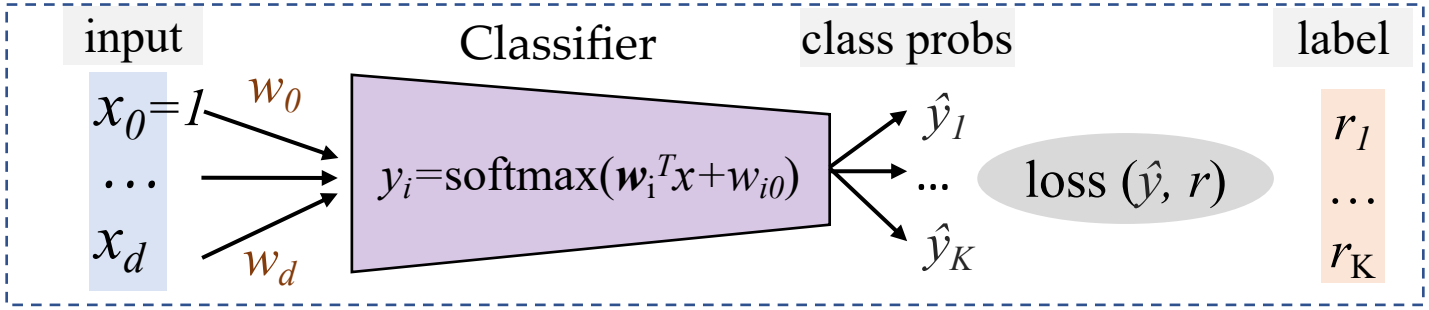
$$r | x \sim \text{Bernoulli}(y)$$

$$p(r|x) = y^{\mathrm{r}} (1-y)^{(1-r)} = \begin{cases} y & \text{if } r=1 \\ 1-y & \text{if } r=0 \end{cases}$$

Softmax Regression

- A classifier that estimates the decision boundaries as Softmax functions:

$$y_i = \text{softmax}\left(w_i^T x + w_{i0}\right) = \frac{\exp\left[w_i^T x + w_{i0}\right]}{\sum_{j=1}^{K} \exp\left[w_j^T x + w_{j0}\right]} \quad (i = 1,..,K)$$



- Train:
  - estimate the parameters $\boldsymbol{w}_i$ and w_{i0} $(i=1{:}K)$ from data
- Test:
  - calculate $y_i = \text{softmax}\left(\boldsymbol{w}_i^T x + w_0\right)$ and choose C_i if $y_i = \max\{y_{1:K}\}$ (y can be interpreted as a posterior probability).

Loss Function

$$\boldsymbol{r} = \underbrace{(0, \ldots, 0, 1, 0, \ldots, 0)}_{\text{entry } k \text{ is } 1}$$



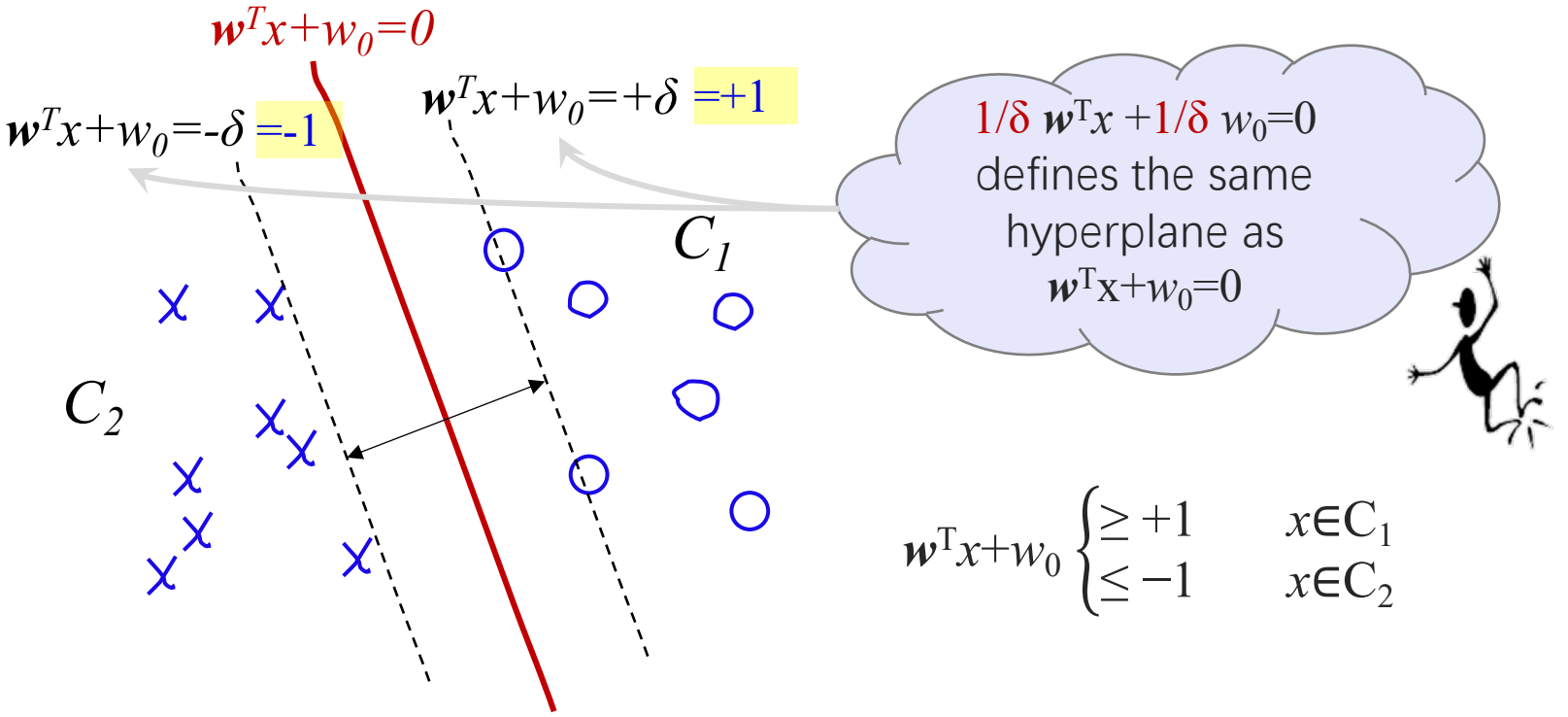
- For a given input x, the model outputs a vector of class probabilities $y = (y_1, \ldots y_K)$, and the label of target class is a one-hot vector $r = (r_1, \ldots, r_K)$ $(r_i{=}1{:}x \in C_i, r_i{=}0{:}\ x \notin C_i)$
  - if $r_1 = 1$: we aim to maximize $\log p(C_1 | x) = \log y_1$, cost is $-\log y_1$
  - if $r_2 = 1$: we aim to maximize $\log p(C_2 | x) = \log y_2$, cost is $-\log y_2$
  - ……
- We can write this succinctly as a cross-entropy loss function:

$$L_{\text{CE}}(\boldsymbol{y}, \boldsymbol{r}) = -\sum_{i=1}^{K} r_i \log y_i = -\boldsymbol{r}^T(\log \boldsymbol{y})$$

where the *log* is applied elementwise.

Maximize Margin?

- A linear discriminant function models a linear decision boundary of two classes.



Maximizing the Margin

$$\boldsymbol{w}^{\mathrm{T}}x+w_0 = \begin{cases} 1 & \text{for the closest points on one side} \\ -1 & \text{for the closest points on the other} \end{cases}$$

- Let $x^{(1)}$ and $x^{(2)}$ be two closest points on each side of the hyperplane.
- Note that

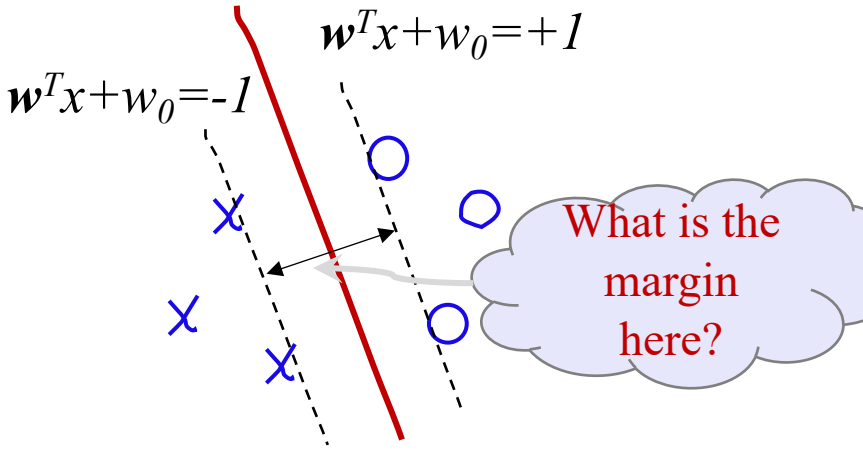
$$\boldsymbol{w}^{\mathrm{T}}\, x^{(1)} + w_0 = +1$$
$$\boldsymbol{w}^{\mathrm{T}}\, x^{(2)} + w_0 = -1$$

which imply

$$\boldsymbol{w}^{\mathrm{T}}(x^{(1)} - x^{(2)}) = 2.$$

Hence, the margin can be given by

$$\text{margin} = \frac{\boldsymbol{w}^{\mathrm{T}}(x^{(1)} - x^{(2)})}{\|\boldsymbol{w}\|} = \frac{2}{\|\boldsymbol{w}\|}$$

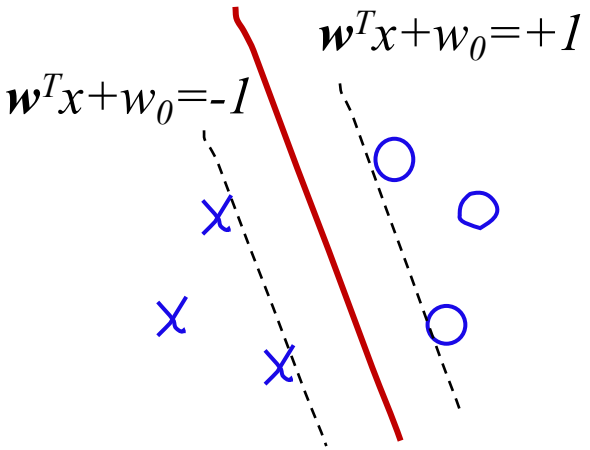


Maximizing the margin is equivalent to minimizing $\frac{1}{2}\|\boldsymbol{w}\|$

Inequality Constraints

- Given: $D = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(N)}, y^{(N)})\}$, we want w and w_0 to satisfy

$$w^T x^{(\ell)} + w_0 \begin{cases} \geq +1 & \text{if } y^{(\ell)} = +1 \\ \leq -1 & \text{if } y^{(\ell)} = -1 \end{cases}$$



- Or, equivalently,

$$y^{(\ell)} (w^T x^{(\ell)} + w_0) \geq 1$$

SVM = Solving an Optimization Problem

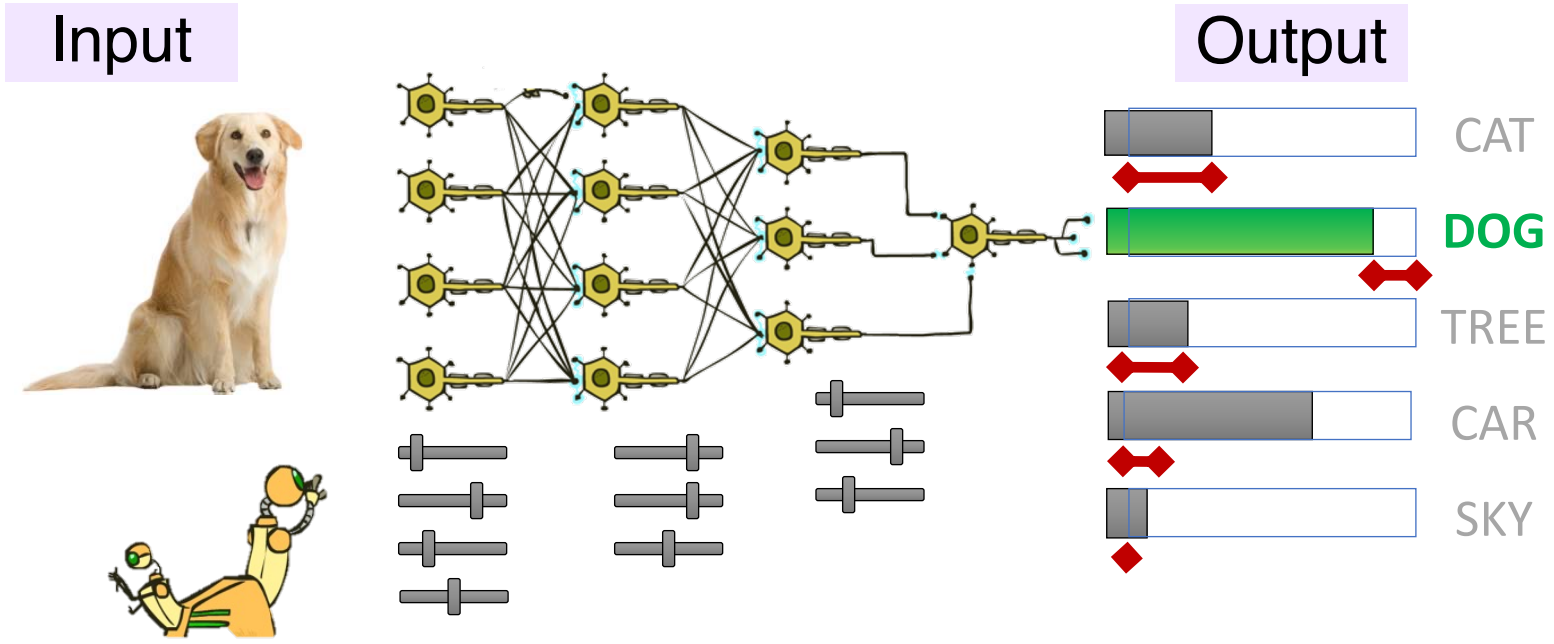
In summary, SVM aims to solve a constrained optimization problem:

$$\begin{aligned} &\text{minimize} && \frac{1}{2}\|\boldsymbol{w}\|^2 \\ &\text{subject to} && y^{(\ell)}(\boldsymbol{w}^{\mathrm{T}}x^{(\ell)}+w_0)\geq 1,\ \ell=1,\ldots,\mathrm{N} \end{aligned}$$

- This is a quadratic programming (QP) problem, which is one type of convex optimization problem.
- The complexity depends on the dimensionality d of inputs

Artificial Neural Networks

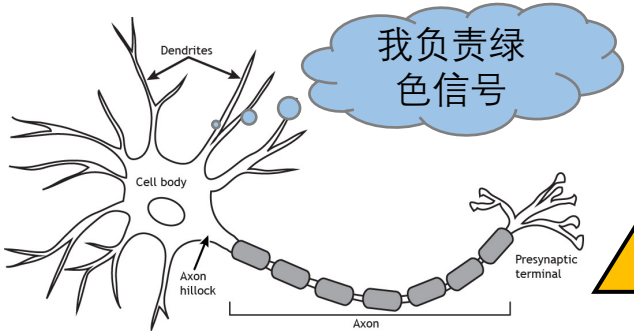
- **Artificial Neural Networks (ANN)** mimic some characteristics of the human brain, especially with regard to the computational aspects.



Perceptron (感知机) [Frank Rosenblatt, 1957]

- Basic modeling of the "Neuron".





我负责绿色信号

报告大脑,发现目标!

1	0	9	0	0
3	2	5	0	2
5	5	0	8	0
0	4	6	0	1
0	0	3	0	0

x_1 x_2 x_3 x_4 w_0 w_1 w_2 w_d

$a = \sum w_j x_j$ $y = \sigma(a)$

z $\sum$ σ y

输入函数 激活函数 输出

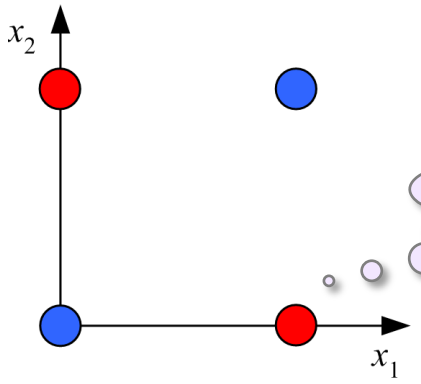
$y = \begin{cases} 0 & \text{没有绿色} \\ 1 & \text{发现绿色} \end{cases}$

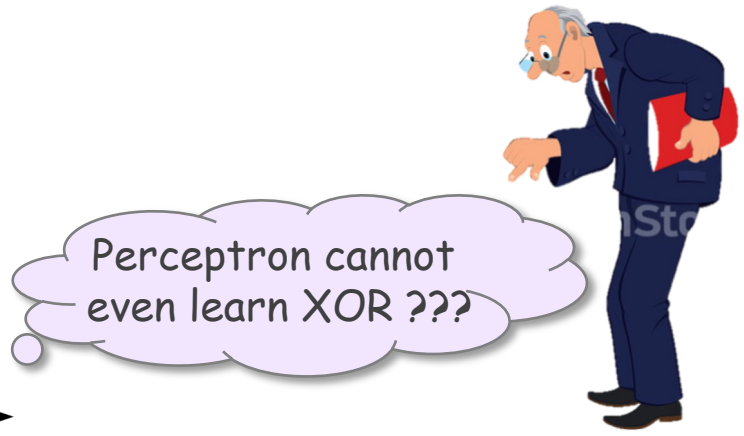
通过计算我也能识别绿色

Perceptron and XOR

- However, Minsky and Papert [1969] proved that some rather elementary computations, such as the XOR problem, could not be done by Rosenblatt's one-layer perceptron.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



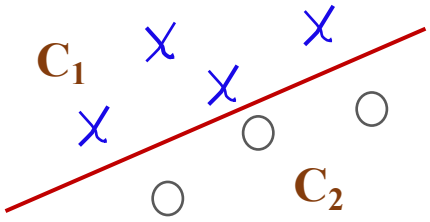


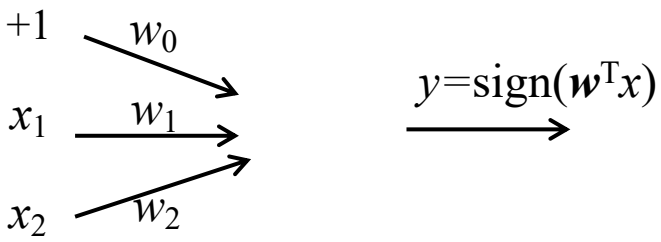
Limitation1: perceptron cannot learn data that are not linearly separable

The Magic of Hidden Layers !

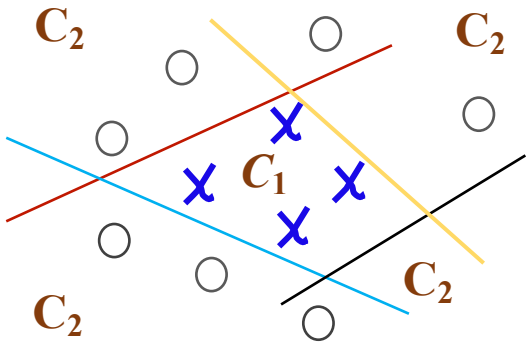


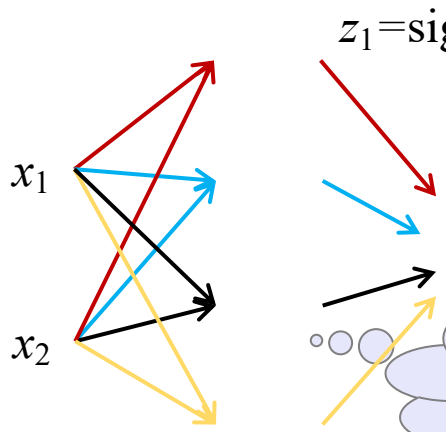
- But, adding hidden layer(s) (internal presentation) allows for learning a mapping that is not constrained by linear separability.





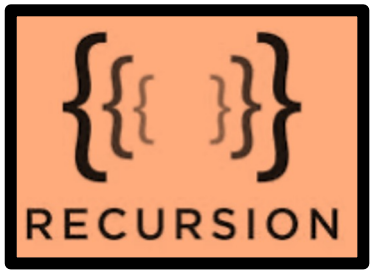
decision boundary: $x_1w_1+x_2w_2+w_0=0$

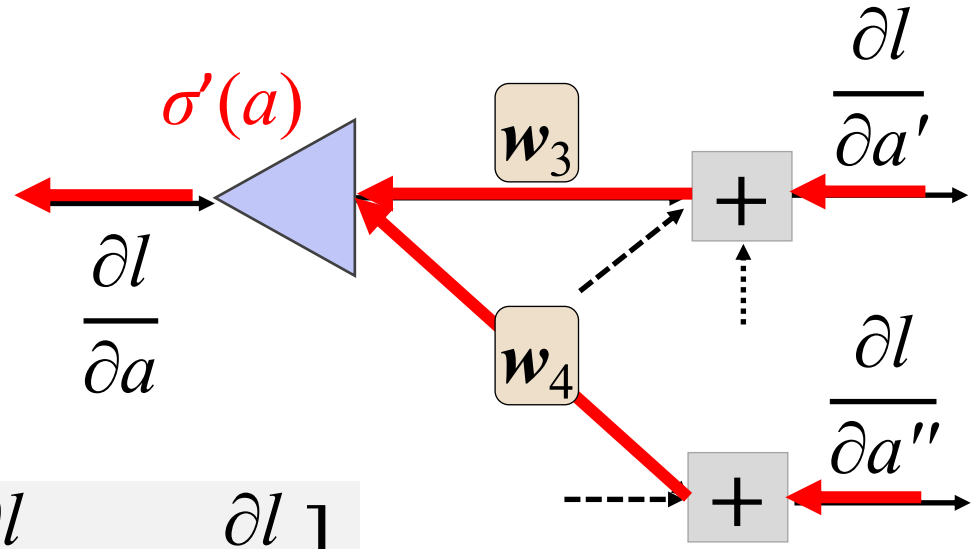




Each hidden unit is responsible for one of these linear boundaries

Backpropagation – Backward pass





$$\frac{\partial l}{\partial a} = \sigma'(a)\left[w_3 \frac{\partial l}{\partial a'} + w_4 \frac{\partial l}{\partial a''}\right]$$

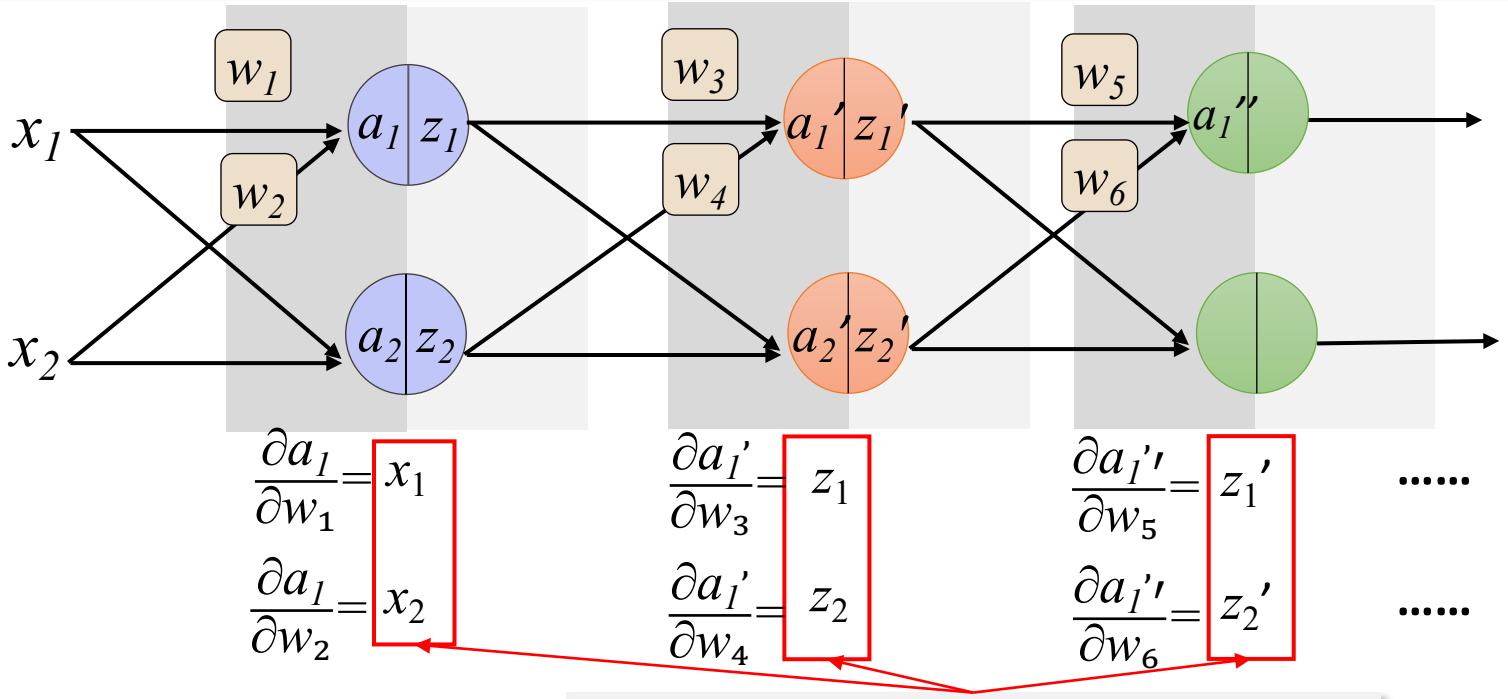
constant because a has already been determined in the forward pass.

Backward pass:

compute $\partial l / \partial a$ for all linearity outputs a recursively.

Backpropagation – Forward Pass

For every w, we need to compute $\partial a/\partial w$ within each linearity unit.



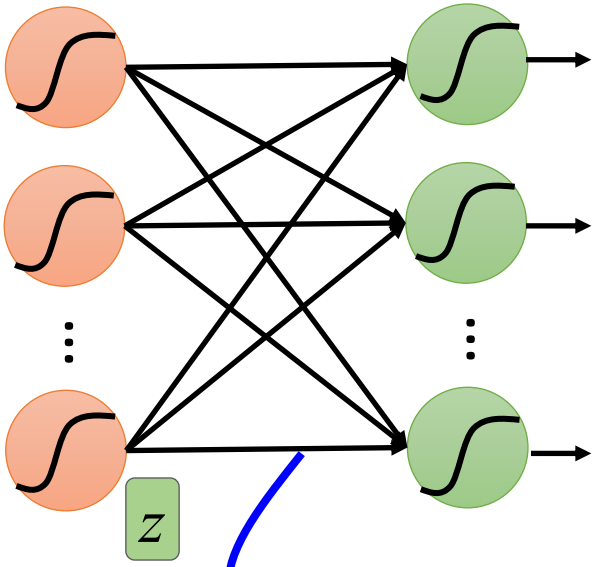
$\partial a/\partial w$ = the input connected to each w



Idea: since the linear unit for each neuron is easy and fixed, we can pre-compute their gradients $\partial a/\partial w$ straightforward.

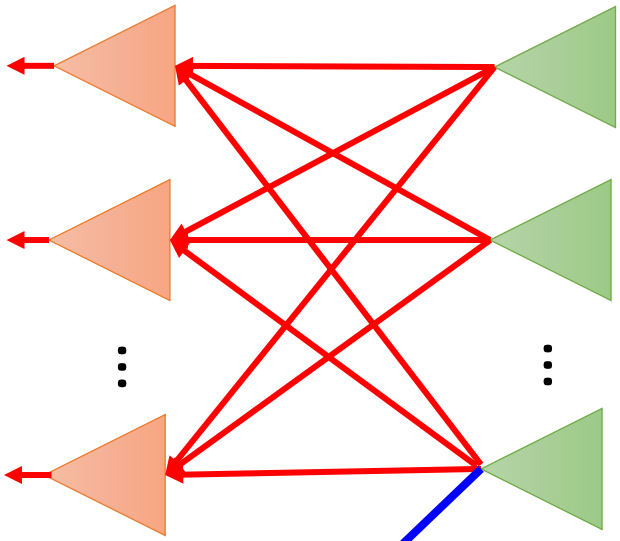
Backpropagation – Summary

Forward Pass



$$\frac{\partial a}{\partial w} = z$$

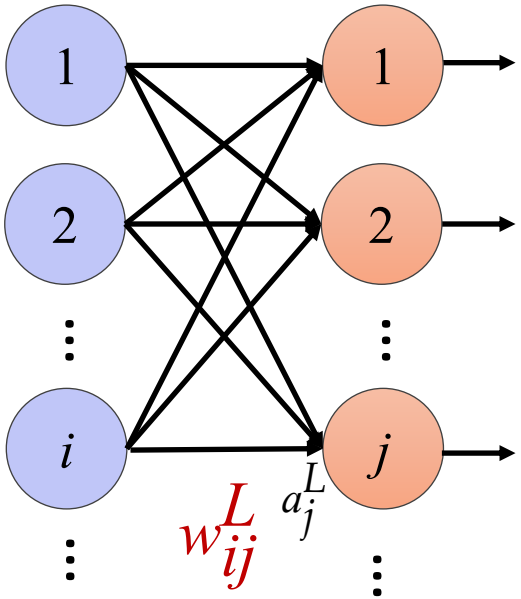
Backward Pass



$$\times \quad \frac{\partial l}{\partial a} \quad = \frac{\partial l}{\partial w}$$

for all $\boldsymbol{w}$

Perspective from Error-Propagation



$$\frac{\partial l}{\partial w_{ij}^L} = \frac{\partial a_j^l}{\partial w_{ij}^l} \frac{\partial l}{\partial a_j^L}$$

Input signal

Error signal

$$\begin{cases} z_i^{L-1} & L > 1 \\ x_i & L = 1 \end{cases}$$

$$\delta_j^L$$

Forward Pass

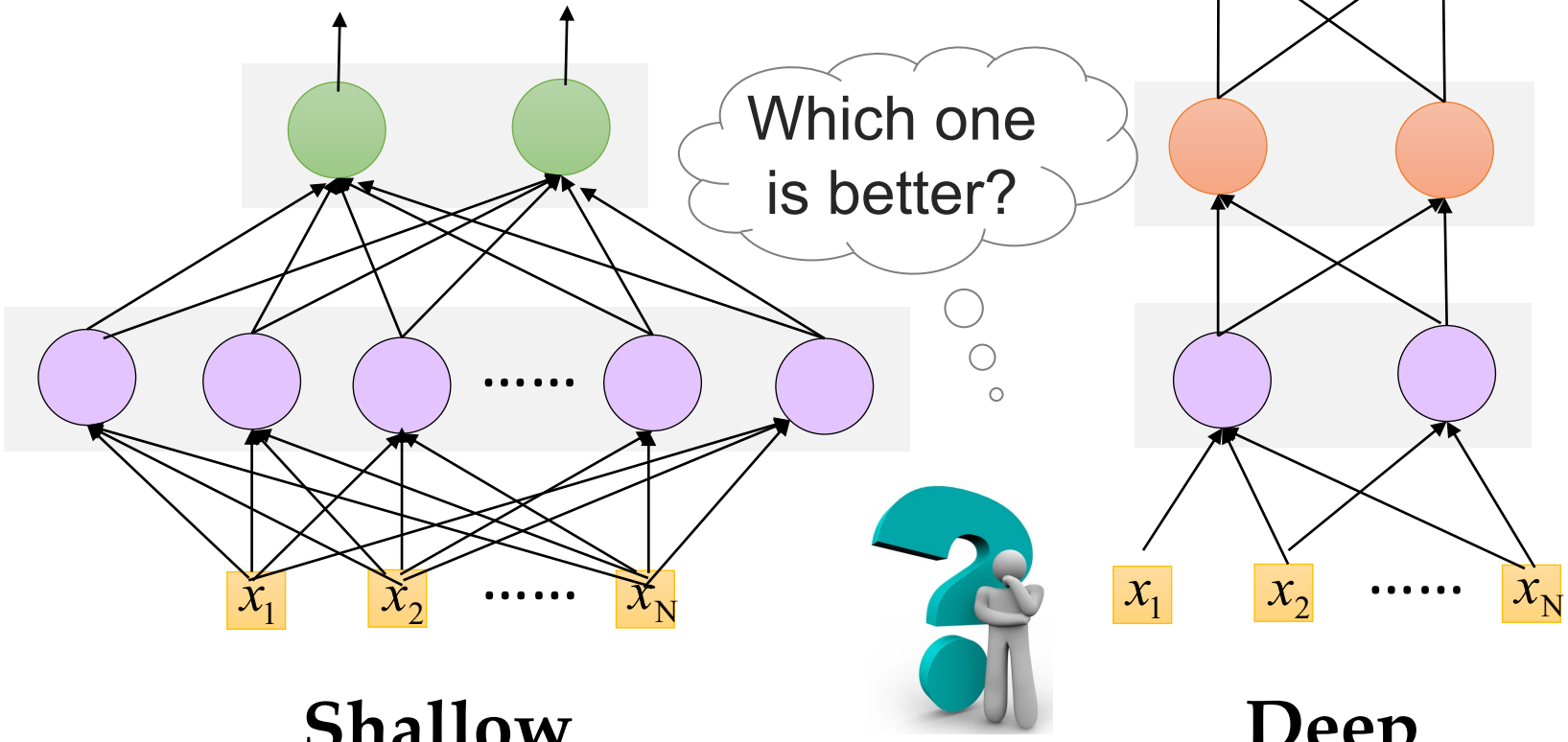
$$z^1 = W^1 x$$
$$a^1 = \sigma(z^1)$$
$$\cdots\cdots$$
$$z^{l-1} = W^{l-1} a^{l-2}$$
$$a^{l-1} = \sigma(z^{l-1})$$

Backward Pass

$$\delta^L = \sigma'(z^L) \bullet \nabla_y Loss$$
$$\delta^{L-1} = \sigma'\left(z^{L-1}\right) \bullet \left(W^L\right)^T \delta^L$$
$$\cdots\cdots$$
$$\delta^l = \sigma'\left(z^l\right) \bullet \left(W^{l+1}\right)^T \delta^{l+1}$$
$$\cdots\cdots$$

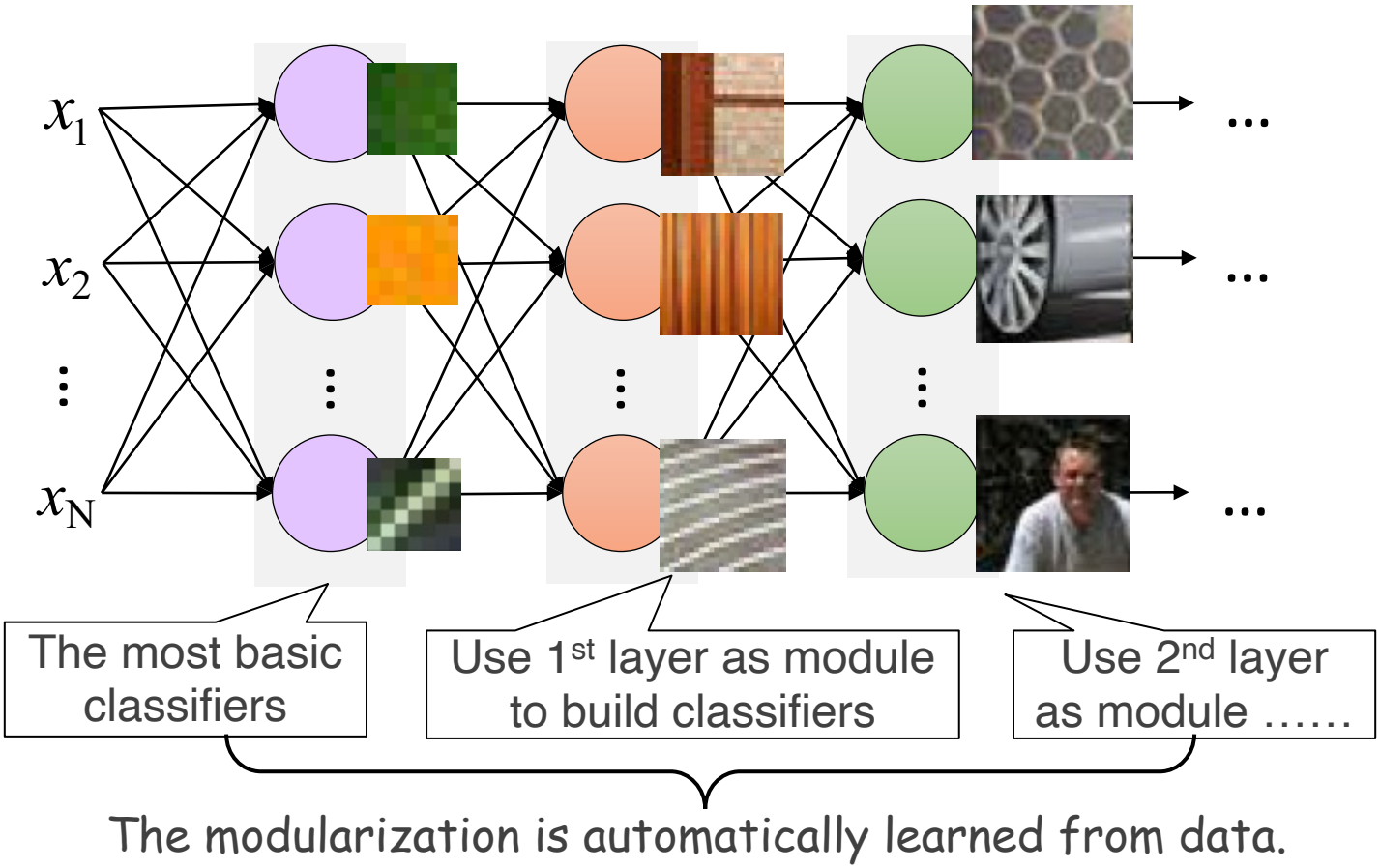
The Deeper, The Better?

The same number of parameters



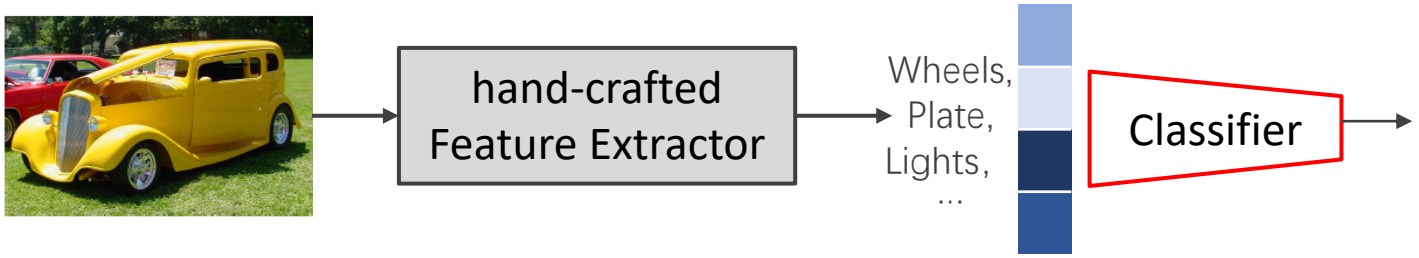
Modularization - Image

- Deep → Modularization → **Less training data?**

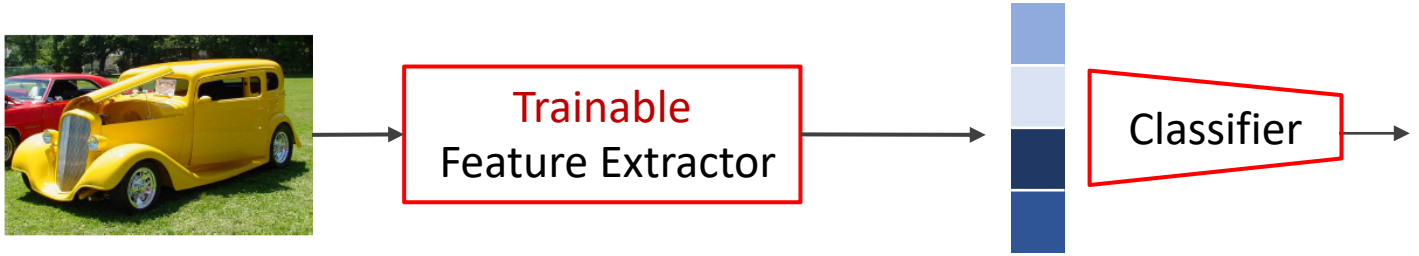


DL = Learning Feature Representations

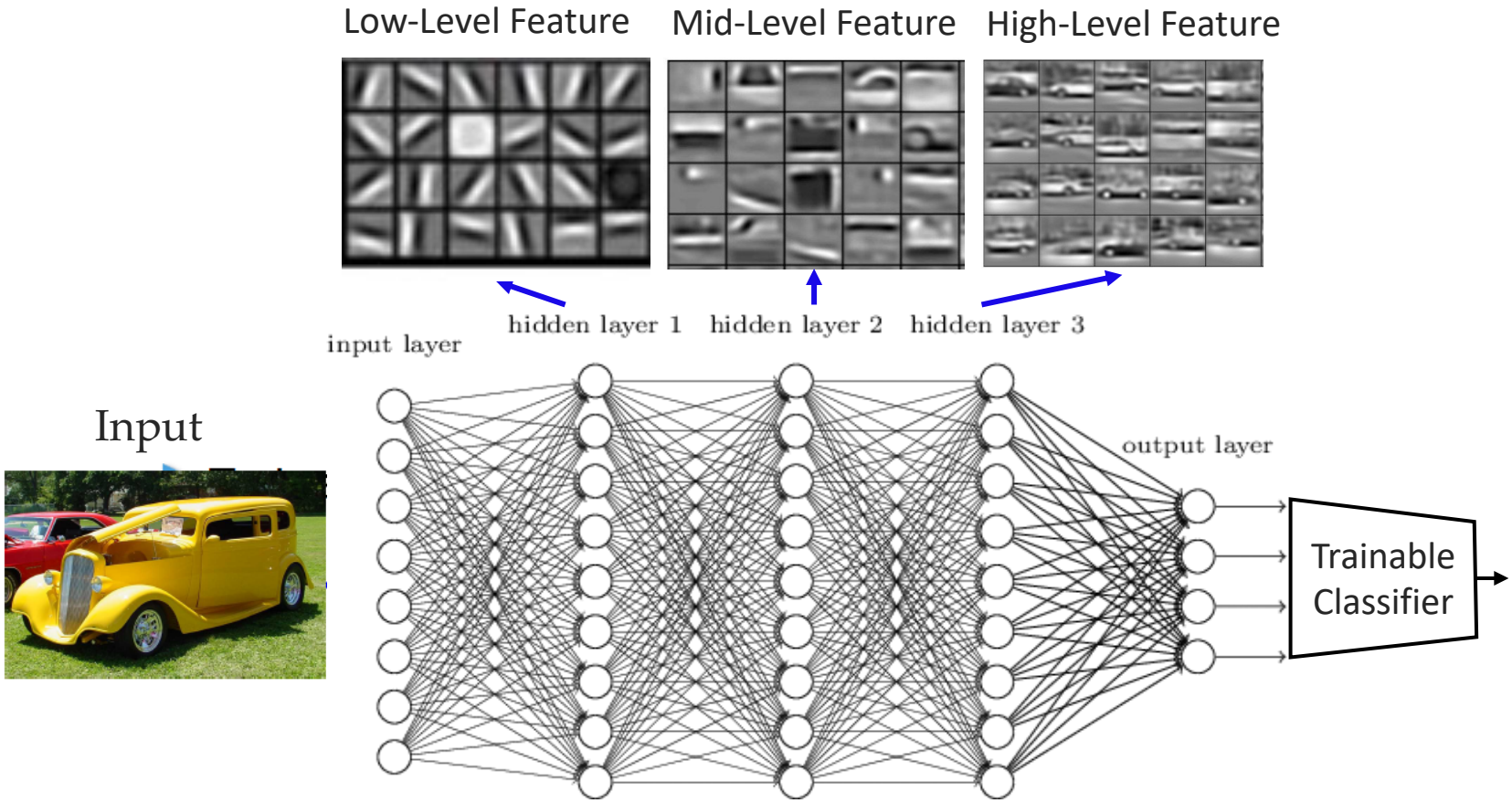
- Traditional machine learning for pattern recognition
  - ▶ fixed/engineered features + trainable classifier



- Deep Learning
  - ▶ **trainable** features + **trainable** classifier

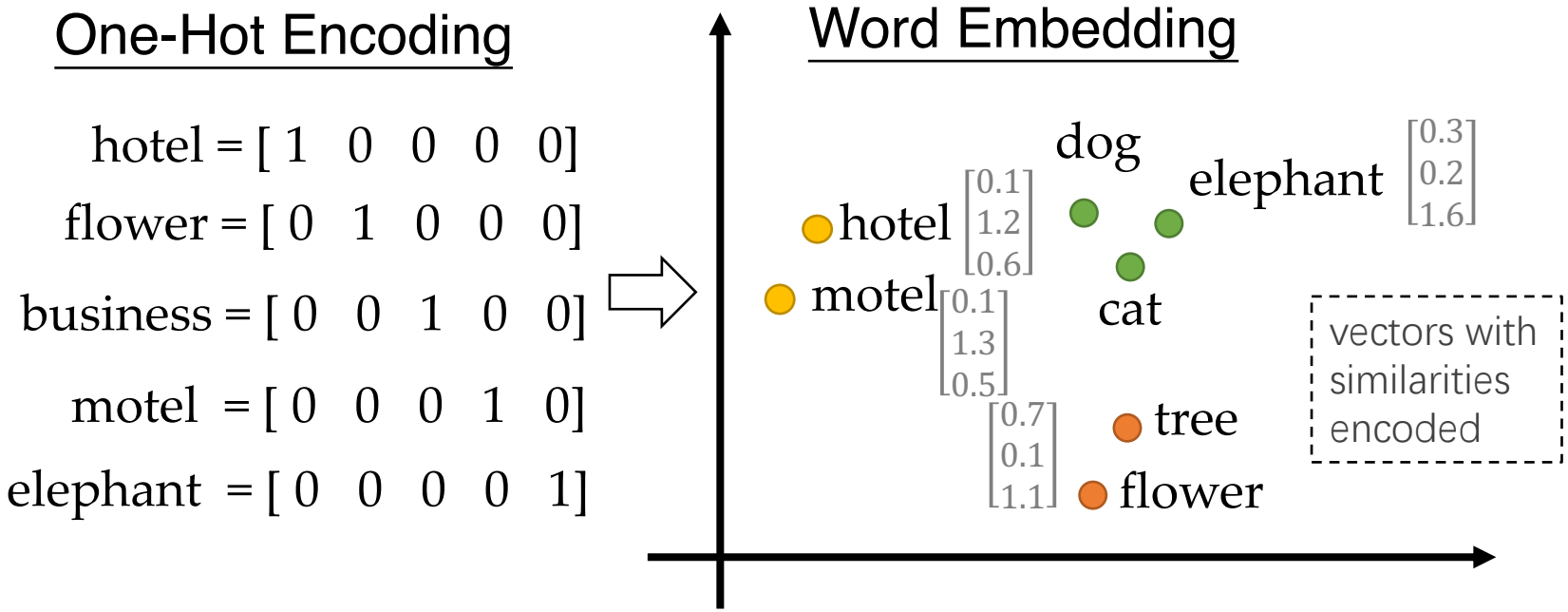


DL = Learning Hierarchical Representations



Word Embeddings

Represent words as low-dimensional **dense vectors** that can reflect their semantic similarities.



Note: word vectors are sometimes called word embeddings or word representations. They are distributed representations.

Word2Vec: Overview

- **Word2vec** (Mikolov et al. 2013) is a framework for **learning** word vectors.



Idea:

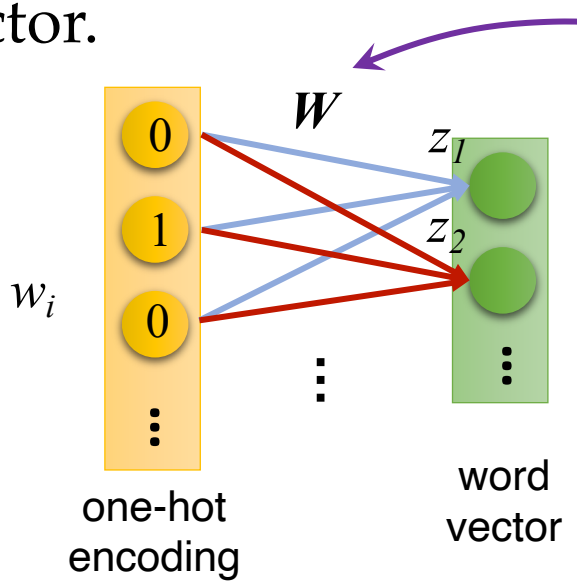
- we have a large corpus of text.
- every word in a fixed vocabulary is assigned with a **learnable vector**;
- go through each position t in the text, which has a center word c and context ("outside") words o;
- calculate the probability of o given c (or vice versa) using their vector similarities;
- keep updating the word vectors to maximize this probability.

Mikolov T, Sutskever I, Chen K, et al. Distributed representations of words and phrases and their compositionality. NIPS 2013.

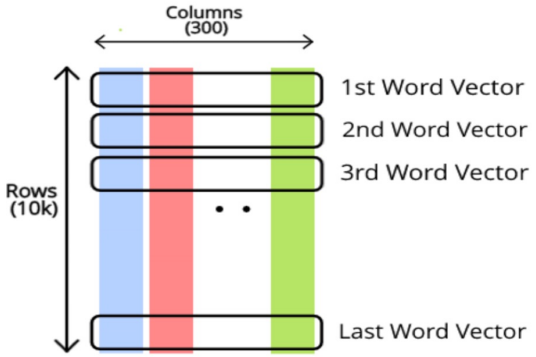
Model Architecture

- Input Layer: represent any word into a vector.

$$z_i = \boldsymbol{W} x_i = \boldsymbol{W}_i$$



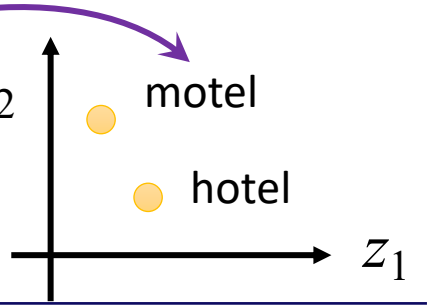
The weight matrix $\boldsymbol{W} \in R^{|V| \times d}$ is a **lookup table** with each row $\boldsymbol{W}_i$ being the vector for word w_i.



Example

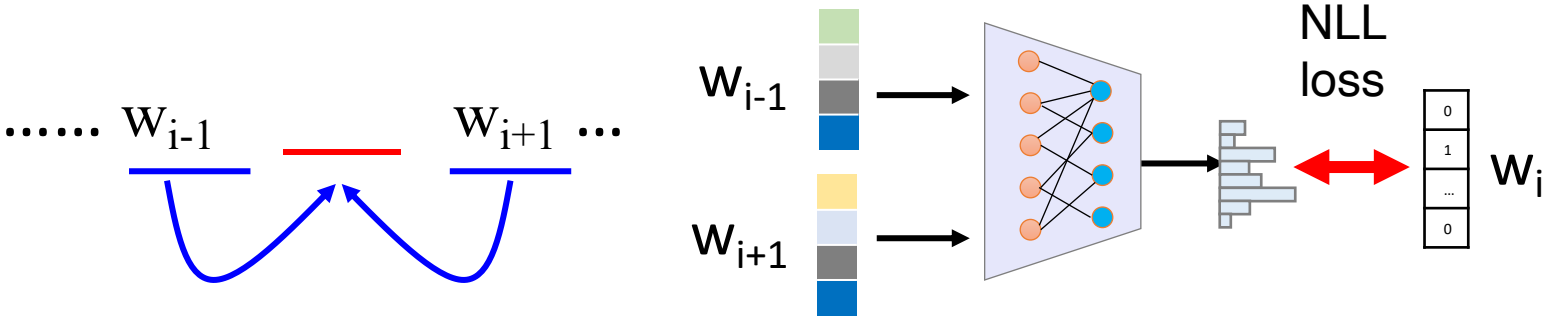
$$[0 \quad 0 \quad 0 \quad 1 \quad 0] \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = [10 \quad 12 \quad 19]$$

"motel"



Model Architecture

- **CBOW** predict the word given its context



Language Model

- A probabilistic model of how likely a given string appear in a given "language".
- For any sequence $x = (w_1, w_2, \ldots, w_N)$, a **language model** can be defined as:

$$p(x) = p(w_1, w_2, \ldots, w_N)$$

Example:

P_1=P("他明天去上课")
P_2=P("明天他将要会去上课")
P_3=P("明儿个他上课")
P_4=P("侬明朝得返课啦，唔好迟到啊嘅")

Chinese: $P_1 > P_2 > P_3 > P_4$

- Applications: 他明天将去上__？__

  message suggestion; document generation; spelling correction; machine translation; speech recognition;…

A Sequence Modeling Problem

- Idea #1: Counting?

“天 将 降 大 任 于 是”

given these words

predict the next word

$$p(\text{是} \mid \text{天将降大任于}) = \frac{\textbf{count}\,(\text{天将降大任于是})}{\textbf{count}\,(\text{天将降大任于})}$$

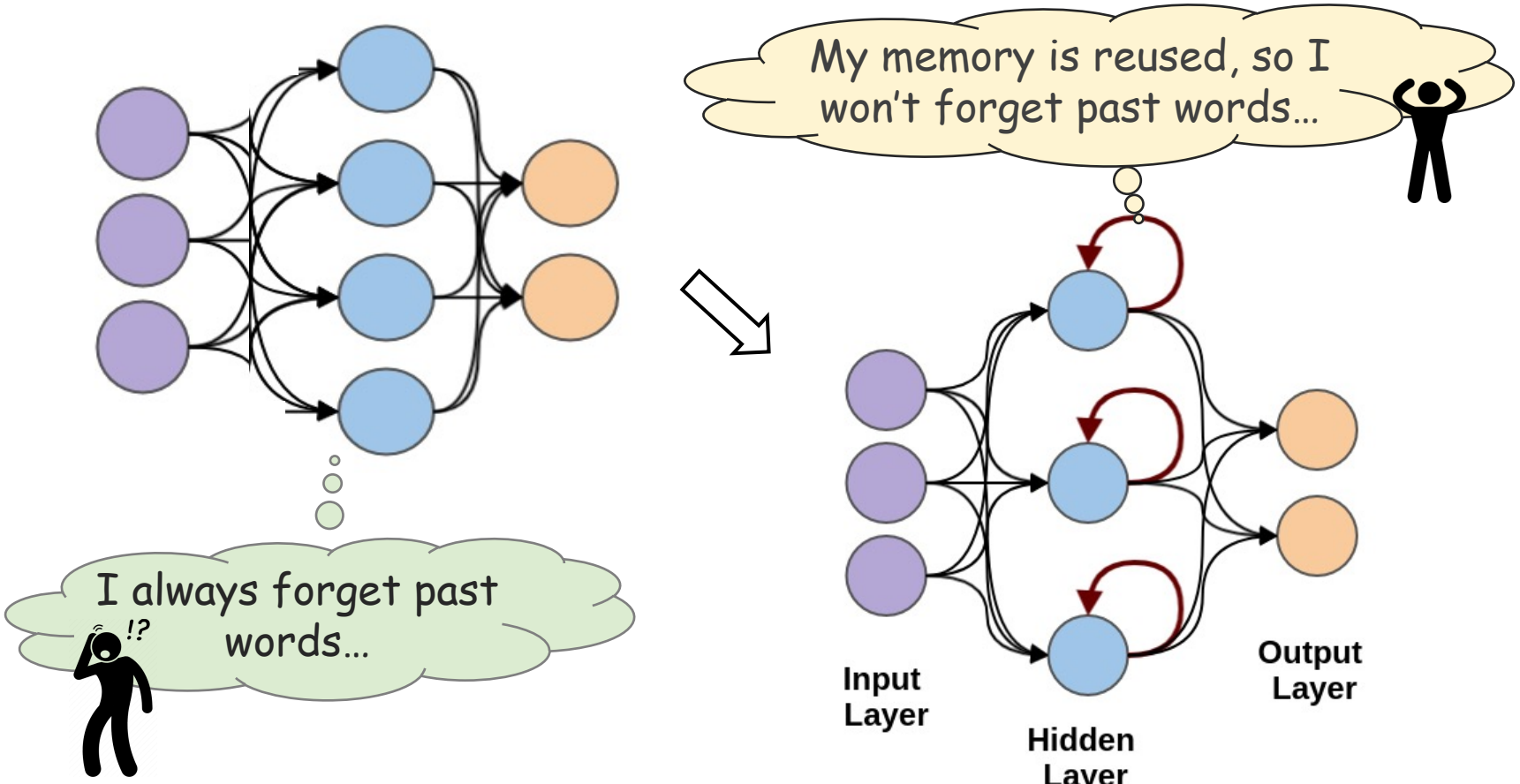
Problems:

- Discrete symbols,
- No word semantics,
- Insensitive to similar words and sentences.

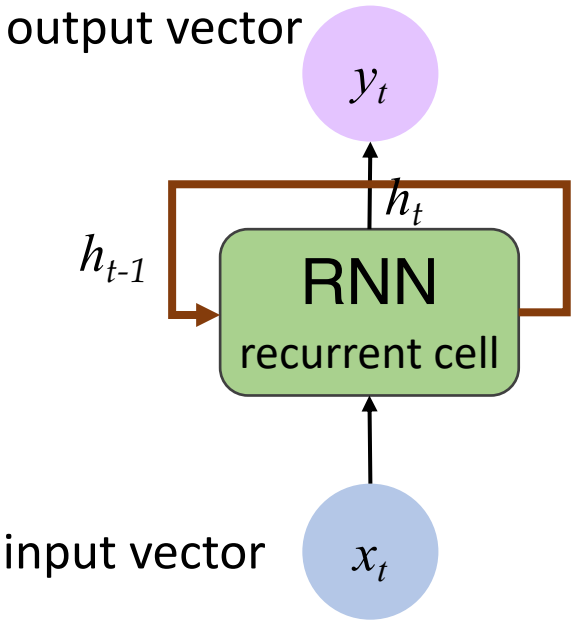
How about “天欲把重担给__”?

Recurrent Neural Networks

- Reuse the hidden layer to store the input information.



Recurrent Neural Network (RNN)



Apply a recurrent relation at every time step to process a sequence:

$$h_t = f_W(h_{t-1}, x_t)$$

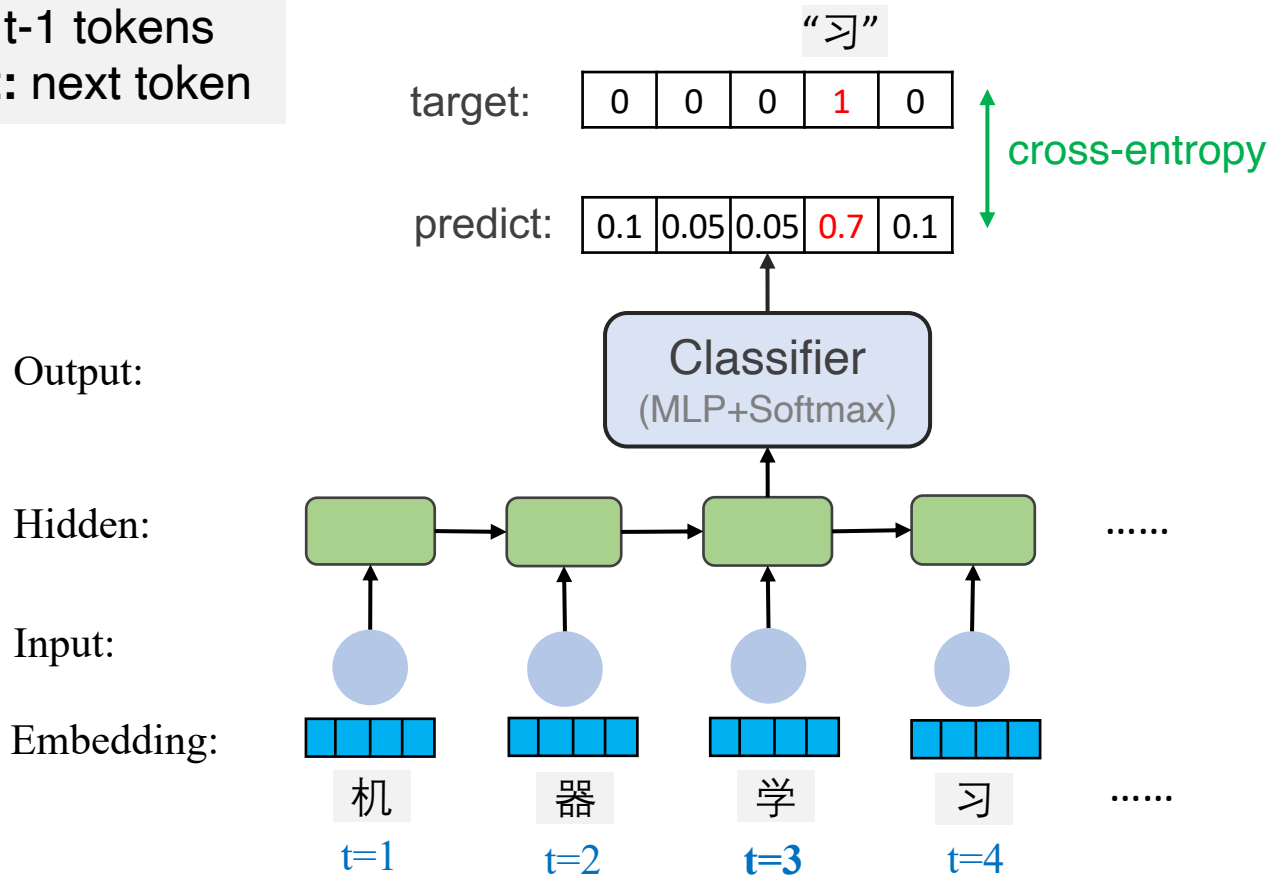
cell state — function parameterized by W — old state — Input vector at time step t

Note: the same function and set of parameters are used at every time step

RNNs have a **state**, h_t, that is updated **at each time step** as a sequence is processed.

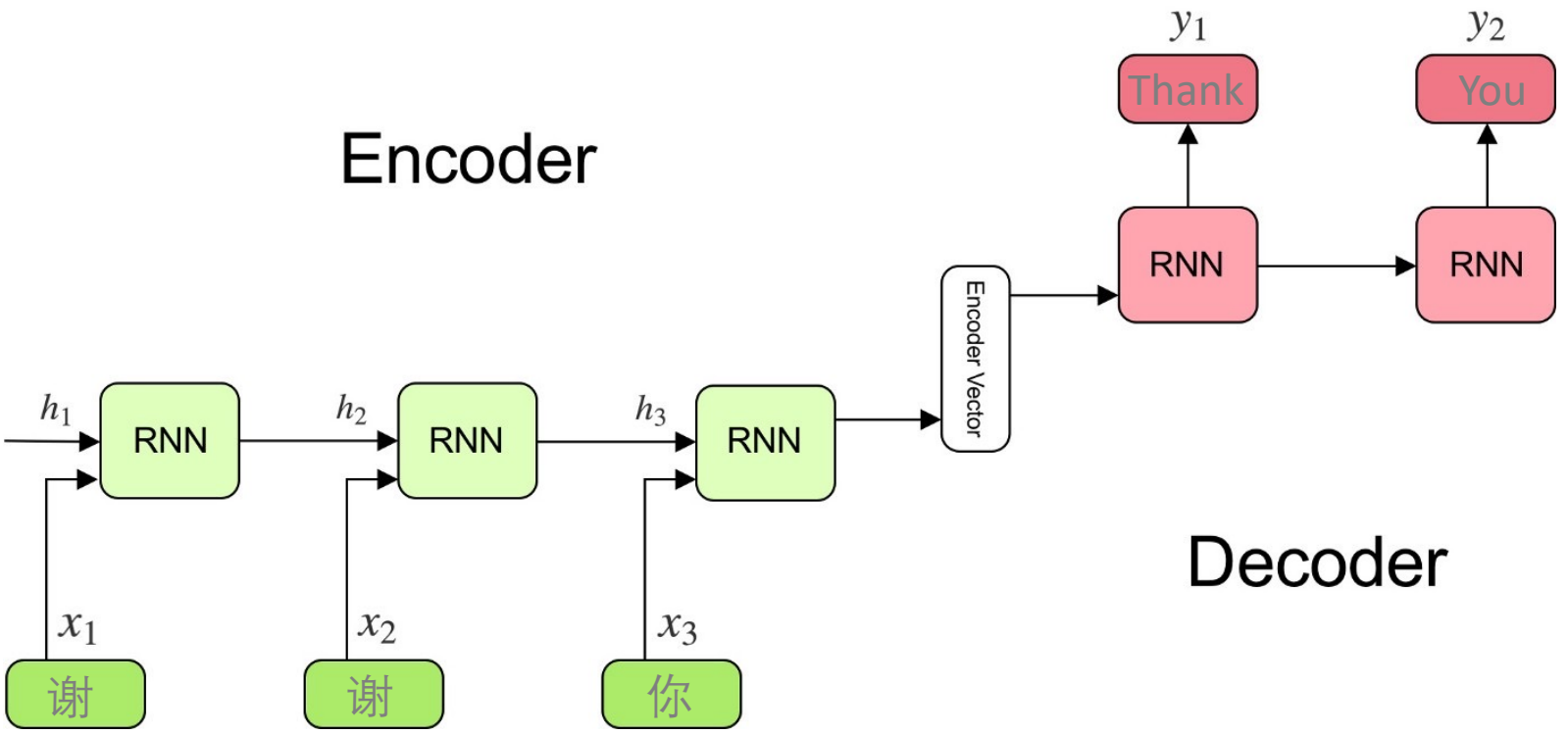
RNN for Sequence Generation

Input: t-1 tokens
Output: next token



RNN Encoder-Decoder (Cho 2014)

- given $x = (x_1, \cdots, x_{T_x})$, generate $y = (y_1, \cdots, y_{T_y})$

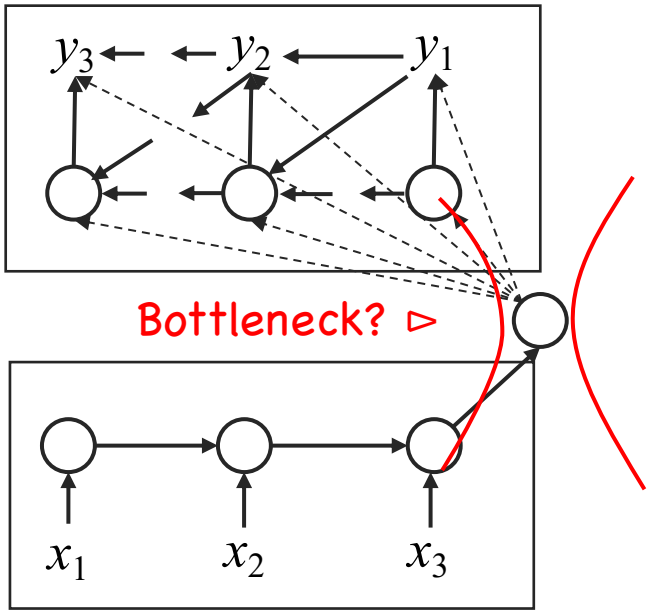


Sequence-to-Sequence with Attention

No Attention

decoding based on a single, fixed encoding vector

$h_i = \text{RNN}_{\text{out}}(h_{i-1}, y_{i-1}, \boldsymbol{c})$

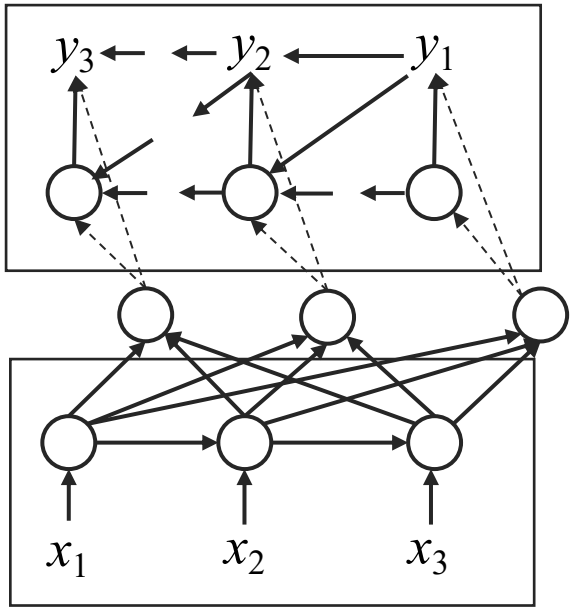


VS

With Attention

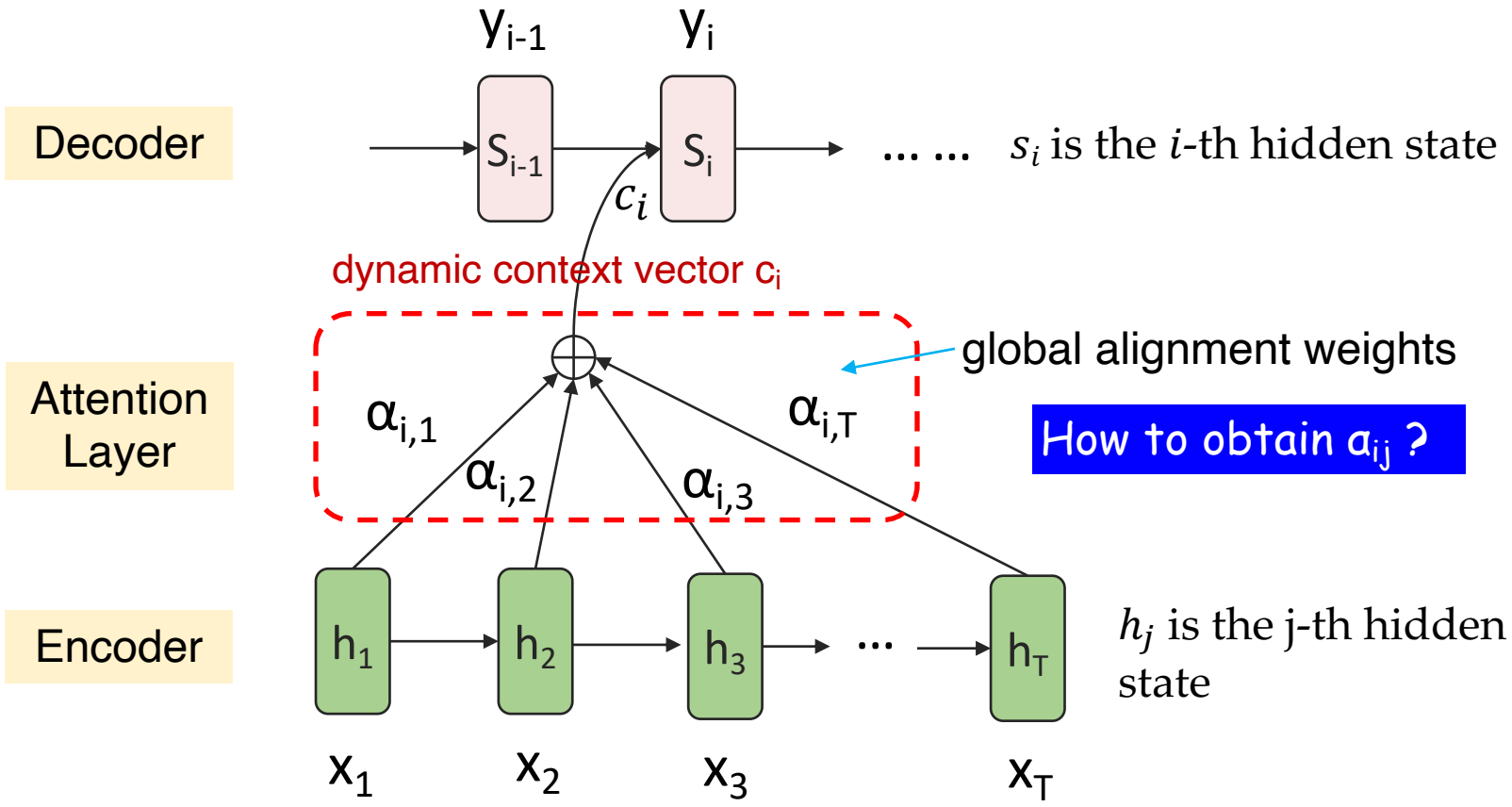
select what to encode in a contextual manner.

$h_i = \text{RNN}_{\text{out}}(h_{i-1}, y_{i-1}, \boldsymbol{c_i})$



Sequence-to-Sequence with Attention

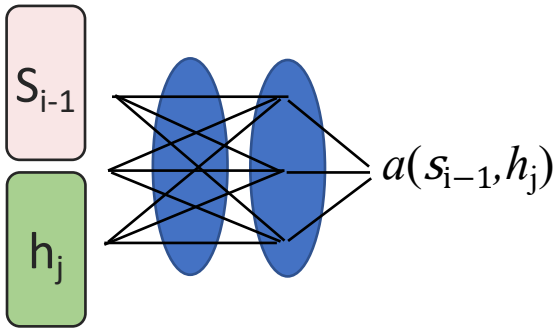
The decoder **dynamically** pays attention to different tokens in the source sentences during decoding.

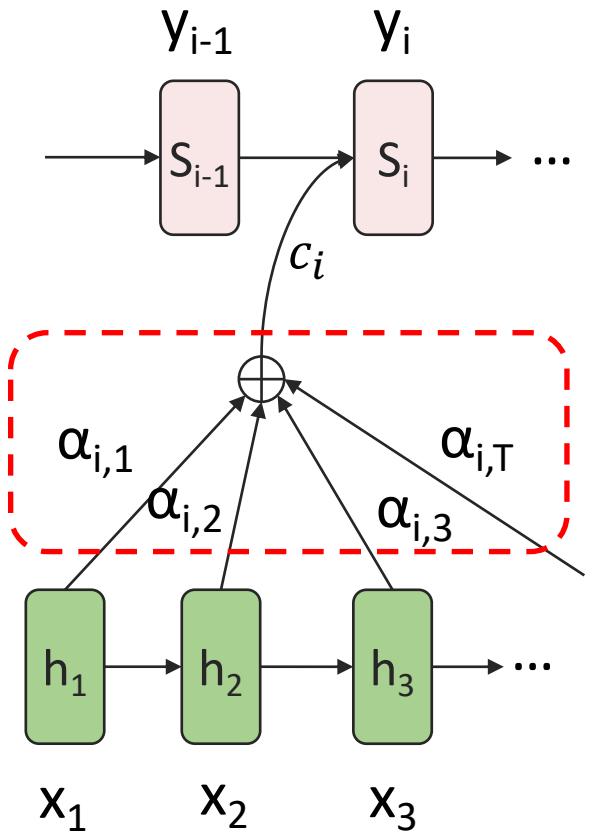


Attention-based Model

$$\alpha_{ij} = \frac{\exp\left(a(s_{i-1}, h_j)\right)}{\sum_{k=1}^{N} \exp\left(a(s_{i-1}, h_k)\right)}$$

- scores how well the inputs around position j and the output at position i match.
- where $a(\cdot)$ denotes a **neural network**:
  e.g., $a(s_{i-1}, h_j) = v_a^T \tanh\left(W_a s_{i-1} + U_a h_j\right)$



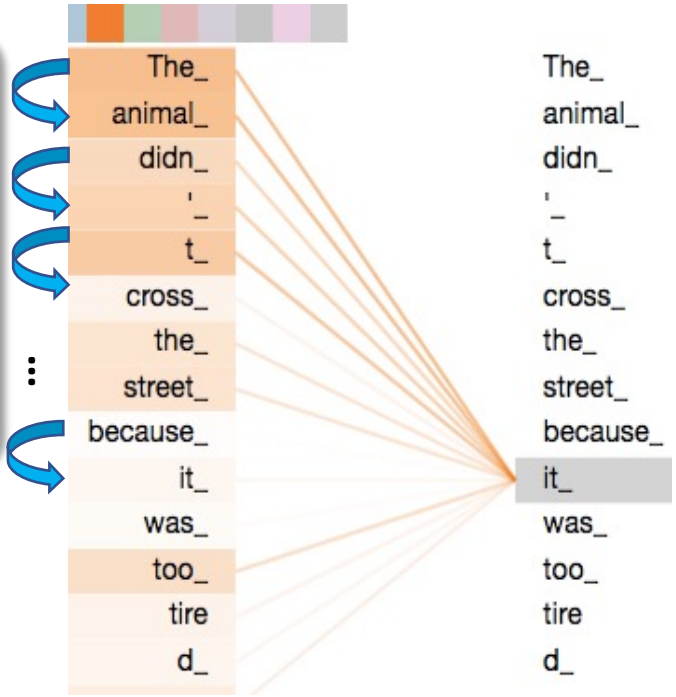


From RNN to Self-Attention

What does "**it**" in this sentence refer to?

RNN

maintain a hidden state to sequentially incorporate previous words with the current one it's processing.



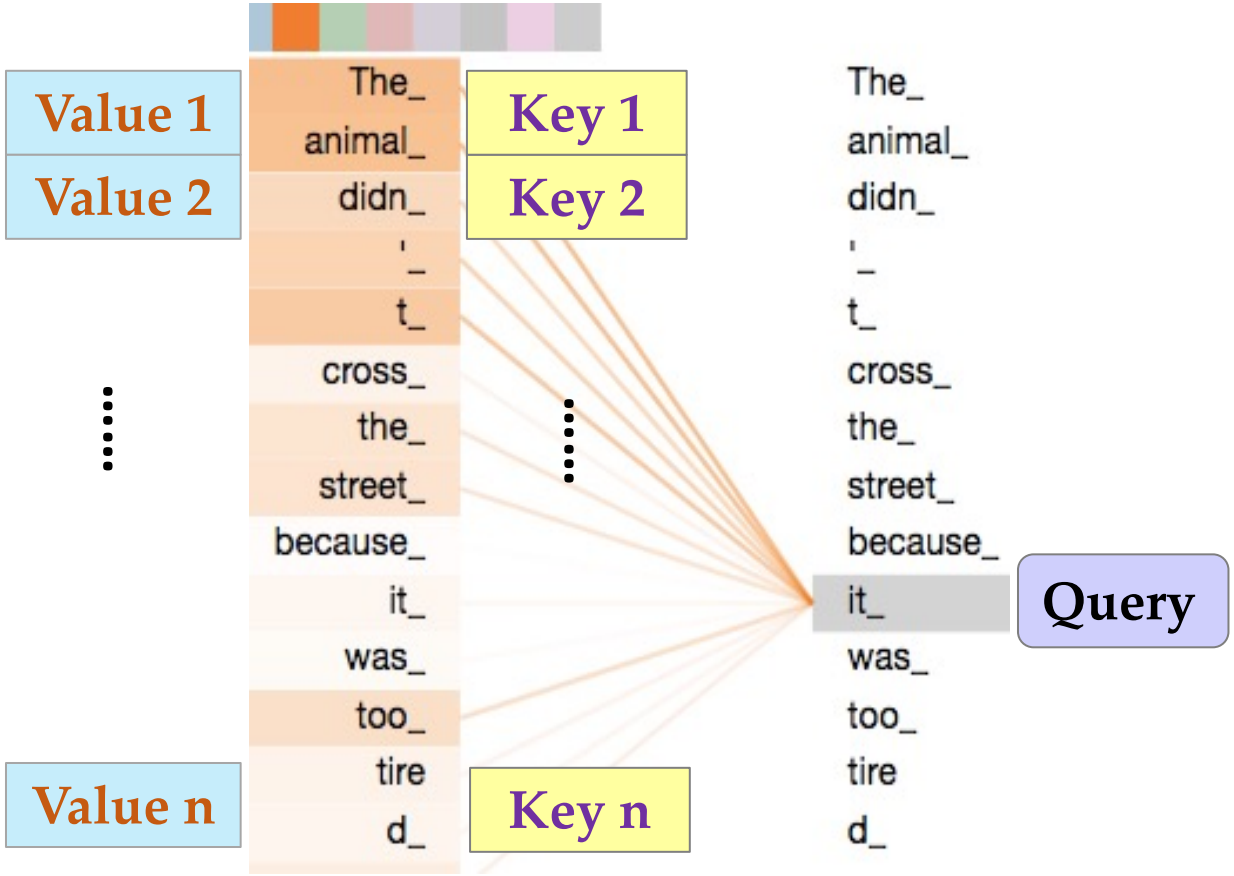
Self-Attention

bake the "understanding" of other relevant words into the one we're currently processing simultaneously.

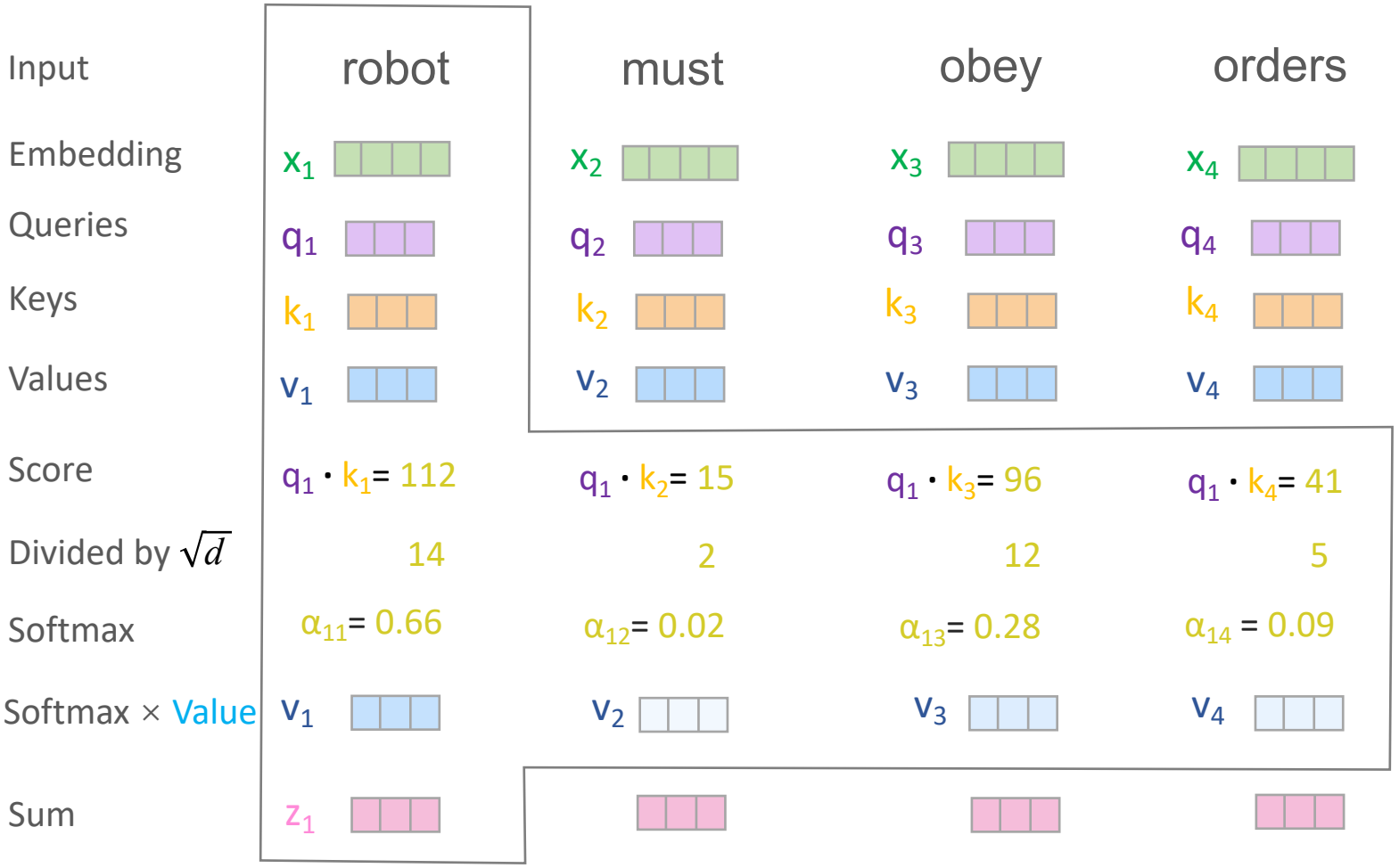
The animal didn't cross the street because **it** was too tiered.

Self-Attention: The Idea

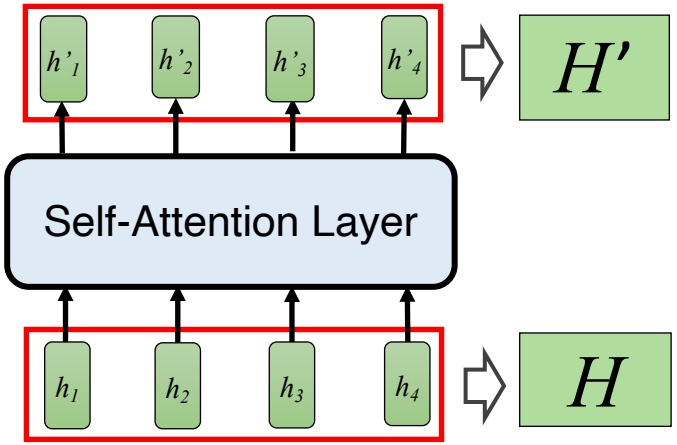
- Let each word pay attention to all other words.



Self-Attention



Self-Attention as Matrix Multiplication



$$H' \qquad Q = W^q H$$

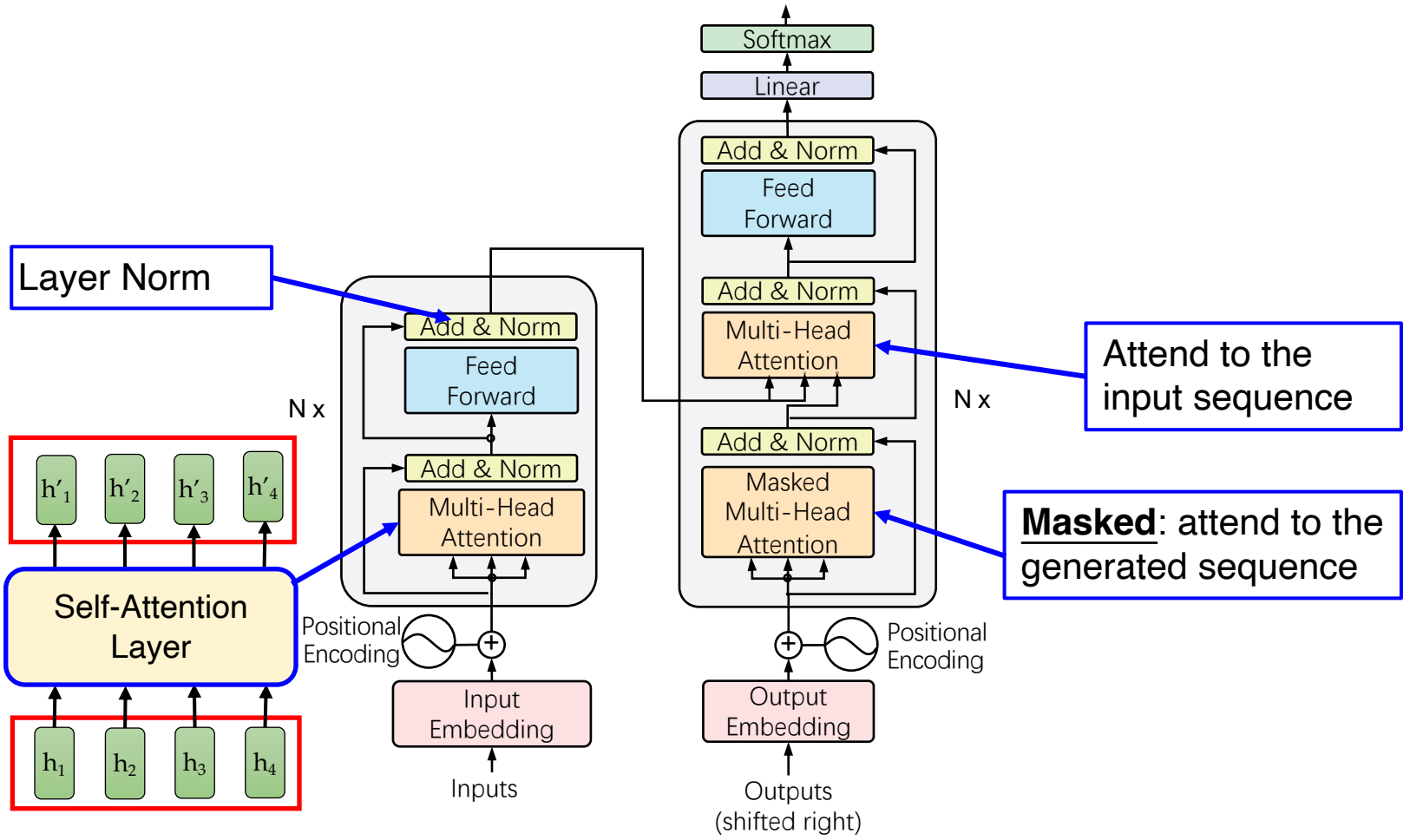
$$K = W^k H$$

$$H \qquad V = W^v H$$

$$\hat{A} = \text{softmax}(A) = K^T Q$$

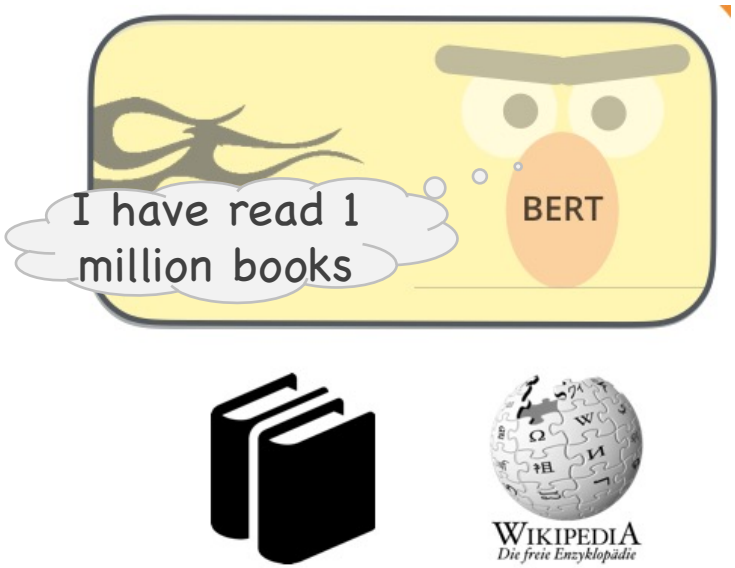
$$H' = V \hat{A}$$

Transformer



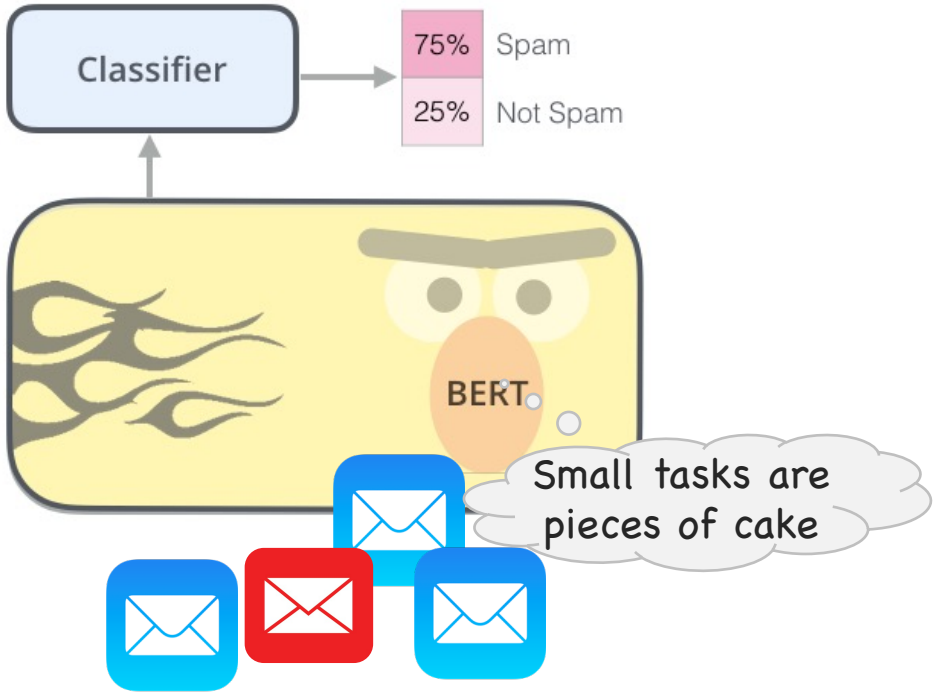
Pre-trained Language Models

Unsupervised training on large amounts of text (e.g., books, Wikipedia, etc)



Phase 1: Pre-Training

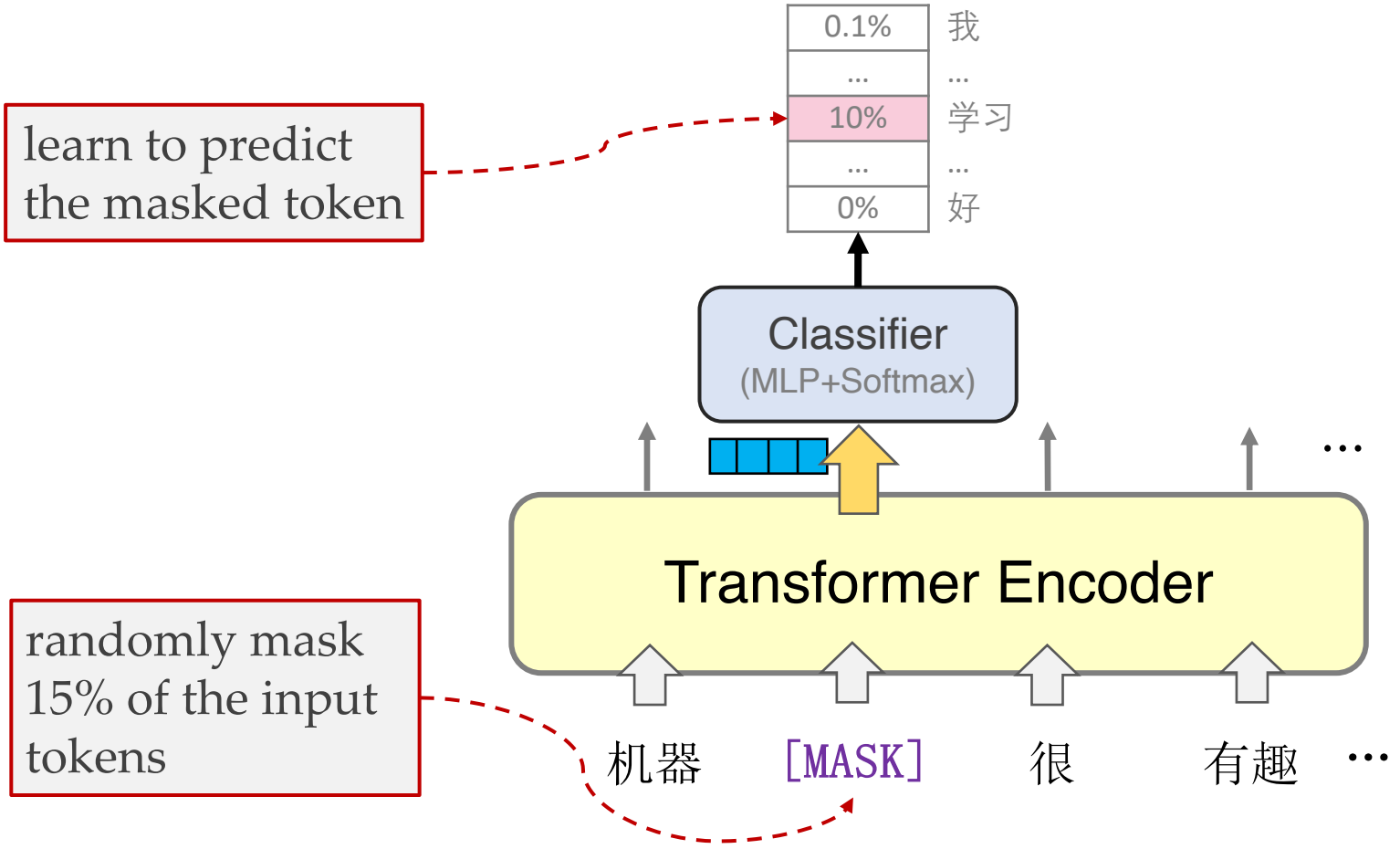
Supervised training on a specific task with a labeled dataset. (e.g., spam detection)



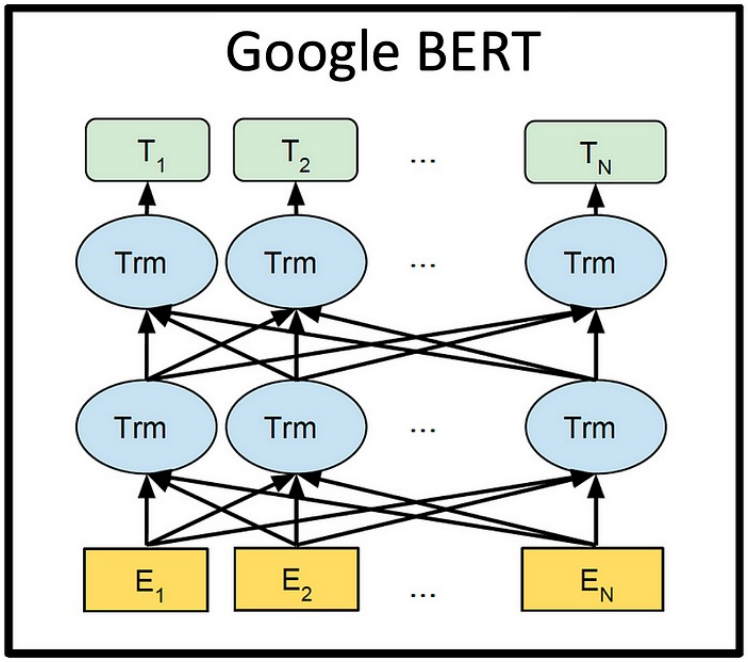
Phase 2: Fine-Tuning

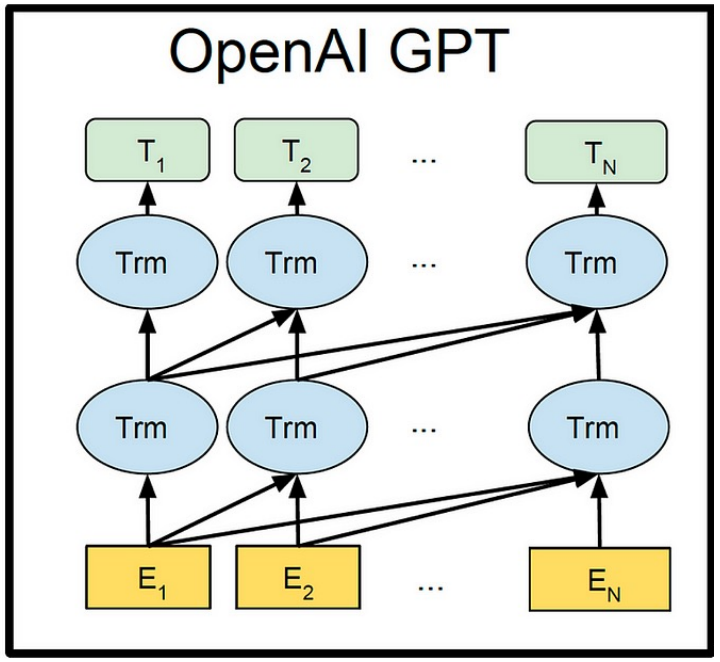
Pre-Training

Task 1: Masked Language Model (MLM)



BERT vs GPT

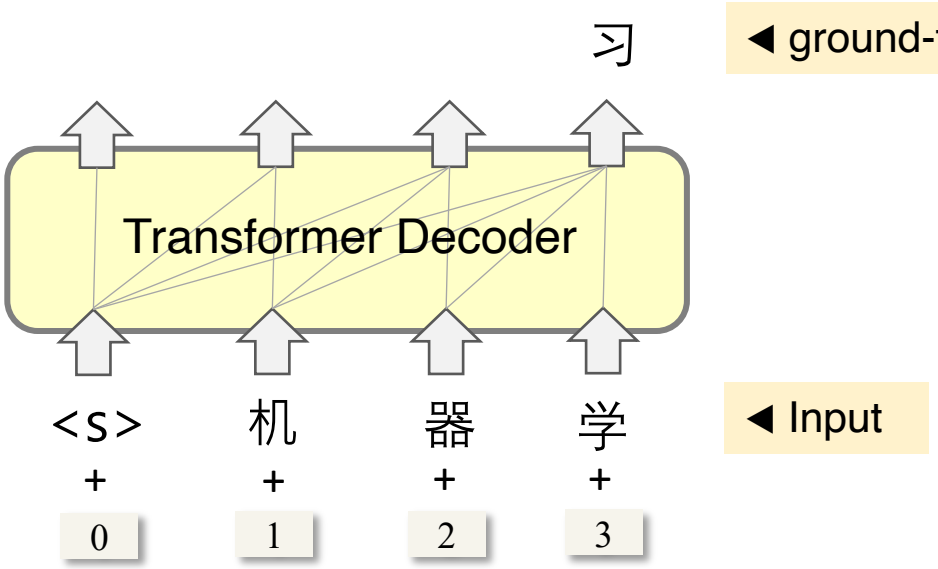

Google BERT
T1
T2
...
TN
Trm
Trm
...
Trm
Trm
Trm
...
Trm
E1
E2
...
EN


OpenAI GPT
T1
T2
...
TN
Trm
Trm
...
Trm
Trm
Trm
...
Trm
E1
E2
...
EN

Pre-Training GPT

Simply Language Model:

— Given the preceding n-1 tokens, predict the n-th token.



Training objective:

$$l(\theta) = -\sum_{t=1}^{T} \log p_\theta\left(x_t | x_1, \ldots, x_{t-1}\right)$$

From MLP to CNN

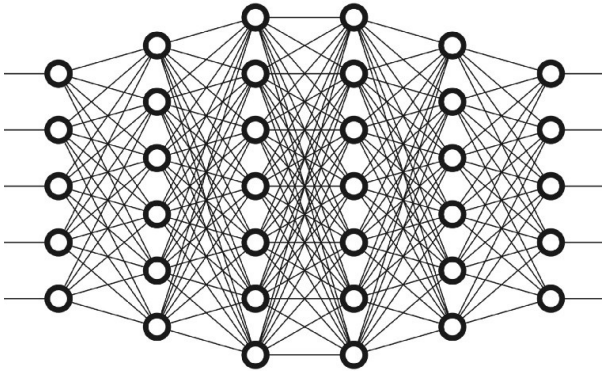


Input Image

matrix of pixel
(32x32x3)

17
255
2
28
45
⋮
122

vector
3072x1



NN

Problems with MLP

Seq-of-Pixels: The spatial structure of images is destroyed

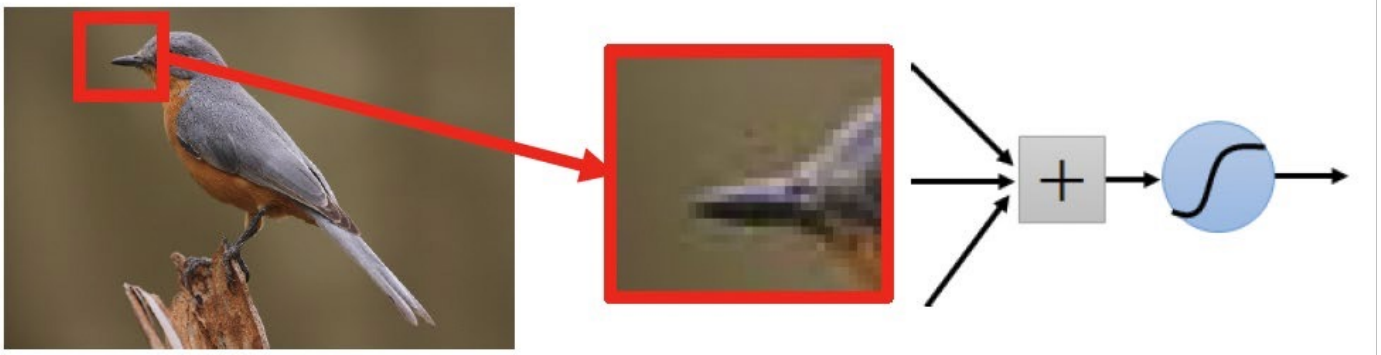
Locality insensitive: concern the entire picture instead of local features.

Translation variant: sensitive to the global position of a feature

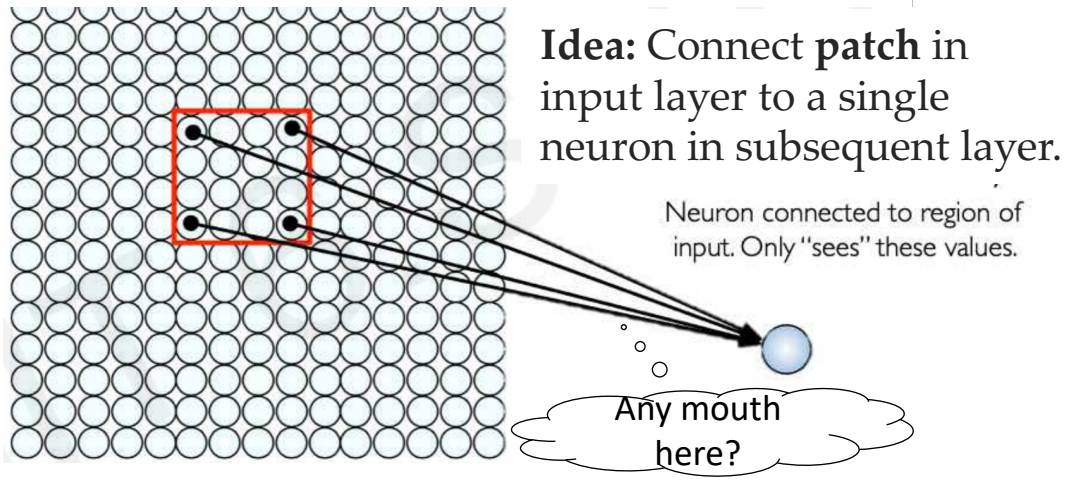
Too many parameters: fully connected layers yield a huge volume of parameters.

Convolution: The Idea

This inspires us to scan over the image for a specific feature through weighing and activating.



Input: 2D image.
Array of pixel values

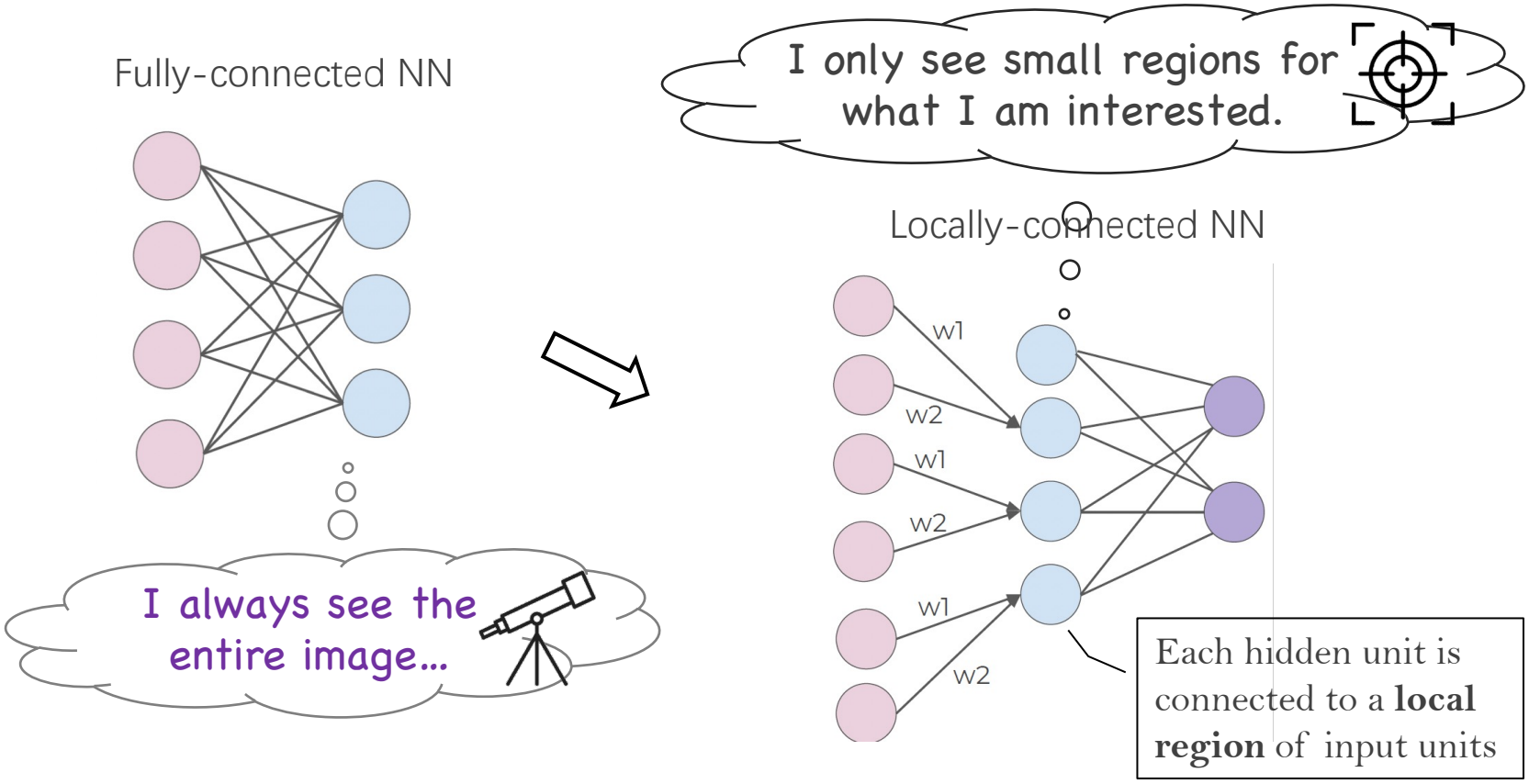


Idea: Connect **patch** in input layer to a single neuron in subsequent layer.

MLP vs CNN

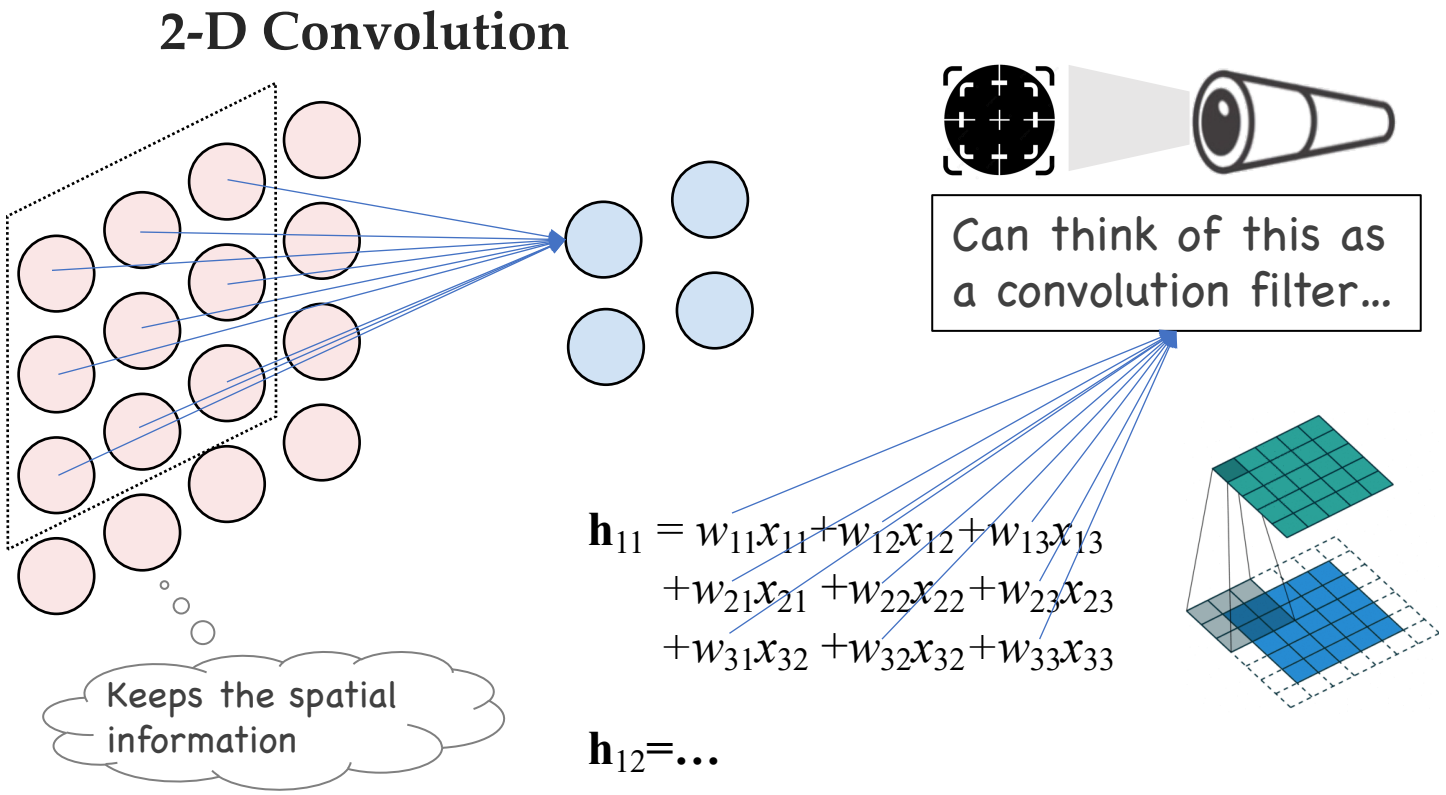
Restrict the scope of hidden units to focus on local features.

Sharing weights among hidden units



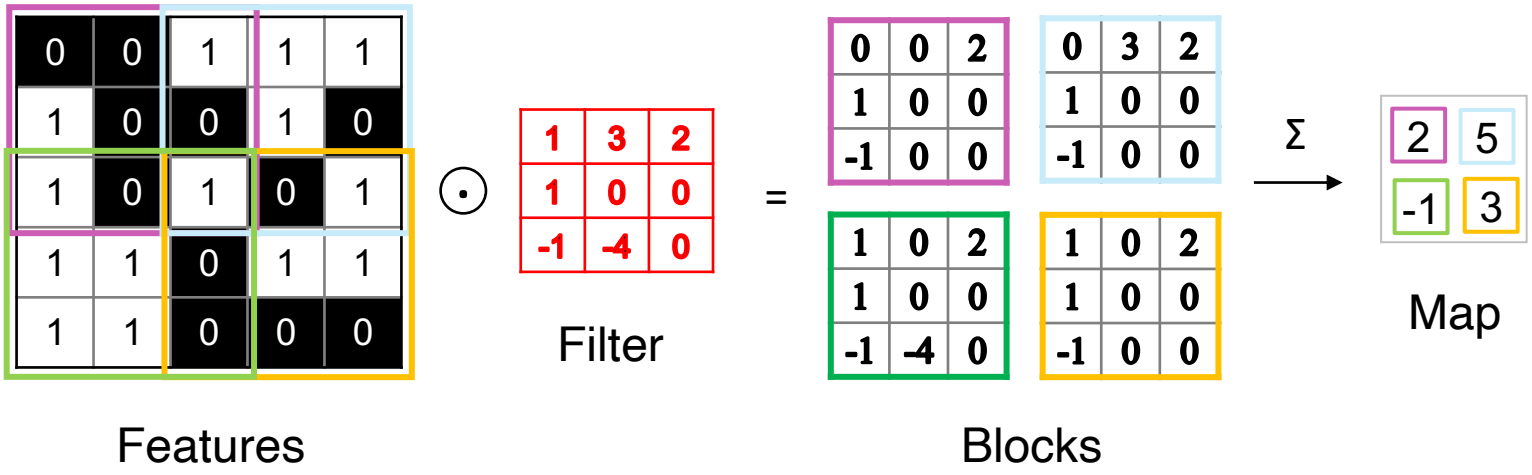
2-D Convolutions

Now let's expand these concepts to 2-D Convolution, since we'll mainly be dealing with images.



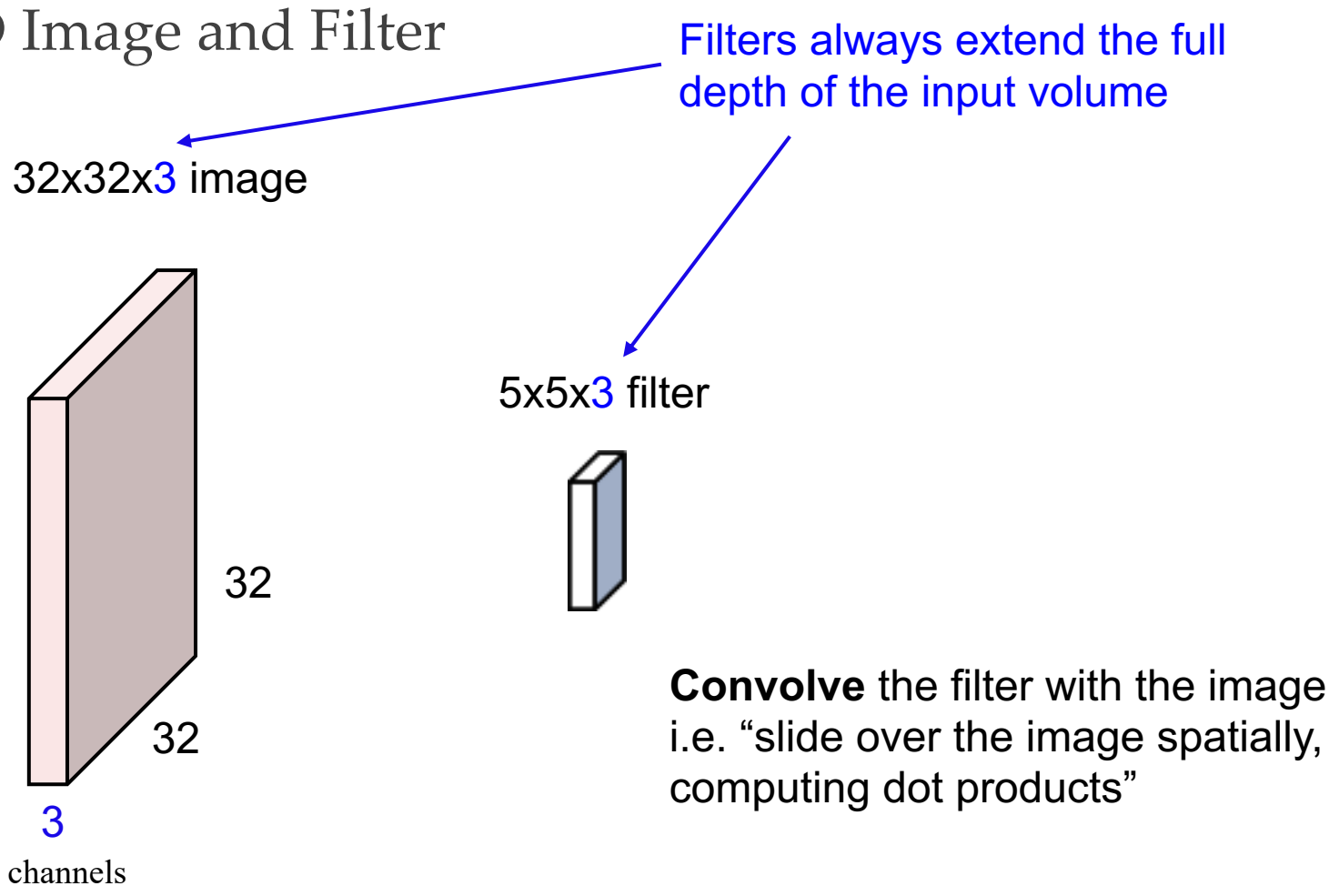
2-D Convolutions

Example:



"3-D" Convolution

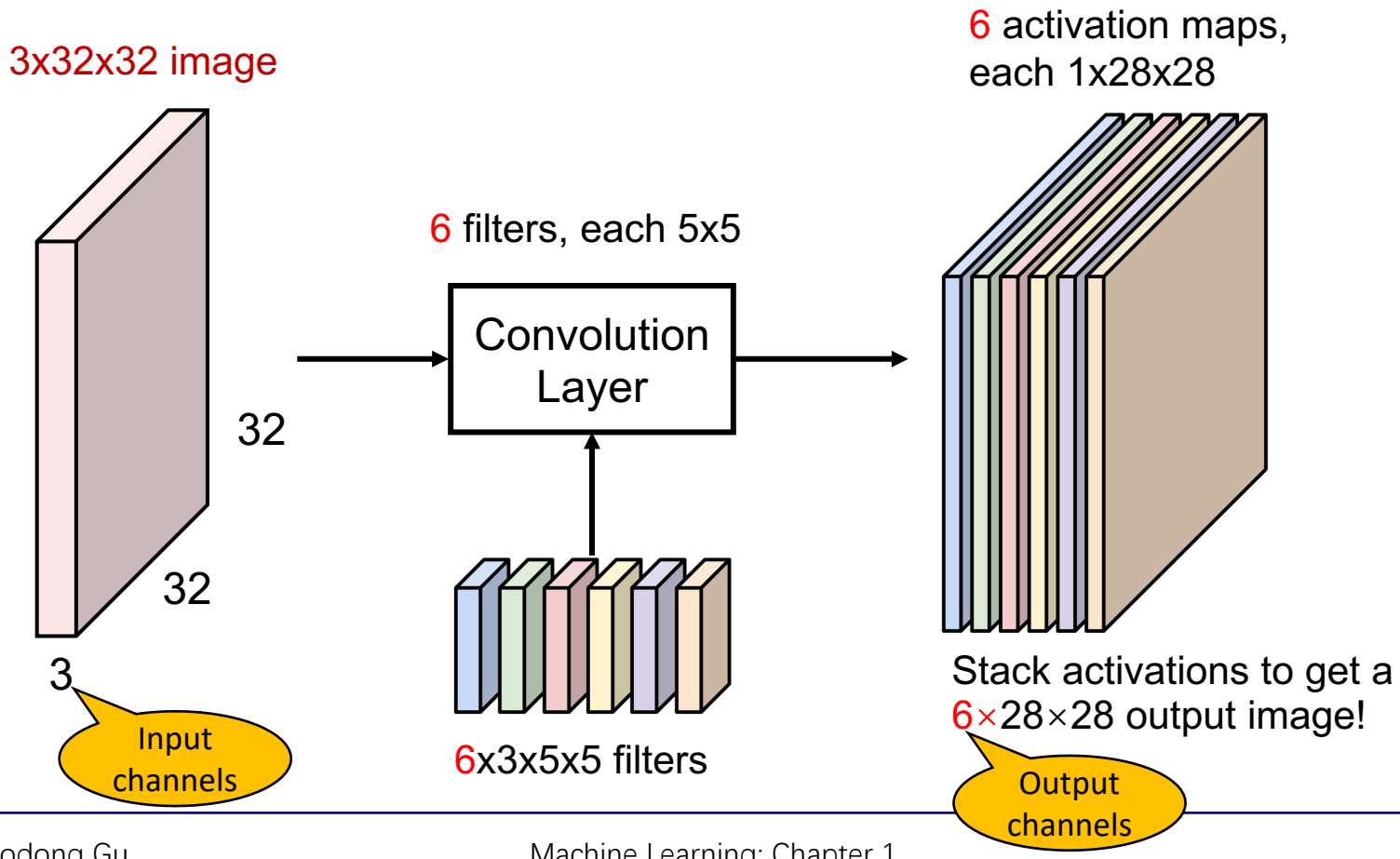
3-D Image and Filter



Convolve the filter with the image i.e. "slide over the image spatially, computing dot products"

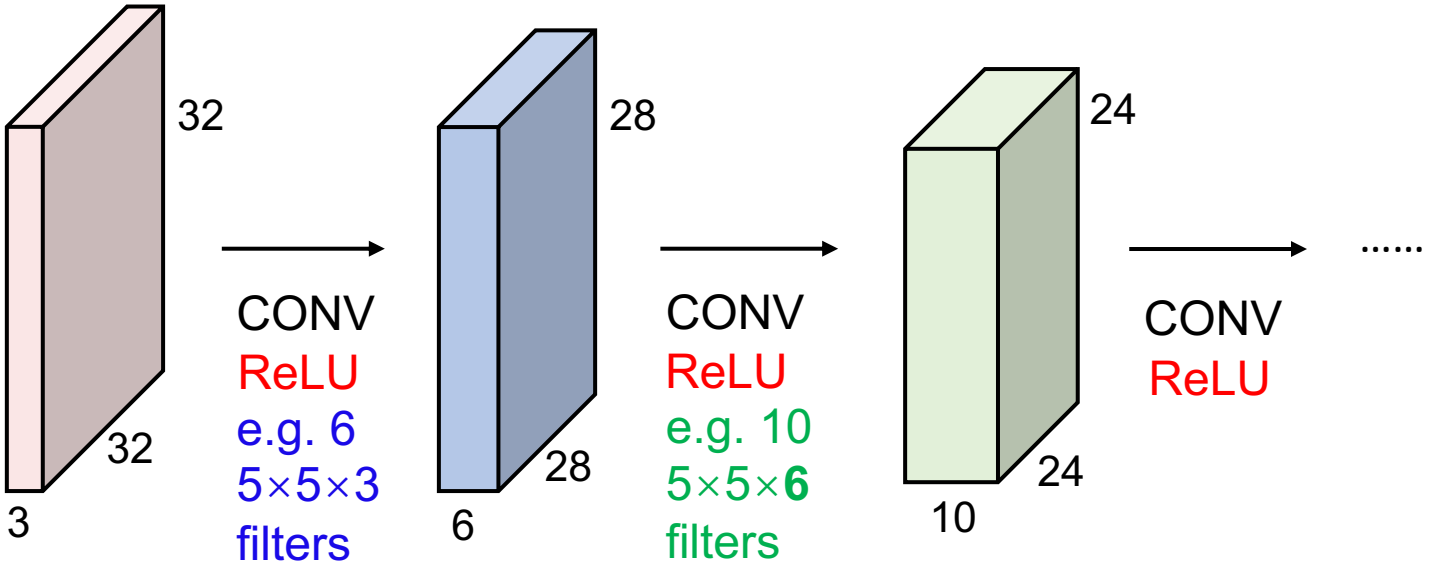
Multiple Feature Maps

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



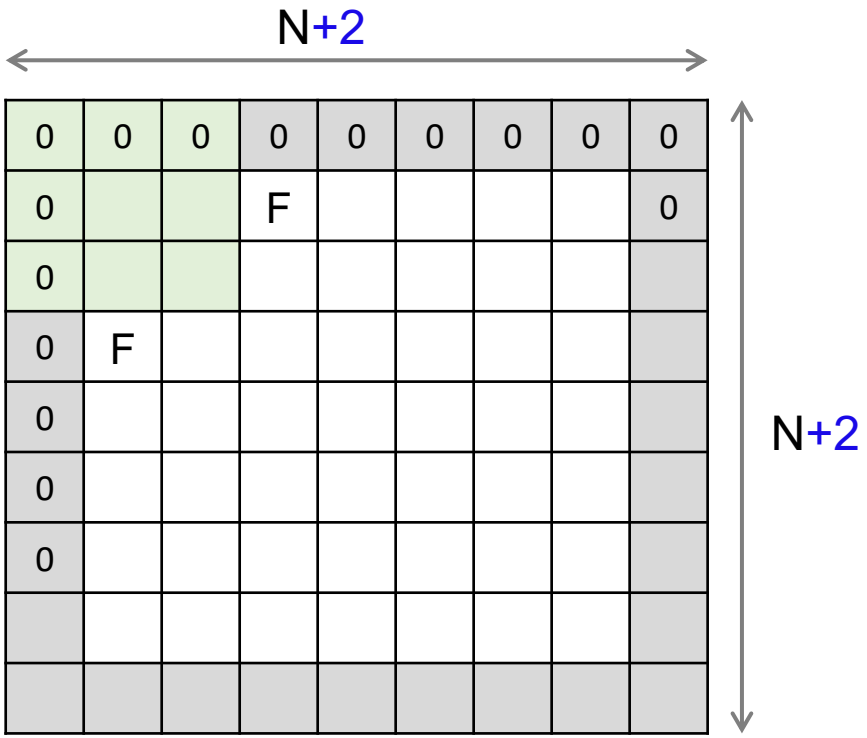
Multiple Layers

- A sequence of convolutional layers, interspersed with activation functions (e.g., ReLU).



A Closer Look at Spatial Dimensions

In practice: Common to zero pad the border



Pad with P pixel border

Output size:
(N +2P - F) / stride + 1

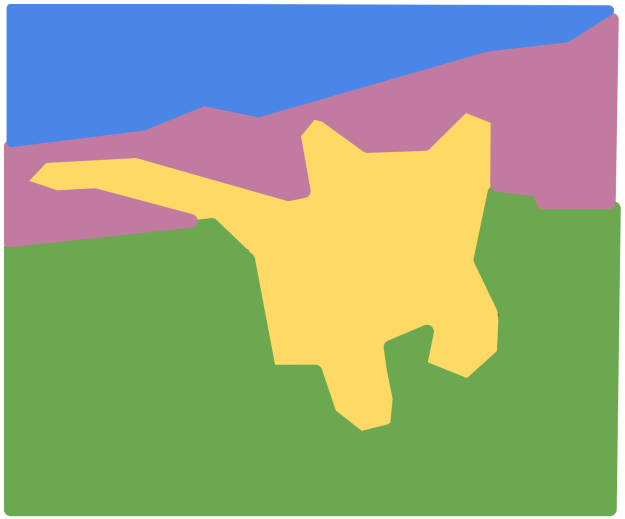
e.g. N = 7, F = 3:
stride 1 => (7 +2 - 3)/1 + 1 = 7
stride 2 => (7 +2 - 3)/2 + 1 = 4
stride 3 => (7 +2 - 3)/3 + 1 = 3

Semantic Segmentation

- Segment an image into disjoint regions that have independent semantics.

Label each pixel in the image with a category





GRASS,
CAT,
TREE,
SKY, ...

Input: an image

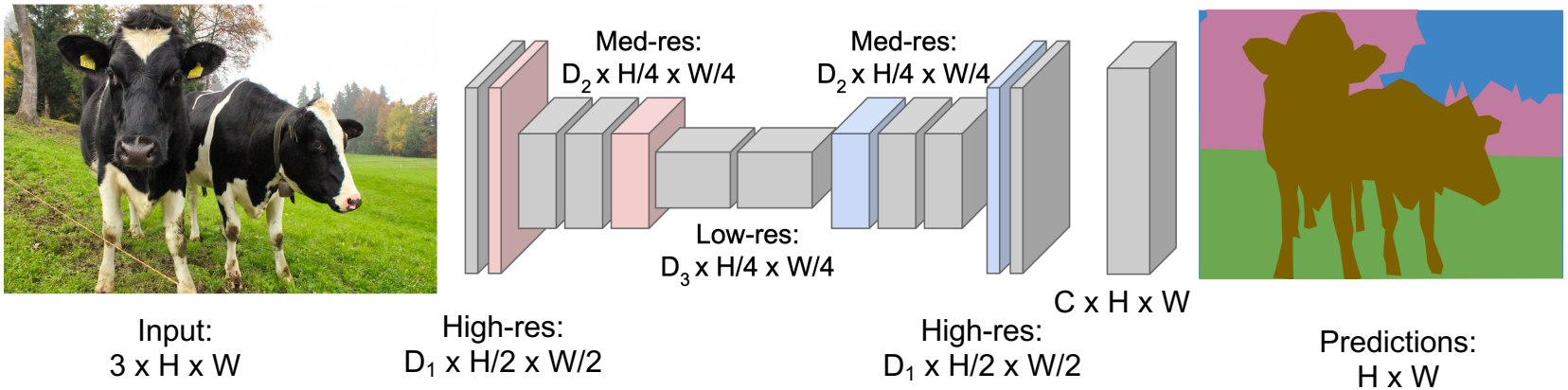
Output: class of each pixel

Fully Convolutional?

Instead: Design networks as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Downsampling:
Pooling, strided convolution

Upsampling:
???



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

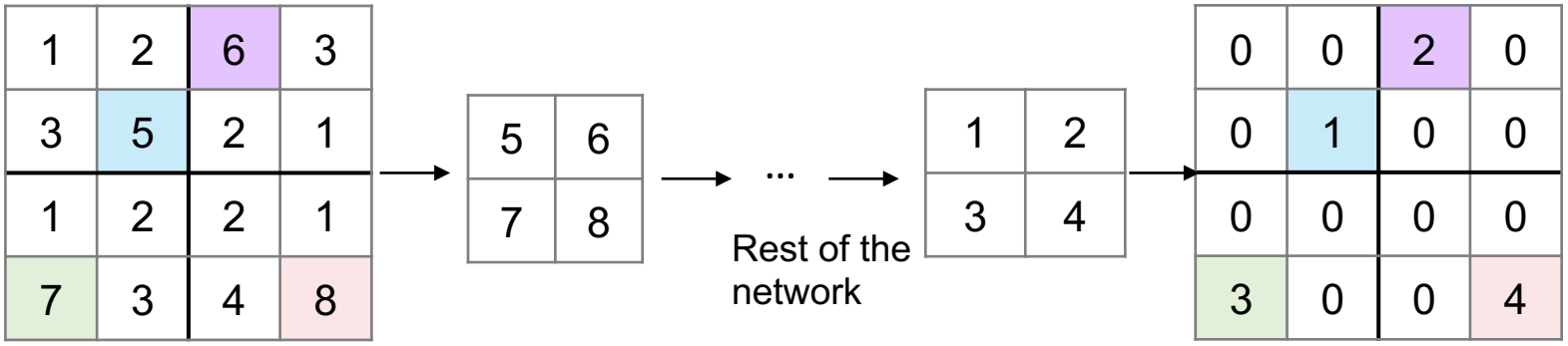
In-Network Upsampling: "Max Unpooling"

Max Pooling

Remember which element was max!

Max Unpooling

Use positions from pooling layer



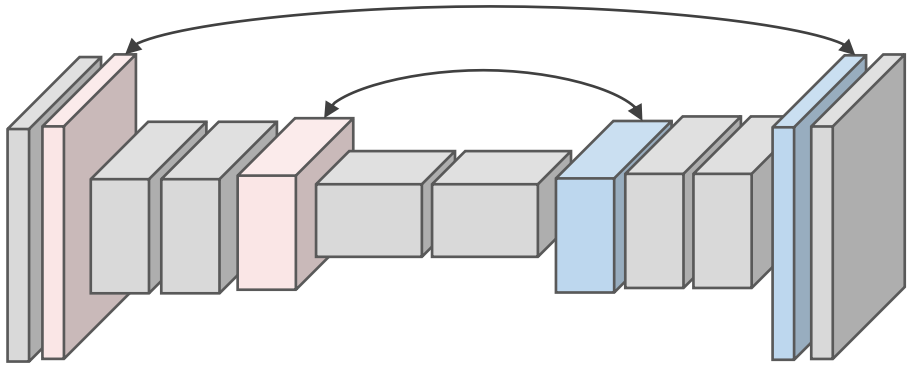
Input: 4 x 4

Output: 2 x 2

Input: 2 x 2

Output: 4 x 4

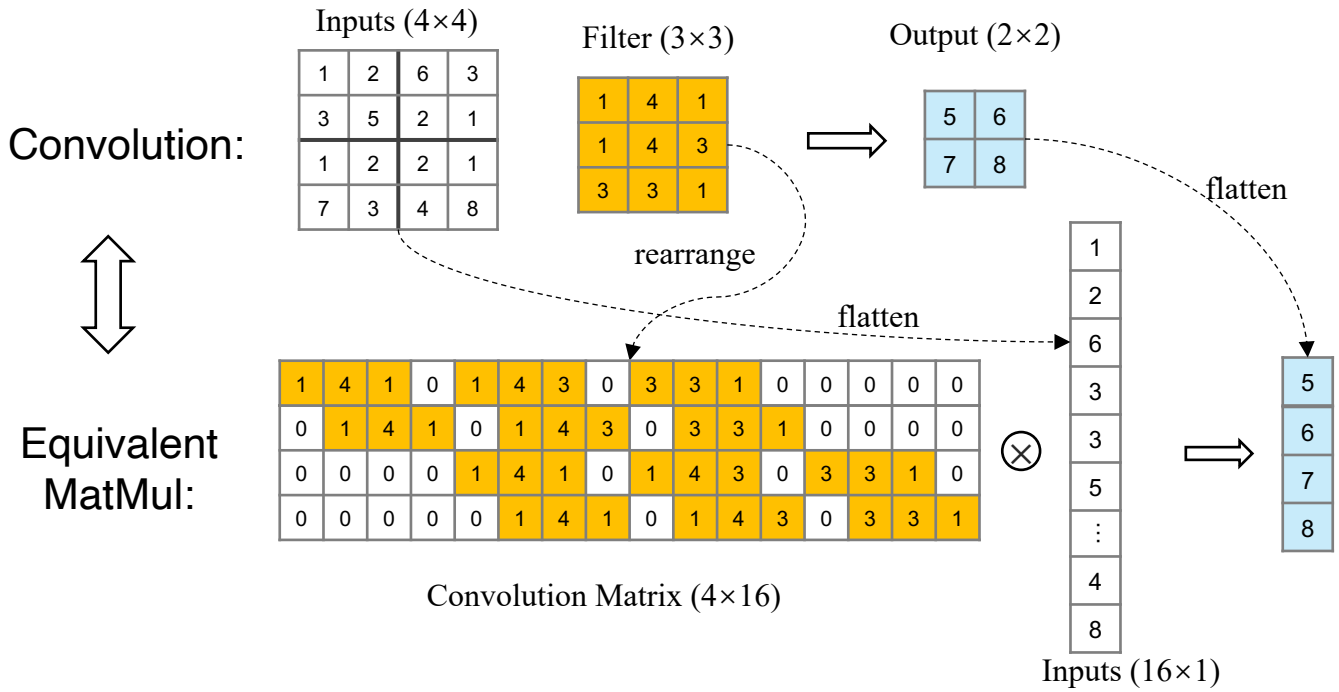
Corresponding pairs of downsampling and upsampling layers



Upsampling with Transposed Convolution

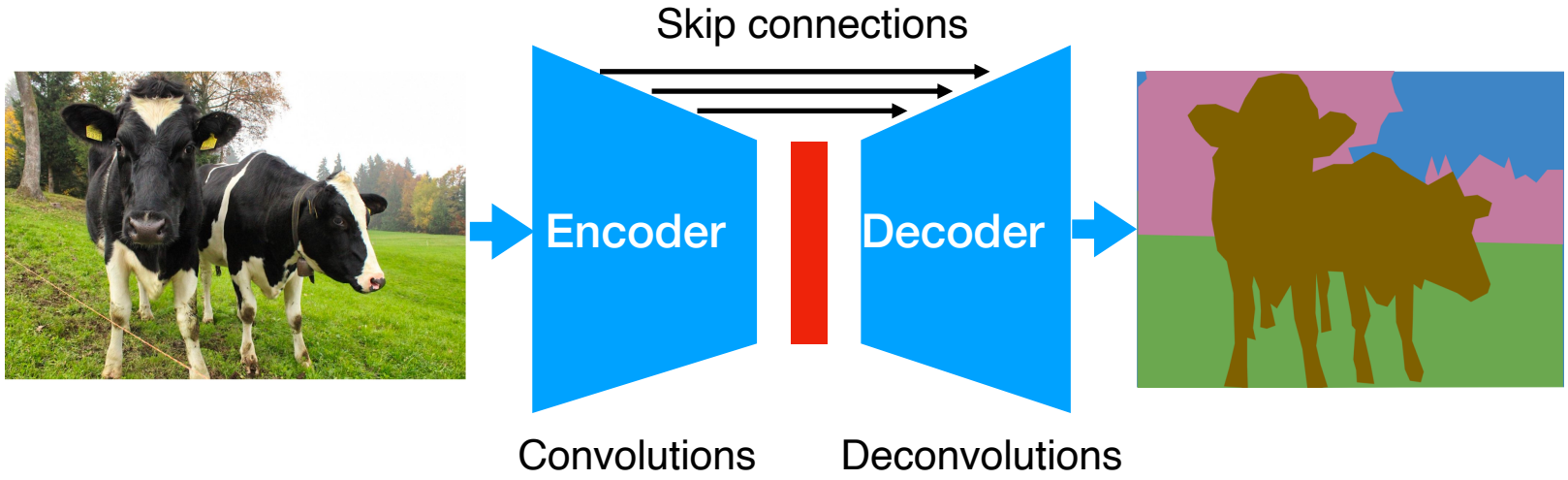
Convolution Matrix

We can reformulate convolution as **matrix multiplication** by rearranging the kernel matrix into a general matrix.



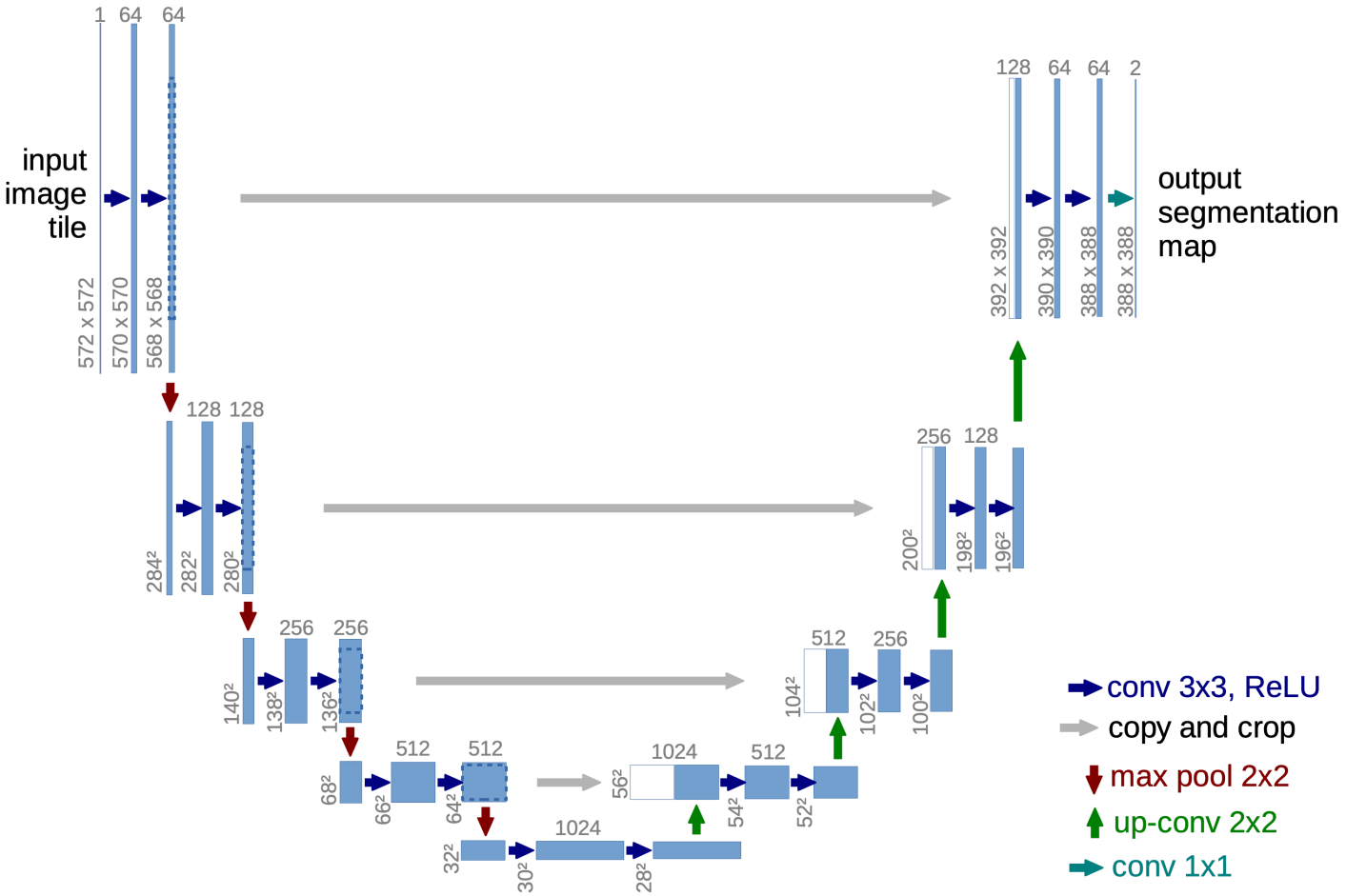
Encoder-Decoder Architectures

- U-Net



Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation

U-Net



Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation

Fast R-CNN Crop the conv feature instead!

- Idea: Pass the image through convnet before cropping!

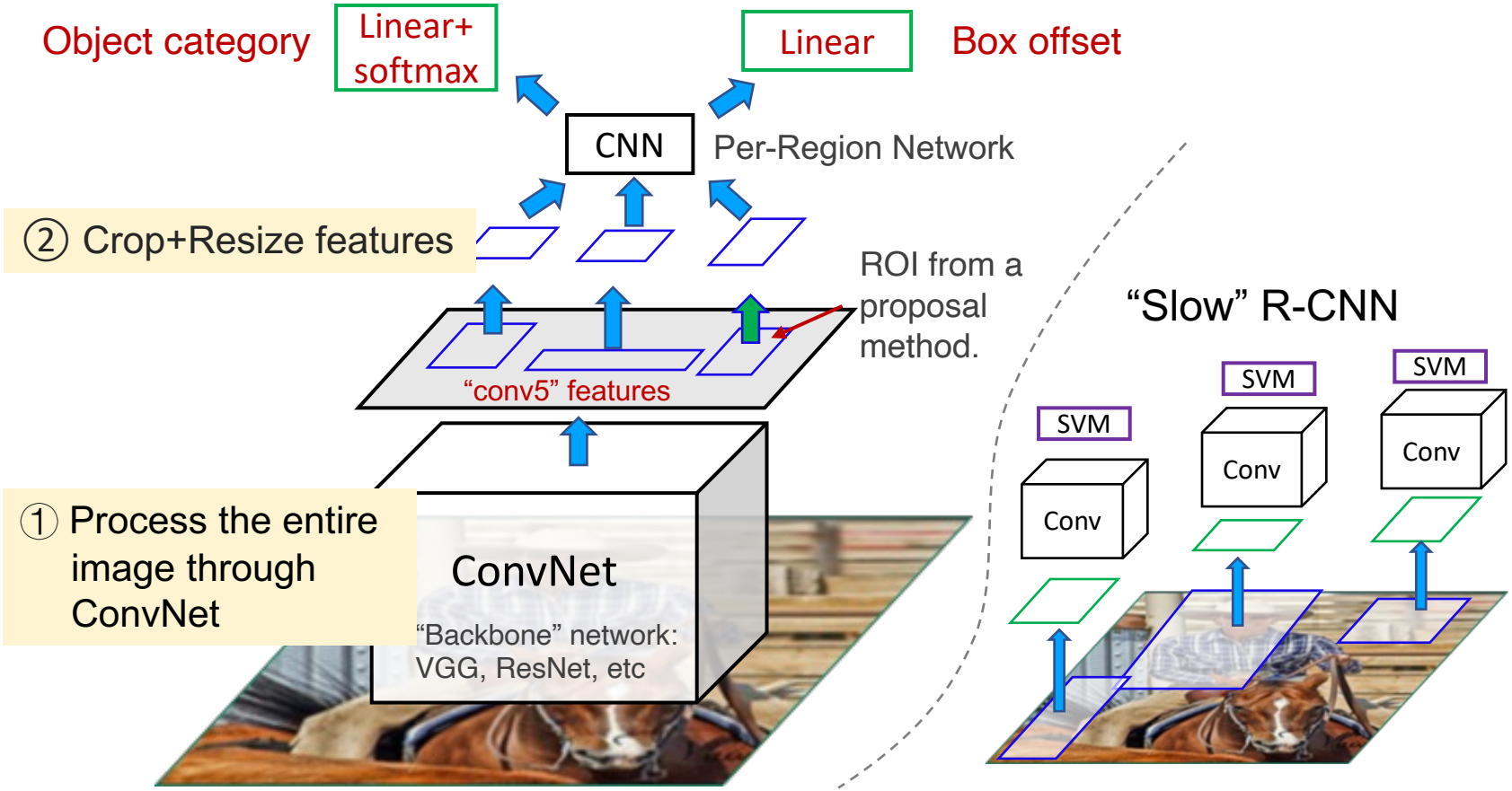
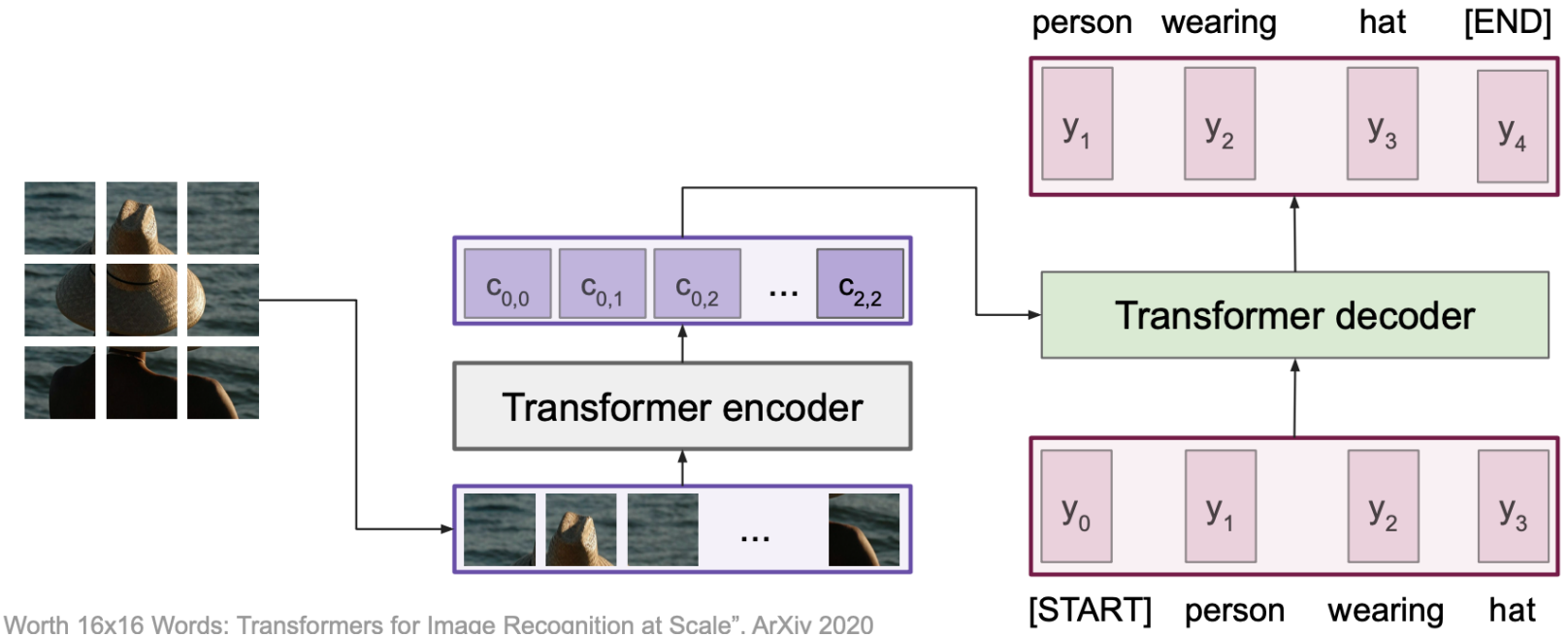


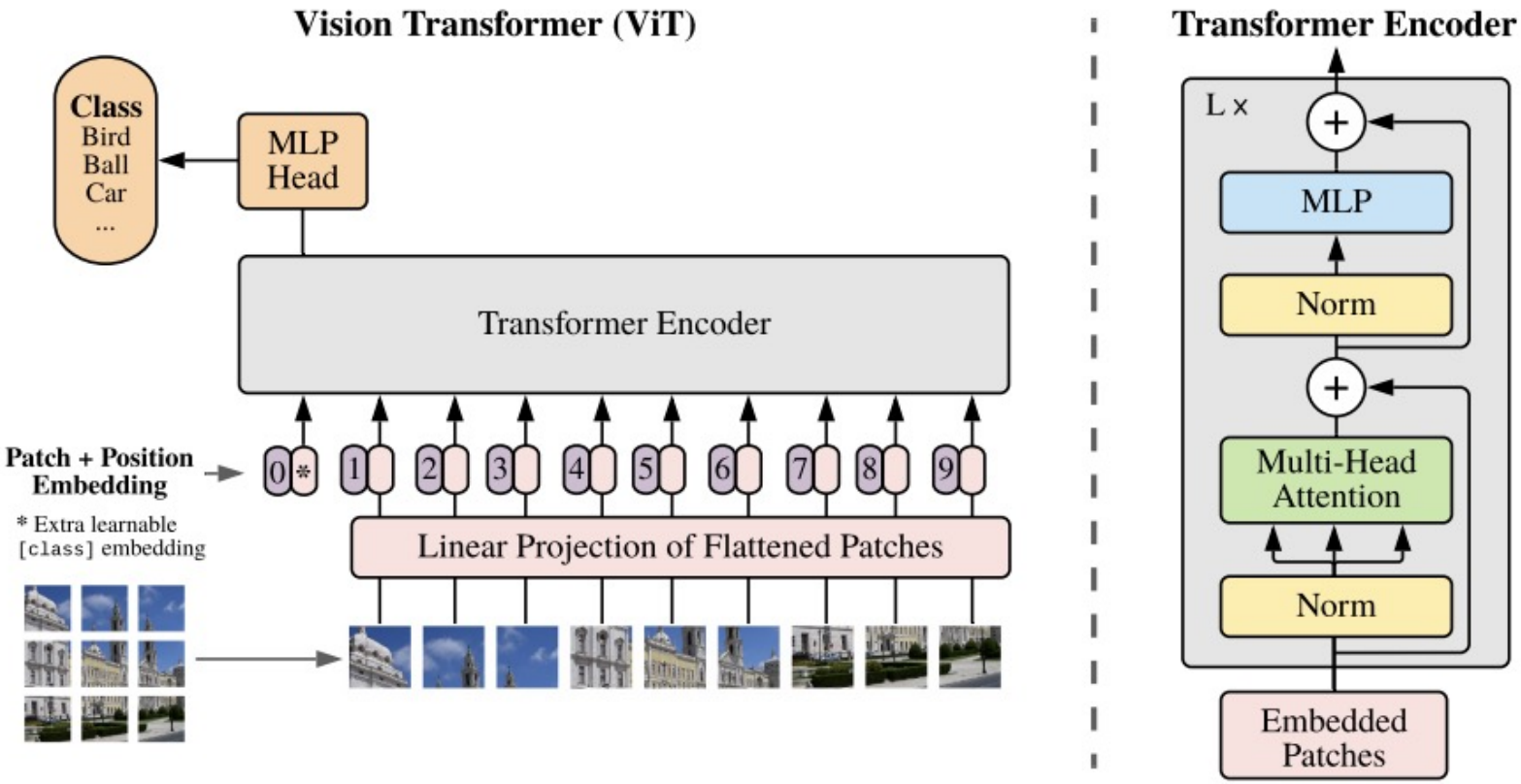
Image Captioning using ONLY Transformers



Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020

Vision Transformer (ViT)



Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020

Unsupervised Learning



Clustering

Learning similarity

- K-means (✓)
- Gaussion Mixture



Generative

Density estimation

- GAN
- VAE
- Diffusion



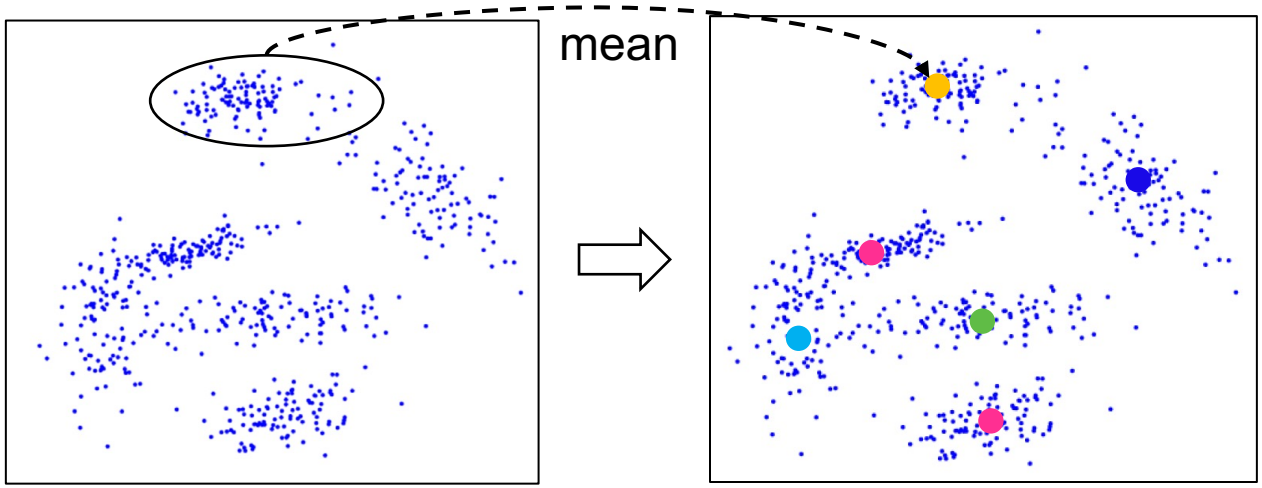
Dimensional Reduction

Data compression

- Principal component analysis
- Auto-encoder

K-Means [MacQueen, 1967]

- Partition the data into ***k*** clusters, where the **center** for each cluster is defined as the **mean** of all instances within the cluster, called **centroid**.



Steps

1. Choose k (random) data points (seeds) to be the initial centroids, cluster centers.
2. Assign each data point to the closest centroid.

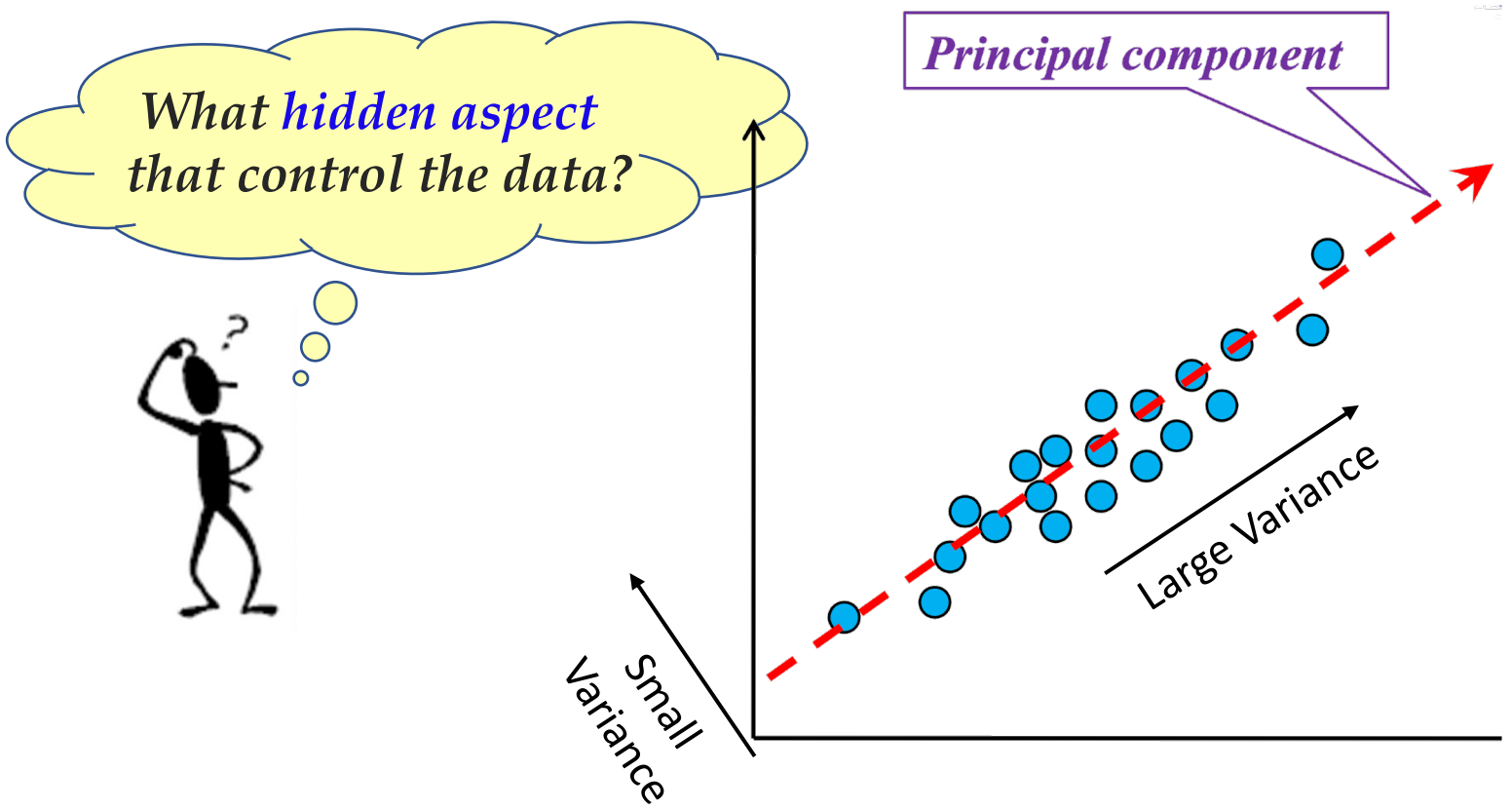
$$L_2(x, \mu^k) = \|x - \mu^k\| = \sqrt{\sum_{m=1}^{d} (x_i - \mu_m^k)^2}$$

3. Re-compute the centroids using the current cluster memberships.

$$\mu^k = \frac{1}{C_k} \sum_{x \in C_k} x$$

4. Repeat steps 2 and 3 until a convergence criterion is met.

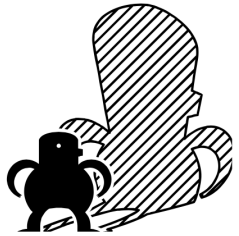
Principal Component Analysis (PCA)



Principal component = the spreading tendency

= dimensionality with the maximum variance

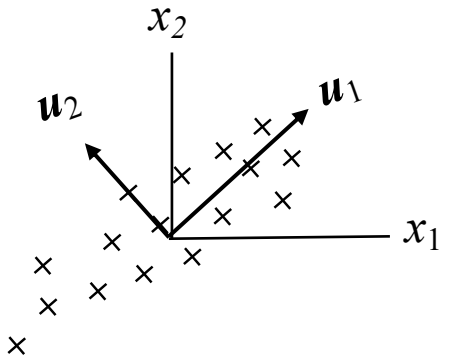
Problem Formulation



Can simply subtract each x_i by the sample mean.

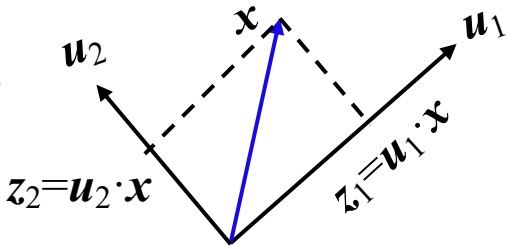
- Data representations

$$\boldsymbol{X} = [x_1, \ldots, x_N],\ x_i \in \mathbb{R}^d, \text{ where } \sum_{i=1}^{N} x_i = 0$$



- Projection of a data point

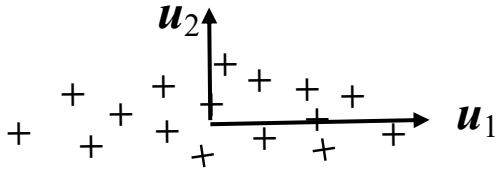
$$z_j = \boldsymbol{u}_j^{\mathrm{T}} x \qquad \boldsymbol{Z} = \boldsymbol{U}^{\mathrm{T}} \boldsymbol{X}$$



- Variance in any projected dimension

$$\begin{aligned} var(\boldsymbol{u}^{\mathrm{T}}\boldsymbol{X}) &= \tfrac{1}{N} (\boldsymbol{u}^{\mathrm{T}}\boldsymbol{X})(\boldsymbol{u}^{\mathrm{T}}\boldsymbol{X})^{\mathrm{T}} \\ &= \tfrac{1}{N} \boldsymbol{u}^{\mathrm{T}}\boldsymbol{X}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{u} \\ &= \boldsymbol{u}^{\mathrm{T}}\boldsymbol{S}\boldsymbol{u} \end{aligned}$$

$$S = \mathrm{cov}(X) = \tfrac{1}{N} XX^{\mathrm{T}}$$



PCA = Solving an Optimization Problem

- In summary, PCA aims to solve a constrained optimization problem:

$$\max_{\mathbf{u}} \ \boldsymbol{u}^{\mathrm{T}}\boldsymbol{S}\boldsymbol{u}$$

$$\text{s.t.} \ \ \boldsymbol{u}^{\mathrm{T}}\boldsymbol{u} = \mathbf{1}$$

- Lagrangian: $\mathcal{L} = \boldsymbol{u}^{\mathrm{T}}\boldsymbol{S}\boldsymbol{u} - \lambda(\boldsymbol{u}^{\mathrm{T}}\boldsymbol{u} - 1)$

$$\nabla_{\boldsymbol{u}}\mathcal{L} = 0 \ \Rightarrow \boldsymbol{S}\boldsymbol{u} = \lambda\boldsymbol{u}$$

- Solving PCA subjects to **eigen decomposition**:

  $\boldsymbol{u}$: eigen vector, λ: eigen value

- Since $\boldsymbol{S}$ is positive semi-definite, we have

$$\boldsymbol{S} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{U}^{\mathrm{T}} = [\boldsymbol{u}_1,\ldots,\boldsymbol{u}_{\mathrm{d}}]\mathrm{diag}(\lambda_1, \lambda_2,\ldots,\lambda_{\mathrm{d}})[\boldsymbol{u}_1,\ldots,\boldsymbol{u}_{\mathrm{d}}]^{\mathrm{T}}$$

The Idea

- Find k orthogonal principal components

  correspond to the k eigen vectors with k **largest** eigen values

$$S = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{U}^{T} \approx \boldsymbol{U}_{1:k}\boldsymbol{\Sigma}_{1:k}\boldsymbol{U}^{T}_{1:k}$$

- Project each input vector x into this subspace

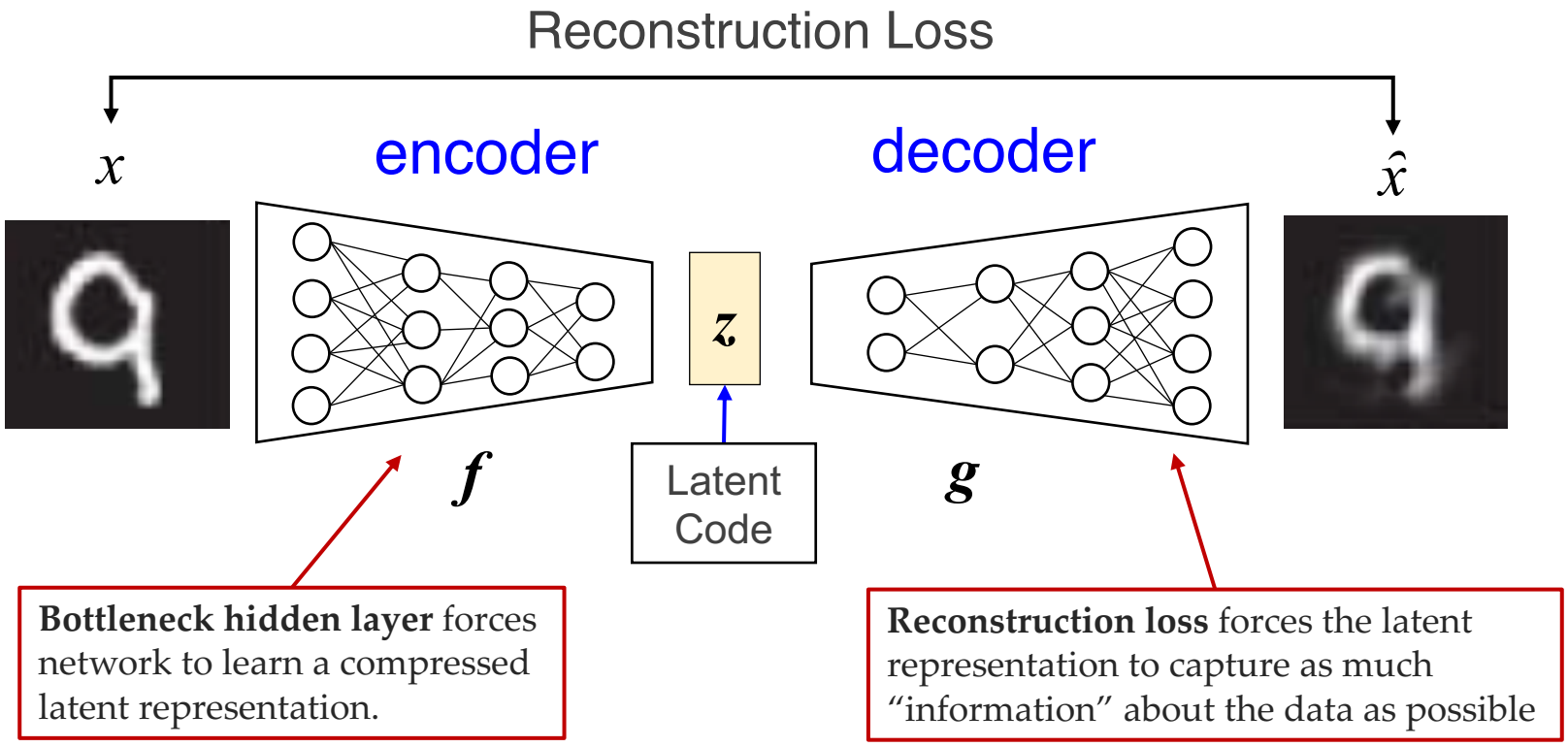
$$z_j = \boldsymbol{u}_j^{T}x \qquad \boldsymbol{z} = \boldsymbol{U}_{1:k}^{T}\boldsymbol{x}$$

Intuitions:

- *Eigen value is the variance of the data projected to the corresponding eigen vector*
- *Eigen vectors are essentially orthogonal*

Deep Auto-encoder

- Of course, the auto-encoder can be deep



Hinton, Geoffrey E., and Ruslan R. Salakhutdinov. "Reducing the dimensionality of data with neural networks." *Science* 313.5786 (2006): 504-507

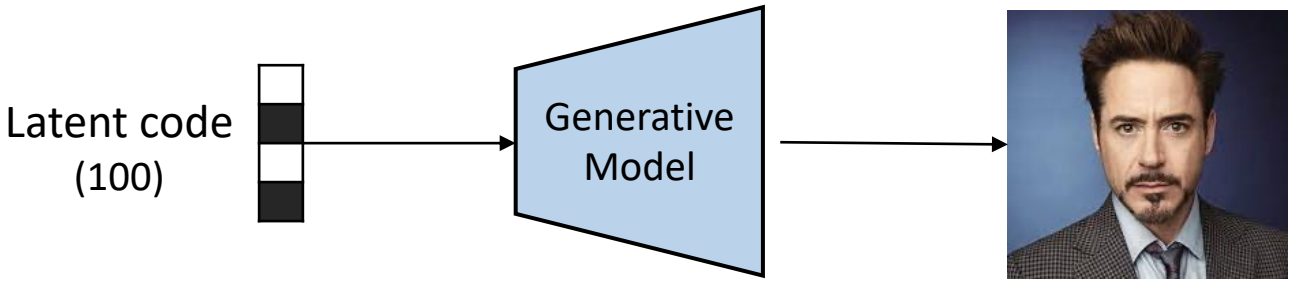
Generation Model

- **Language Model**: a probabilistic model of how likely a given string appear in a given "language".

P_1=P("我爱机器学习")
P_2=P("我爱学习机器")
P_3=P("机器我爱学习")
P_4=P("爱我机学习器")

$$p(x) = p(w_1, w_2, \ldots, w_N)$$

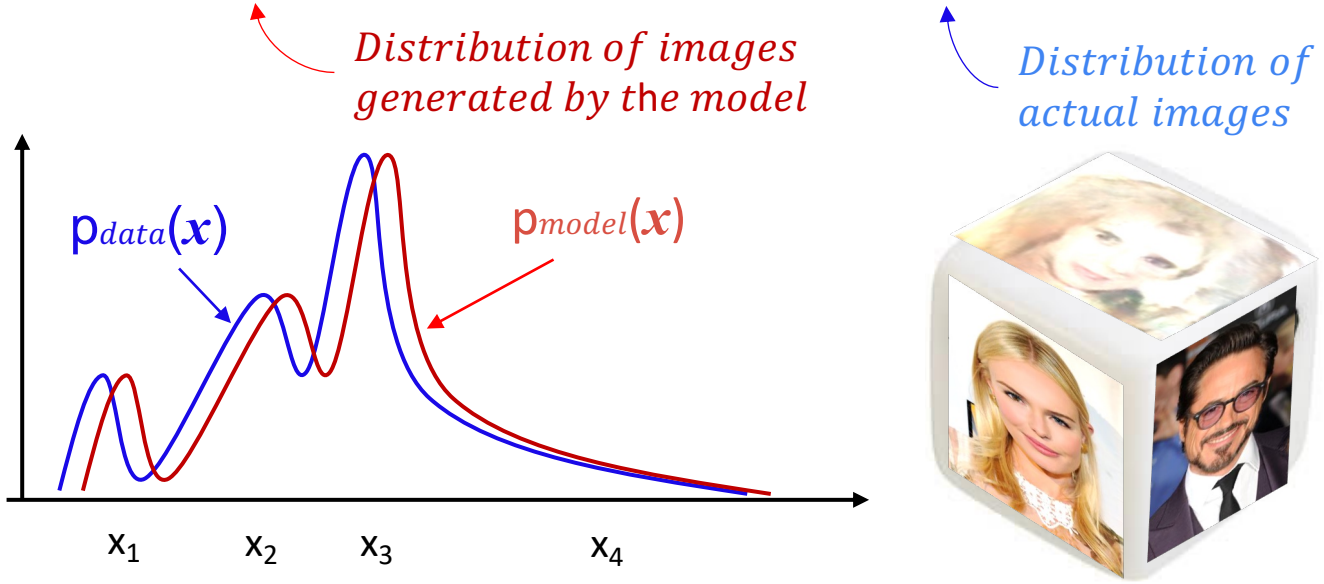
Chinese: $P_1 > P_2 > P_3 > P_4$



Generation = learns the probability distribution of training data.

Generative Model

- **Goal**: find a $p_{model}(x)$ that approximates $p_{data}(x)$ well.

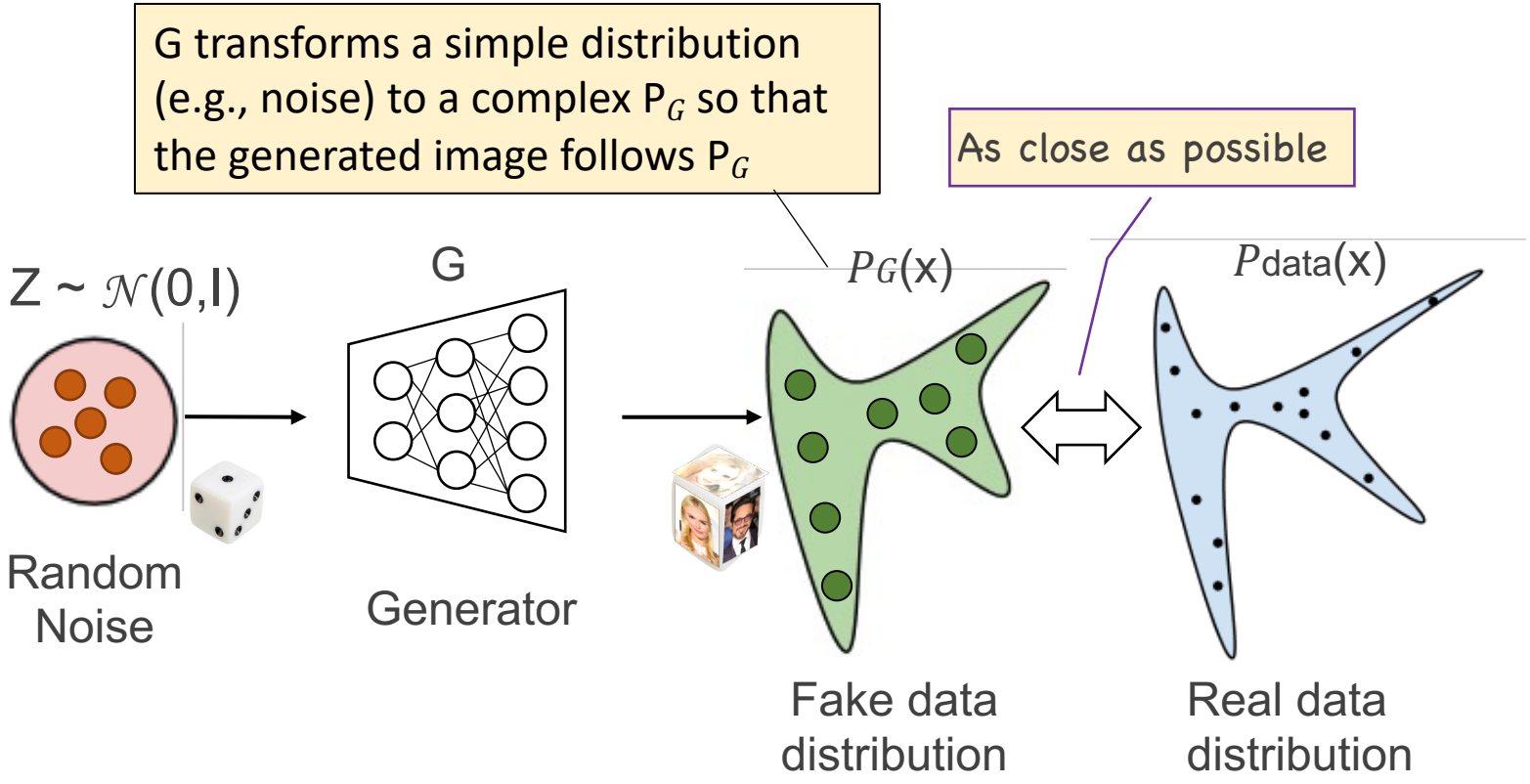


Q1: How to generate an image that follows a distribution $P_{model}(x)$?
In other words, how to represent $P_{model}(x)$?

Q2: How can we learn a $P_{model}(x)$ that is close to $P_{data}(x)$?

How to represent $p_{model}(x)$?

- **Generator G:** a neural network that generates an image x with a probability $\boldsymbol{p_{model}(x)}$ given a random code.



Objective of Discriminator

Recall: binary cross-entropy loss:
$l(h(x), y) = -y \log h(x) - (1-y)\log(1-h(x))$

- The objective function can be formulated as the **binary cross-entropy**:

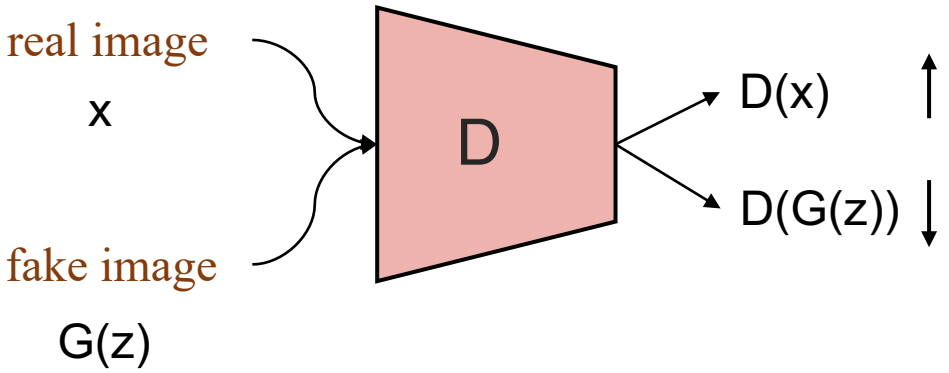
Sample x from real data distribution

Sample x from model distribution

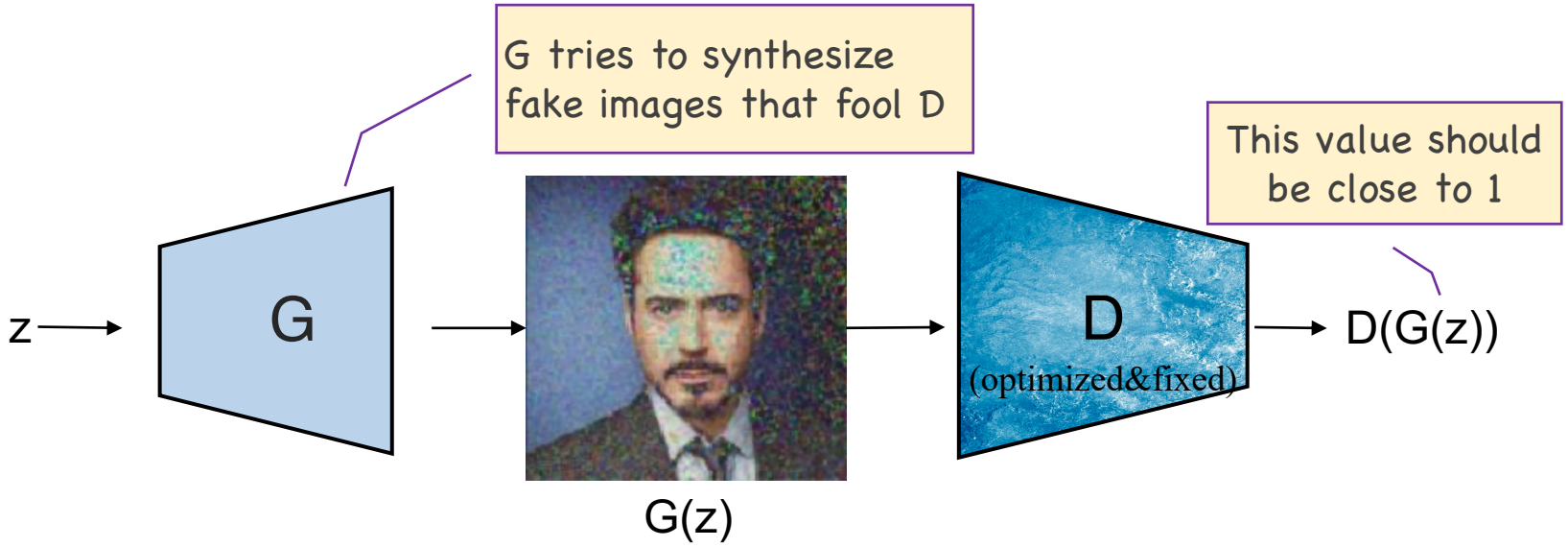
$$D^* = \arg\max_D E_{x\sim P_{data}}[\log D(x)] + E_{x\sim P_G}[\log(1-D(x)]$$

$$= \arg\max_D E_{x\sim P_{data}}[\log D(x)] + E_{z\sim P_z(z)}[\log(1-D(G(z))]$$

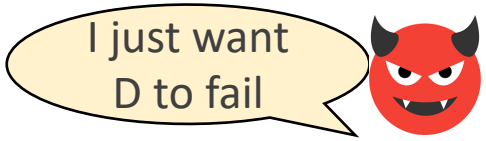
Sample z from unit Gaussian



Objective of Generator



G has an opposite (adversarial) objective to D !

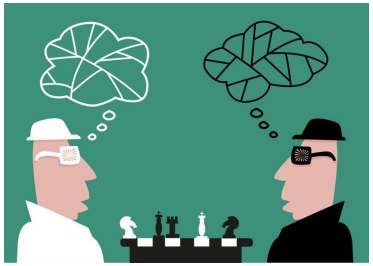


G is independent of this part

$$G^* = \text{argmin}_G \; \mathbf{E}_{x \sim P_{data}}[\log D(x)] + \mathbf{E}_{z \sim P_z(z)}[\log(1-D(G(z)))]$$

$$= \text{argmin}_G \; \mathbf{E}_{z \sim P_z(z)}[\log(1-D(G(z)))]$$

Objective of GAN



Adversarial Objectives for G and D

$$\min_G \max_D V(G,D) = E_{x\sim P_{data}}[\log D(x)] + E_{z\sim P_z(z)}[\log(1-D(G(z))]$$

D: By maximizing V(G, D), I can distinguish real and fake, I am getting stronger 😏

D

G: No matter how strong you are, I can minimize V(G,D) to fool you. I am also stronger ✌️.

G

D: Though you are strong, I can be much stronger by maximizing V(G, D) . 🤗

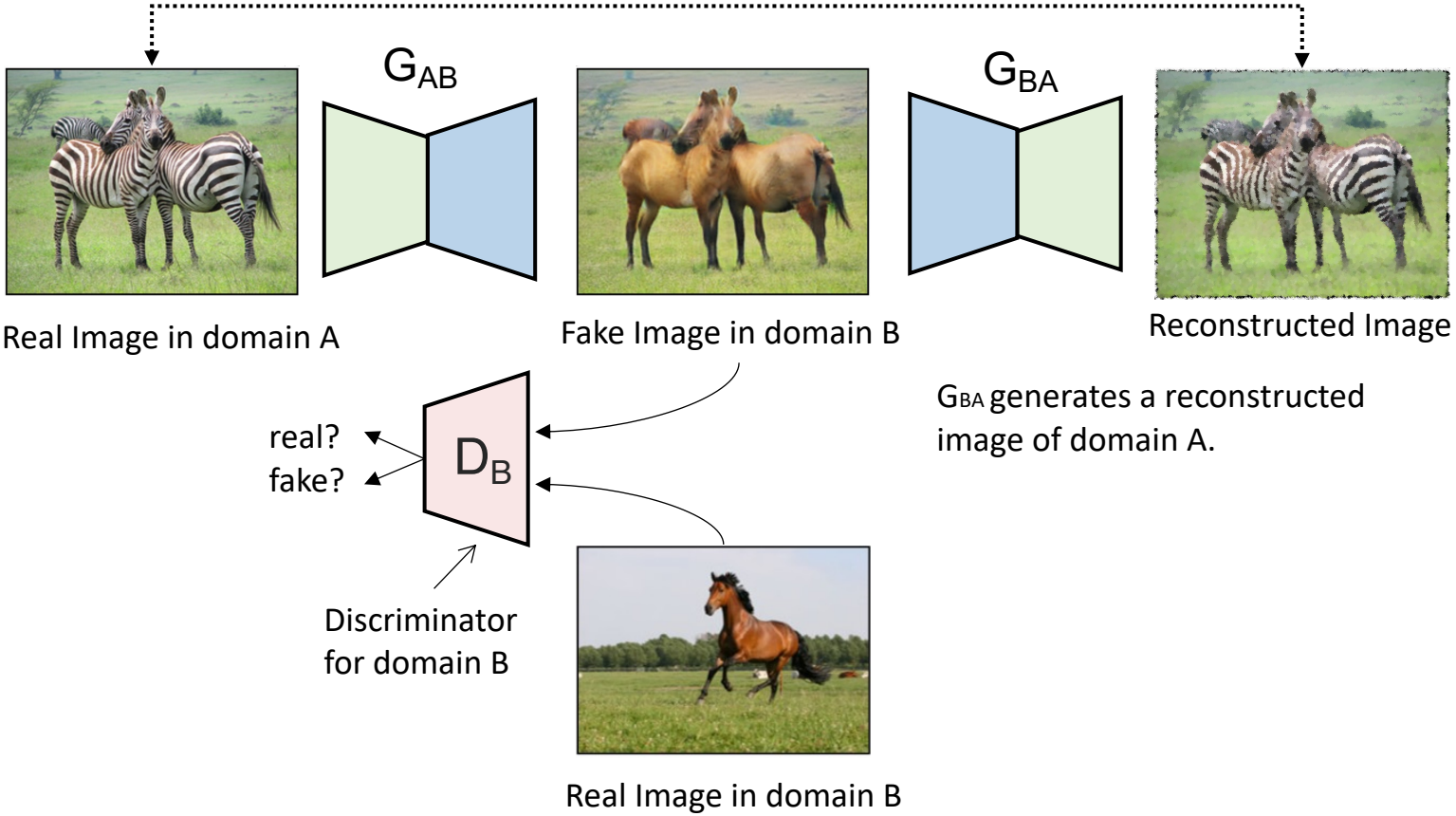
⋮

Global Optimum: G reproduces the true data distribution

CycleGAN

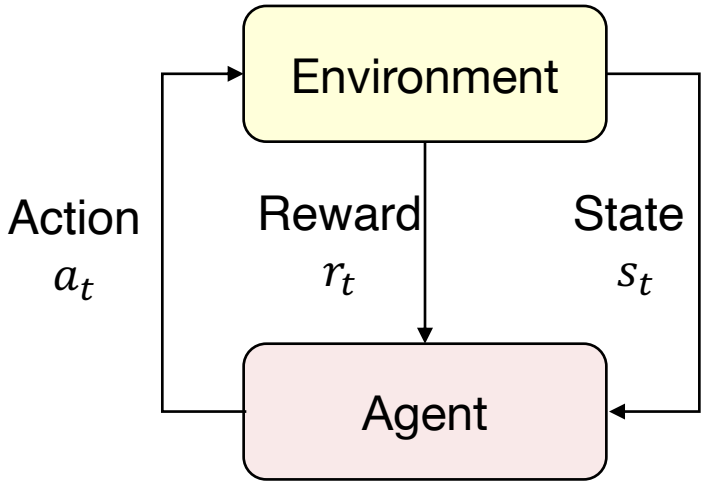
How does it work?

This makes the shape to be maintained when G_{AB} generates a horse image from the zebra.



Reinforcement Learning Problem

- Learning from **interacting** with an **environment.**

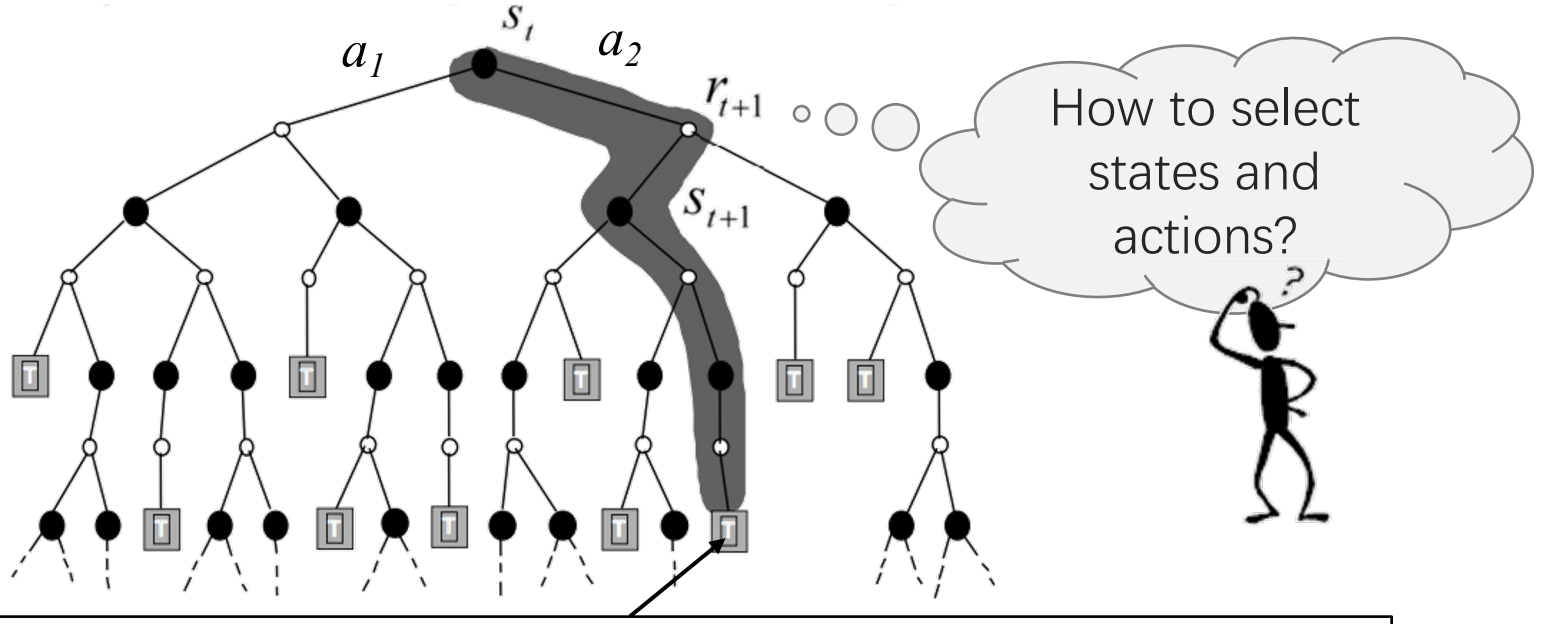


- a set of environment states S;
- a set of actions A;
- rules of transitioning between states;
- rules that determine the scalar immediate reward of a transition;
- rules that describe what the agent observes.

Objective: learning a policy (i.e., state-action mapping) that maximizes the long-term payoff.

In the View of State Space

- The agent finds a path in the state space that potentially leads to the maximum long-term return R_t.



Actual long-term return following s_t: $\boldsymbol{R}_t = \sum_{i=t}^{\infty} r_i = r_t + r_{t+1} + \ldots$

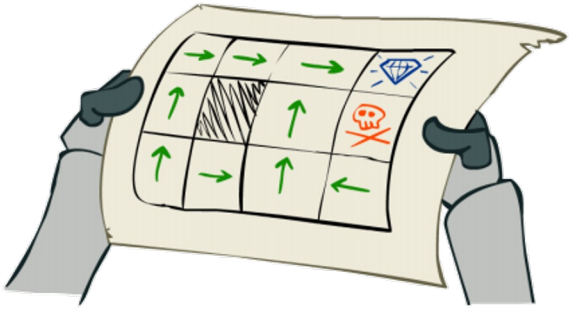
Discounted long-term return: $\boldsymbol{R}_t = \sum_{i=t}^{\infty} \gamma^i r_i = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots$

Scoring (s, a) pairs: Q-function

- The Q-function captures the expected total future reward an agent in state s can receive by executing a certain action a

$$Q(s_t, a_t) = \mathbb{E}[R_t \mid s_t, a_t]$$

score(state, action)



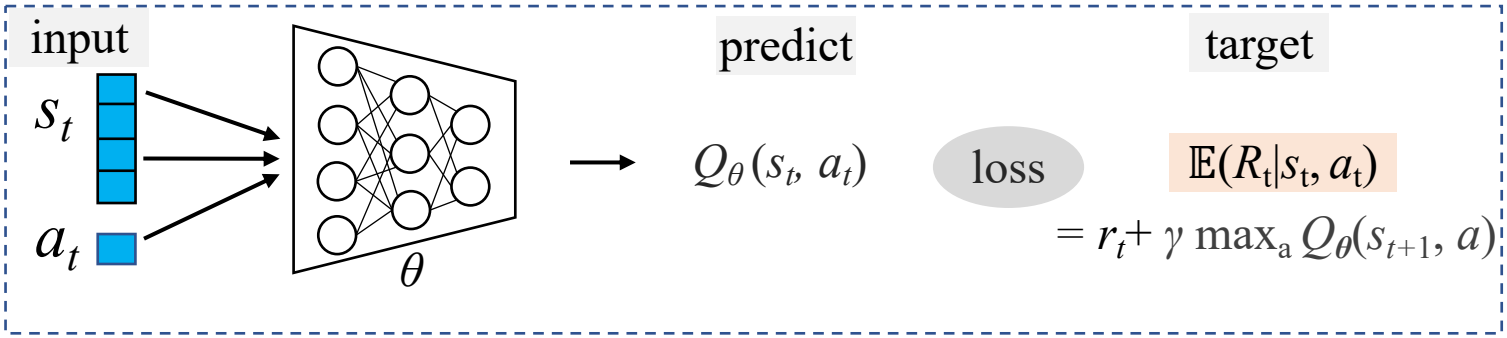
$$R_t = \sum_{i=t}^{\infty} \gamma^i r_i = r_t + \gamma r_{t+1} \ldots + \gamma^n r_{t+n} + \ldots$$

$$\begin{aligned} Q(s_t, a_t) &= \mathbb{E}[R_t \mid s_t, a_t] \\ &= \mathbb{E}[r_t + \gamma r_{t+1} \ldots + \gamma^n r_{t+n} + \ldots] \\ &= r_t + \gamma\, \mathbb{E}(r_{t+1} \ldots + \gamma^{n-1} r_{t+n} + \ldots)] \\ &= r_t + \gamma\, Q(s_{t+1}, a_{t+1}) \end{aligned}$$

Recursion

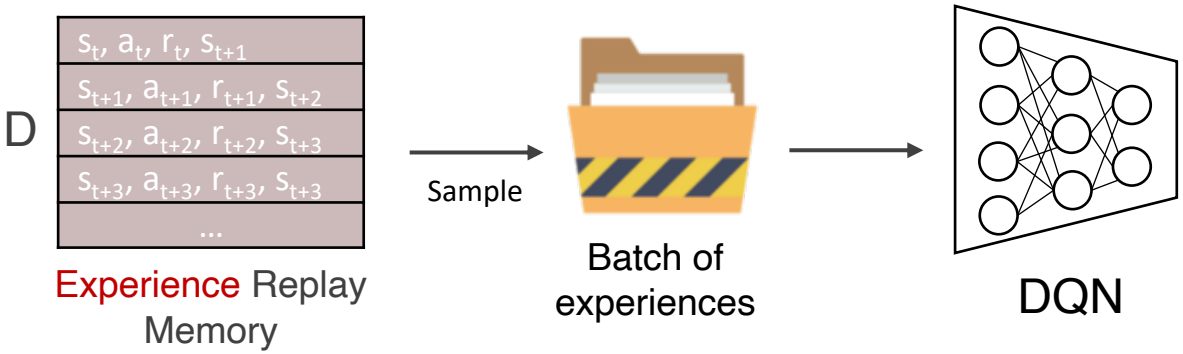
Deep Q Networks (DQN)

Use deep neural networks (e.g., CNN) to model the Q-function.



Data and Loss Function

- Experience Replay: run agent multiple episodes and build a dataset using agent's own experiences: $D = \{(s^{(1)}, a^{(1)}, r^{(1)}, s'^{(1)}), \ldots, (s^{(N)}, a^{(N)}, r^{(N)}, s'^{(N)})\}$



- For multiple $(s, a, r, s') \in D$:

$$L(\boldsymbol{\theta} \mid D) = \mathbb{E}_{(s,a,r,s')\sim D}[\| (r + \gamma \max_{a'} Q_{\theta}(s',a') - Q_{\theta}(s, a) \|^2]$$

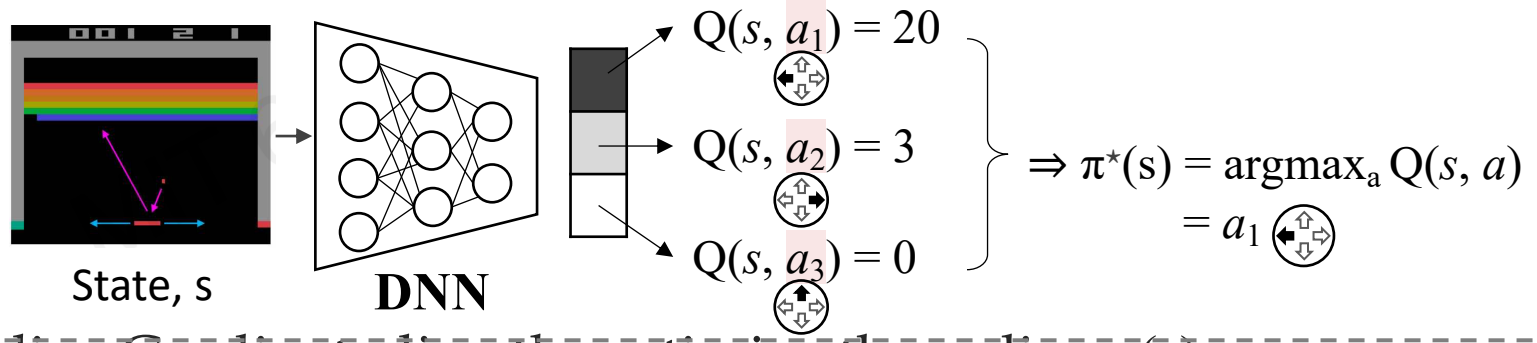
mini-batch experience replay

Maximum possible Q-value for the next state (=Q_target)

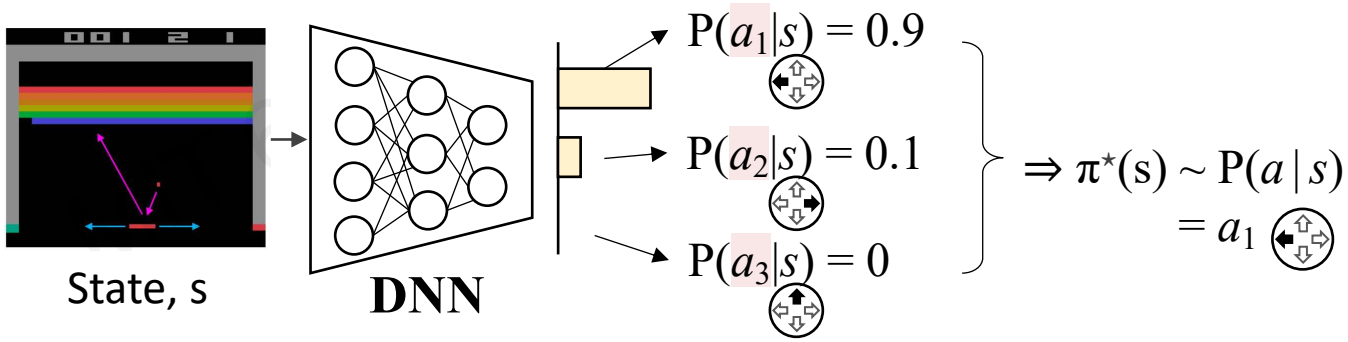
Current predicted Q-value

Policy Gradient: Key Idea

DQN: Approximate a Q-function and use it to infer the optimal policy $\pi^{\star}(s)$

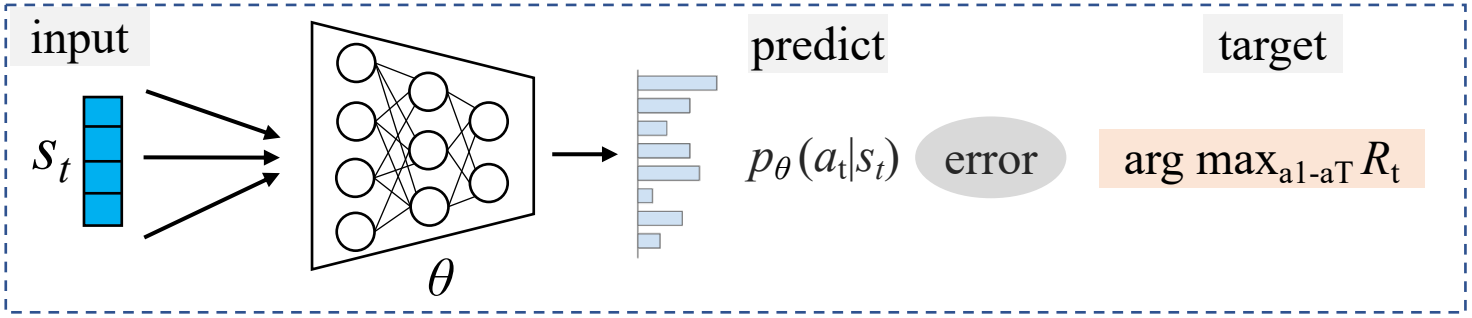


Policy Gradient: directly optimize the policy $\pi(s)$



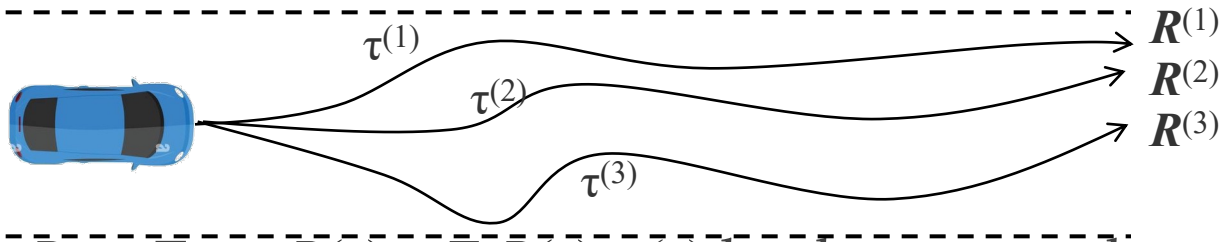
Policy Network

- Use deep neural networks to model the policy π_θ.



Data and Learning Objective

- Run the agent for multiple episodes under a policy π_θ, and sample multiple trajectories $\tau^{(1)}, \tau^{(2)}, \ldots, \tau^{(N)} \sim p_\theta(a|s)$



- Let $R_\theta = \mathbb{E}_{\tau\sim p(\tau)} R(\tau) = \Sigma_\tau R(\tau) p_\theta(\tau)$ be the expected reward from the environment following the policy, our goal is to find θ that maximize R_θ.

$$\nabla_\theta R_\theta = \Sigma_\tau R(\tau) \nabla p_\theta(\tau) = \sum_\tau R(\tau)\, p_\theta(\tau) \frac{\nabla p_\theta(\tau)}{p_\theta(\tau)}$$

$$= \sum_\tau R(\tau)\, p_\theta(\tau) \nabla \log p_\theta(\tau) = \mathbb{E}_{\tau\sim p(\tau)}[R(\tau) \nabla \log p_\theta(\tau)]$$

$$\approx \frac{1}{N} \sum_{l=1}^{N} R^{(l)} \nabla \log p_\theta(\tau^{(l)}) = \frac{1}{N} \sum_{l=1}^{N} \sum_{t=1}^{T} R^{(l)} \nabla \log p_\theta(a_t^{(l)} | s_t^{(l)})$$